

Verified Sequence MRDTs: Embedded Chains, Compaction, and Runtime

Sal / FP Launchpad (KC + Claude)

July 20, 2026

Abstract

We develop and evaluate a verified family of sequence MRDTs based on immutable birth chains. The construction yields a tombstone-free replicated list, entropy-coded coordinates, sided variants encompassing Fugue and FugueMax, and a mechanized maximal-non-interleaving result. Its storage layer includes settled-cut recoding, concrete compaction, run tables, rich-text mark garbage collection, and continuation-aware multi-epoch correctness. Independent sequential specifications distinguish convergence from user-visible intent, while continuation arguments give representation lower bounds. A content-addressed peer-to-peer runtime, a prefix-sharing implementation with direct guarded collection, and reproducible cross-system measurements connect the verified model to the shipped implementation.

Contents

Metatheory interface used by this paper	3
I The embedded-chain family	4
1 The datatype: sort keys are birth chains	4
2 The representation: chains as entropy-coded coordinates	8
3 The insert prefix: for the proof, and the proof alone	18
4 Validation	19
5 The sided generalization: two-sidedness as a parameter	20
6 Maximal non-interleaving, closed: the traversal theory and Theorem 9	25
6.1 Where the family sits in the published table	30
7 The chain price: what delete must remember	30

8	The storage layer: re-coding, compaction, and the run table	33
8.1	Re-coding at a settled cut: the theorem cluster	33
8.2	The verified concrete compaction, and what real traces say	35
8.3	The run table: the representation lever	43
8.4	The sided run table: the same lever under Fugue	49
8.5	From projection to shipped bytes, and the cross-system question	51
9	The marks layer: document-order rich text, and its GC	52
9.1	The document-order mark read model	52
9.2	Why plain-text compaction is unsound for marks	55
9.3	Two attacks, falsifiable forms first	55
9.4	The growth window: the guarded root-freeing drop	57
9.5	The mechanization	58
9.6	The runtime: refuse becomes fire-with-certificate	61
10	Comparison with Eg-walker	62
11	Related work	64
12	Mechanization: the conditioned discharge and the compaction theorem	65
II	Sequential specifications: convergence is not correctness	70
13	The specification gap	70
14	The obligation, and the rules of engagement	71
15	The observation gap: positional interfaces, witnessed ops	72
16	The specifications, datatype by datatype	73
16.1	No gap: the alphabets already coincide	73
16.2	Gap: arbitration by stamp instead of program order	73
16.3	Gap: overwrite by snapshot instead of “everything”	73
16.4	Gap: dequeue() becomes delete-what-I-saw	75
16.5	Gap: insert-at-index becomes insert-after-who-I-saw	75
16.6	Gap: one mark becomes a boundary pair	76
17	The proofs: four tiers, three kinds of invariant	77
18	The refutations	79
19	The two-axis separation	81
20	What composition does not give	82

21 The verified matrix	83
III The verified peer-to-peer runtime	85
22 The runtime: one content-addressed replica	85
23 Performance	87
24 Push-button discharge: an SMT translation-validation layer	91
IV Audit and mechanization scope	93
25 Mechanization scope	93

Metatheory interface used by this paper

An MRDT state is indexed by a causal event history and merges two divergent states relative to a common-ancestor state. We use five results from the companion metatheory: canonical histories determine reference states; contextual ternary Join adequacy makes reachable merges agree with those states; honest reachability carries datatype-specific applicability facts; observational equality permits representation changes; and a stability epoch simulates settled-cut compaction together with all admitted continuations. The companion paper also proves that asynchronous fetch, head-sync, local GC, and fetch acknowledgements refine the no-GC semantics; this paper uses that result when connecting the sequence implementation to the runtime. This paper discharges those interfaces for embedded-chain sequences and then proves the independent intent and representation results that the generic convergence theorem does not imply.

Part I

The embedded-chain family

The embedded-chain RGA (replicated growable array) is a replicated list that keeps no tombstones, no ledger, and no event log: its entire state is the live nodes. It is a sequence MRDT in Part I’s sense: state-based, reconciled by a three-way merge against the branches’ lowest common ancestor version. A deleted element survives only as $\Theta(\log \delta)$ bits inside its descendants’ sort keys, δ being the Lamport-timestamp gap between the element and its own insertion anchor (1 under continuation typing); that is the log of the event gap the insertion crossed, a race or a cursor jump, paid once, in space. Concurrent inserts provably cannot tie, so the merge contains no repair and never decides an order; the state is a canonical function of the event set, and convergence is literal equality. Observationally the datatype *is* the published RGA with the tombstone set compressed into coordinate entropy: a kernel-checked read-equivalence theorem, with machine lockstep at every step as its tested form. All of this follows from one idea: an element’s sort key is its *birth chain*, the timestamps from the root of the birth tree down to the element, dead ancestors included. The chain is written at birth, never edited, and carried in an order-preserving self-delimiting code. Separating the datatype from its encoding makes the metatheory nearly definitional: correctness is three one-line facts about chains, plus a single faithfulness theorem about the code. The design is machine-validated (the L1–L25 anomaly battery, except L19, RGA’s own, discharged by the sided generalization of §5; 420+ randomized version-DAG executions; 120/120 lockstep), and the central theorem, RA-linearizability at every honestly reachable configuration, is a kernel-checked Lean theorem (§12) discharged by the direct-join route of §. (Working names in the validation artifact: `embed-tree` for the unary code, `embed-code` for the delta-coded one.)

1 The datatype: sort keys are birth chains

Timestamps $t \in \mathbb{N}^+$ are Lamport stamps, globally unique, and double as element identities. Causality gives: the anchor of every insert carries a smaller timestamp than the insert itself.

Convention (Part II plumbing). Four pieces of plumbing are folded out of every Part II display below, once and for all; Part I’s convention (§) remains in force. (i) *The code:* Γ ranges over ordered prefix codes (Definition 2.1), and every display is parametric in Γ unless a concrete instance (`unaryCode`, `binaryCode`, `eliasDeltaCode`) is named. Chains in displays are *positive* (every $\delta \geq 1$, the shape Lamport causality forces); displays write “positive chain” and elide nothing else, the mechanized statements carrying the positivity as explicit `PosChain/PosSChain` antecedents. (ii) *The comparator:* `key` maps a coordinate’s bits into symbols ($0 \mapsto 1$, $1 \mapsto 2$) and appends the terminator 3; `keyLt` is strict first-difference comparison of symbol strings; the sided `sKey` (§5) appends the same terminator to symbols already banded $\{1, 2\} < 3 < \{4, 5\}$. Displays write “the display comparator” for this apparatus and before for the induced strict display order on live ids (`before`: both live, keys descending). (iii) *Axiom provenance:* displayed capstones are kernel-checked on `{propext, Classical.choice, Quot.sound}`; displayed countermodels evaluate concrete executions by `native_decide` under the document’s SPOT convention (adding the compiler axioms), exactly like unit tests. Per-display axiom notes appear only where a display departs from this split. (iv) *The generation layer* (§6 only): the Fugue-family ordering theorems quantify over per-replica lists of generation records (`Know`, `KnowM`) under the reachability

relations **FugueReach**, **MaxReach**, declared once there; the record-reading functions (left/right origin, display order, liveness) are named there and never re-derived per display. (v) *The at-hand domain* (the storage layer, §8.1 onward): for a stable-prefix re-map bundle F (Definition 8.1) with at-rest domain Rest and declared beyond-cut mints MintAt , a coordinate c is *at hand* ($F.\text{Dom } c$) when $\text{Rest } c$ holds or $c = \pi \cdot \Gamma.\text{enc}(d)$ for some declared mint $\text{MintAt } \pi d$. The storage displays' recurring hypothesis pair, “ s at hand, τ declared at F ”, abbreviates exactly: every record coordinate of the cut state s satisfies $F.\text{Dom}$, and every insert $(t, r, \text{ins}(e, \pi, a)) \in \tau$ satisfies $F.\text{MintAt } \pi (t - a)$; both quantifiers appear explicitly, in this form, in every mechanized statement. Timestamps throughout Part II are plain Lamport stamps in $\mathbb{N}$.

The birth tree. Every insert names an anchor. The event set therefore induces a tree: the root is the front sentinel 0; the *birth parent* of each element is its anchor. The tree is append-only and never edited: an element's place in it is fixed at birth.

Birth chains. The *birth chain* of u is the sequence of timestamps from the root down to u (sentinel elided): $\text{chain}(u) = \text{chain}(a) \cdot u$ when u was inserted after anchor a , and $\text{chain}(u) = \langle u \rangle$ at the front. Dead ancestors are *not* elided: their timestamps remain in their descendants' chains. The chain is the sort key, and everything in this section is stated in terms of it; nothing here depends on how chains are stored (§2).

Type. A state is a finite map over *live nodes only*:

$$\sigma : u \mapsto (\text{chain}(u), \text{char}_u).$$

init. $\sigma_0 = \emptyset$.

do (the operations). Two operations, with $\text{rc} = \text{Either}$ everywhere (§3):

- **ins**($x \leftarrow a, c, \pi$): insert element x (its timestamp) with character c after anchor a ($a = 0$ for the front). The payload $\pi = \text{chain}(a)$ is the anchor's birth chain, carried *for the proof alone* (§3). Application on a state where a is live (every reachable application, by honesty):

$$\sigma[x] := (\text{chain}_\sigma(a) \cdot x, c), \quad \pi \text{ unread.}$$

No head-tracking, no reading of siblings.

- **del**(d): remove d 's record. Nothing else changes: survivors' chains are untouched; in particular, d 's descendants still carry d 's timestamp in their sort keys. If d is absent, no-op (concurrent double delete).

read. Sort survivors by *chain-lex*: compare chains timestamp-by-timestamp with *larger first*; a proper prefix precedes its extensions (a live ancestor displays immediately before its descendants). Among same-parent siblings this is exactly RGA's newest-first rule; equivalently, the canonical order is the RGA traversal of the birth tree restricted to survivors.

Definition 1.1 (the chain display order). *In the mechanization a chain is its list of deltas, each entry the timestamp gap to its parent, partial sums recovering the timestamps; at a first divergence the two entries share an anchor, so the larger delta and the larger timestamp are the same verdict. chainBefore is the least relation closed under two rules:*

$$\text{chainBefore } ch \ (ch \cdot ext) \ (ext \neq \langle \rangle), \quad \text{chainBefore } (p \cdot \langle d \rangle \cdot c_1) \ (p \cdot \langle e \rangle \cdot c_2) \ (e < d) :$$

*an ancestor precedes everything in its subtree, and at the first divergence the larger delta (the newer sibling) precedes. Distinct chains are always comparable (**chainBefore_total**). The read above is: sort survivors by chainBefore on their birth chains; every ordering theorem of this part is stated against this relation.* **chainBefore**

merge. $\text{merge}(L, A, B)$, where L is the LCA (the branches' lowest common ancestor version, the merge context the MRDT model stores and supplies). The merge is *survival* (OR-set): live iff in all three, or born in exactly one branch. There is no second step at this layer: survivors' chains come with their insert events, so every input that holds a survivor agrees on its chain. The merge never compares timestamps and never decides an order.

Inv / applicable / honesty. The honesty contract is the standard RGA generation discipline: an insert is *generated* at a replica where its anchor is live (you insert after a character you can see), and ins/del are *applied* on states where they are applicable (anchor live; delete of a live or already-dead node). On every reachable state, application never consults π . Canonicity (Theorem 1.3) doubles as Inv; at the representation layer, Theorem 2.5(iii) is its stored form.

$\approx$. Equality. The state is a canonical function of the event set (Theorem 1.3), so no quotient is needed: convergence is literal state equality.

The components above are mechanized twice, one datatype, two carriers; every Part II theorem names one of the two.

Definition 1.2 (the embedded-chain RGA, as mechanized). *The model layer (Sal/MRDTs/RGA_Embed/RGA_Embed_MRDT.lean) stores a finite map $\sigma : t \mapsto (el, c)$ from live ids to (element, absolute coordinate) pairs, coordinates as bit strings:*

$$\text{do}_\Gamma(\sigma, (t, r, \text{ins}(e, \pi, a))) = \sigma[t \mapsto (e, \pi \cdot \Gamma.\text{enc}(t - a))], \quad \text{do}_\Gamma(\sigma, (t, r, \text{del}(x))) = \sigma \setminus x,$$

*the insert writing the mint of its carried prefix (never reading the state; **mint**, **do_**), the delete removing the record (no path, nothing rehomed). The merge is OR-set survival on identities, values read off any input holding the id (immutable, so the choice is immaterial):*

$$\text{dom}(\text{merge}(l, a, b)) = (l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l) \quad (\text{merge}).$$

rc = *Either everywhere (**rc**, §3); observational equality is extensional map equality, equal domains and equal records (**eq**). The instance layer (Sal/ConditionedMRDTs/MRDT_Instances/EmbedRGA/EmbedRGA.lean) is the same datatype on the canonical carrier: the state a list of records (t, el, c) strictly descending by key (**EState**); insert a sorted insertion, idempotent on a present id, delete a filter (**eUpdate**); the ternary merge the same survival formula re-canonicalized by a sorted two-merge (**eMergeL**); the fold of an operation list **eFold**. It is packaged as the conditioned signature of Part I with $\text{Inv} = \text{app} = \top$ and $\approx =$ (honesty lives in the configuration layer, §12), parametric in Γ and in the element payload α .* **do_**, **merge**, **rc**, **E**

Metatheory. At this layer the design’s theorems all but evaporate; we record them because they are the specification the representation must meet (§2).

Theorem 1.3 (chain-level metatheory). *In every reachable state:*

- (i) *Canonicity:* $\sigma = \{ u \mapsto (\text{chain}(u), \text{char}_u) \}$ over survivors u ; the state is a function of the event set.
- (ii) *Birth constancy:* $\text{chain}(u)$ is fixed at u ’s insert and never edited by any later transition.
- (iii) *Totality (no ties):* chain-lex is a total order on every state’s survivors.

Proof. (i) Survivorship is OR-set, hence a function of the event set, and a survivor’s chain is determined by its insert event alone: by induction, `ins` writes $\text{chain}_\sigma(a) \cdot x = \text{chain}(a) \cdot x$ (the anchor is live at application, by honesty, and its stored chain is canonical), while `del` and the merge only delete or select records, never rewrite them. (ii) is (i) read off per node: no transition writes to an existing record. (iii) Distinct elements’ chains are distinct (they end in distinct timestamps); if neither is a prefix of the other, the first difference compares two distinct timestamps in $\mathbb{N}$, and uniqueness of Lamport stamps is the no-tie property; prefix pairs are ancestor/descendant, ordered by the prefix rule. $\square$

Mechanized: (i) is fold canonicity, `e_fold_canon` (§12); (ii) is definitional there (records are written once, then only removed or copied); (iii) is unique decodability plus totality of the chain order (Theorem 2.3, `chainBefore_total`). One more design-layer fact belongs beside them, the litmus g-column (non-interleaving of concurrent runs) as one lemma. It is stated one layer down, over stored coordinates, because that is where it is consumed; by unique decodability a coordinate prefix is exactly a birth-chain prefix.

Theorem 1.4 (subtree convexity, one-sided). *In any state s (no honesty, no reachability): if before s t_1 t_2 and before s t_2 t_3 , and a coordinate c prefixes both stored coordinates $\text{pos}_s(t_1)$ and $\text{pos}_s(t_3)$, then c prefixes $\text{pos}_s(t_2)$. Instantiated at an anchor’s coordinate: everything displayed between two members of a subtree is in the subtree, so runs typed under distinct anchors cannot interleave. The proof is pure lexicographic convexity of prefix sets; no property of the code is consumed.*

subtree_convex

The one-line separation, and the conservation law. The natural first attempt at tombstone-freedom is the *rehoming* RGA: keep flat (element, anchor) records, and let a delete splice its node out, re-parenting the orphans to the nearest live ancestor (the *climb*). We built and verified that design before this one (it is RA-linearizable, in the same Lean framework), and it fails the L1 delete-reorder litmus (the $[b, a, c] \rightarrow [c, b]$ anomaly, Figure 1): the climb re-sorts a rehomed node by its own timestamp, so *its sort key is the birth chain truncated at each rehoming*. This design keeps the chain whole. That is the entire difference between the two datatypes, and it is the crisp form of the conservation law that our design sweep kept re-imposing: *the sort key must retain dead ancestors’ timestamps*. Every design that forgets them has a named machine-checked countermodel (§11). What is negotiable is only how many bits the retained timestamps cost; that is the subject of §2, and the answer is: the log of each element’s anchor gap, and nothing else.

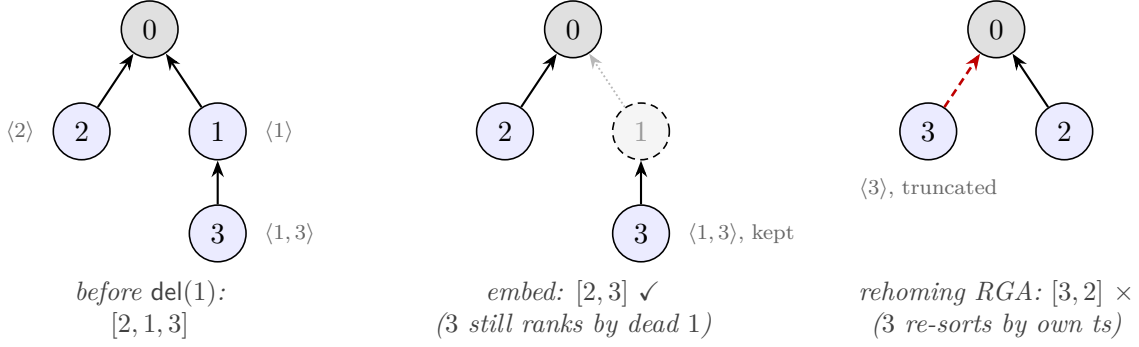


Figure 1: The one-line separation on the L1 delete-reorder litmus: insert 1 and 2 at the front, 3 after 1, delete 1. The rehoming RGA’s climb (red) re-parents 3 and re-sorts it by its *own* timestamp: the truncated key $\langle 3 \rangle$ beats $\langle 2 \rangle$, and 3 jumps over 2. The embedded chain $\langle 1, 3 \rangle$ keeps ranking 3 by its dead parent’s timestamp: $[2, 3]$, the delete-order-preserving read. Arrows point from a node to its anchor; dashed grey = dead.

Worked example (Figure 2). 6 and 10 concurrently insert at the front: chains $\langle 6 \rangle$ and $\langle 10 \rangle$; display $[10, 6]$, no arbitration, at every replica and merge order. Type 22, 16 under 6: chains $\langle 6, 22 \rangle$, $\langle 6, 16 \rangle$. Delete 6: the records of 22 and 16 are untouched and still carry 6’s timestamp. A third party concurrently inserts 8 at the front: $\langle 8 \rangle$ sorts below $\langle 10 \rangle$ and above $\langle 6, 22 \rangle$, so the display is $[10, 8, 22, 16]$ whether 8 arrives before or after 6’s death (machine-checked as the *credential countermodel*, the shape that kills every timestamp-oblivious variant, §4).

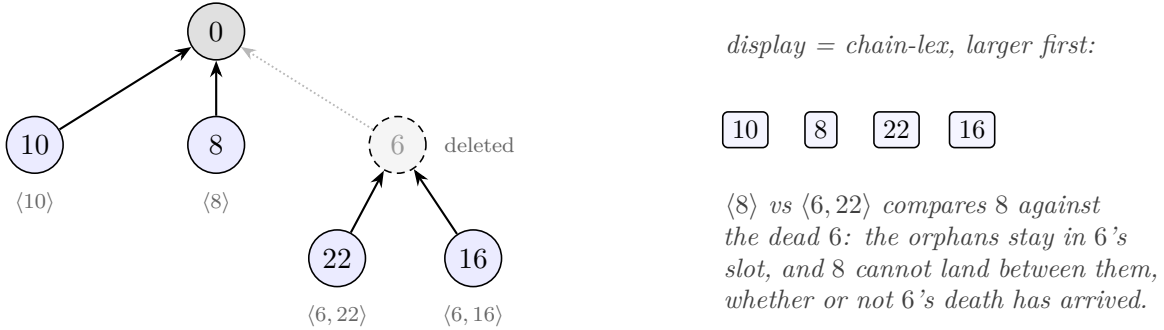


Figure 2: The credential countermodel as a birth tree. 6 and 10 race at the front; 22, 16 are typed under 6; 6 dies; 8 arrives at the front concurrently. Survivors sort by whole birth chains, so the read is $[10, 8, 22, 16]$ at every replica and merge order. Every timestamp-oblivious variant flips this shape (§11).

2 The representation: chains as entropy-coded coordinates

Everything above is the datatype; everything below is bits. A naive carrier would store a full timestamp at every chain level (each $\sim \log(\text{document age})$ bits, even for purely sequential typing) and compare whole chains at every read. This section stores the chains compactly without disturbing their order.

Vocabulary. A *codeword* is the bit string $C(\delta)$ encoding one chain level: one element’s timestamp, stored as its differential δ to its birth anchor (step 1 below). A *coordinate* is the bit string encoding a whole chain: the concatenation, root to element, of one block per level (the level’s codeword plus three framing bits fixed by the mint construction below), with nothing else marking the boundaries. We write $|s|$ for the length in bits of a bit string s . Read as a binary fraction, a coordinate names a dyadic interval in $(0, 1)$; codeword concatenation becomes interval nesting, and the interval is the $\alpha(u)$ of Theorem 2.5. Bit string and interval are two views of one object; we use whichever is clearer.

Purpose of the coding. The coding serves two roles; only one concerns correctness. *Job 1: cost.* The datatype layer already forced sort keys to retain dead ancestors’ timestamps (the conservation law, §1); the only remaining question is how many bits that retention costs. Without entropy coding, “tombstone-free” would be bookkeeping: a full timestamp per level grows with document age however calm the history. The *unary control* (*embed-tree*: the identical datatype with a unary code in the mint, codeword length linear in the value, kept as the experiment’s cost-control arm; step 2 and the entropy-law table) makes the gap measurable: 61 KB against the coded design’s 0.5 KB on the table’s 1000-element chain scenario. The coding is what makes the storage bound true: the dead survive as the entropy of the anchor gaps they were born across, and not more. *Job 2: faithfulness.* The chains must be stored so that plain lexicographic bit comparison reproduces chain-lex exactly, in every reachable state, forever. This is the single obligation the representation owes §1 (Theorem 2.5). Correctness lives upstairs; this layer is an implementation that must not lie. Job 2 needs no entropy optimality at all: it needs exactly two structural properties of the per-level code, stated next.

The contract. The stored form must support four operations: *mint* (insert: produce the new codeword from the two timestamps alone, coordination-free, reading no siblings); *compare* (read: plain lexicographic comparison, with no tie rule); *fold* (delete: splice a dead node out by concatenating its block into each child’s stored string, exactly); and *refold* (merge: recompute survivors’ coordinates by the same concatenation). Each guarantee is bought by a specific property of C :

- **Monotone** ($\delta < \delta' \Rightarrow C(\delta) <_{\text{lex}} C(\delta')$) buys correct *compare*. A chain-lex first difference always compares two same-anchor entries, where timestamp order and δ order coincide (step 1); bitwise comparison of coordinates therefore equals timestamp comparison at the first differing level precisely when C is monotone.
- **Prefix-free** (no codeword is a prefix of another) buys three things at once. *Alignment*: in a comparison of two coordinates, the first differing bit falls inside the first differing level’s codewords, so levels cannot be confused across boundaries and no delimiters are needed; this is also what keeps *fold* and *refold* exact splices. *Geometry*: by the Kraft correspondence, prefix-free codewords occupy pairwise-disjoint dyadic cells, which is Theorem 2.5(i) and the reason a concurrent front-insert can never land inside a dead sibling’s span (Figure 4). *No ties*: cell disjointness plus Lamport uniqueness means sibling coordinates never tie, so the merge never decides an order and the read needs no tiebreak.
- **Universal, with a cheap floor** buys Job 1: C is defined for every $\delta \geq 1$, $|C(1)| = O(1)$

so continuation typing costs a constant per level, and $|C(\delta)| = \Theta(\log \delta)$ so a level pays the log of its anchor gap, whether the gap is a race or a cursor jump (step 1), and nothing else.

Monotone plus prefix-free is all of Job 2: any such code satisfies Theorem 2.5, which is why the Lean capstone is parametric in the code (§12) and why the unary control proves the same theorems at absurd cost. Entropy coding enters only in *which* monotone prefix-free code is chosen; the choice sets the constant in the bits-per-level bill and nothing in the correctness story. One requirement cuts across all four operations: the arithmetic is exact, never rounded (see the binary64 refutation below).

The contract's two structural properties are one Lean structure, and the coordinate is one recursion over it; these two displays are what every theorem of this part is parametric in.

Definition 2.1 (ordered prefix code). *An ordered prefix code is $\Gamma = (\text{enc} : \mathbb{N} \rightarrow \{0,1\}^*)$ satisfying, for all d, e :*

(m) *monotone: $1 \leq d$ and $d < e$ imply $\text{enc}(d) <_{\text{lex}} \text{enc}(e)$ (strict first-difference order on bit strings, $0 < 1$);*

(p) *prefix-free: $1 \leq d, 1 \leq e$, and $d \neq e$ imply $\text{enc}(d)$ is not a prefix of $\text{enc}(e)$.*

Both laws are conditioned on positivity: enc is only ever consumed at $\delta \geq 1$ (causality). The contract's third bullet (universal, with a cheap floor) is deliberately not a field: it is the cost requirement, selecting among inhabitants and entering no proof.

OrderedPrefixCode

Definition 2.2 (coordinate and mint, stored form). *The mechanization's coordinate is the bare codeword concatenation along the delta chain, on **List Bool**:*

$$\text{coordOf}_\Gamma(\langle \rangle) = \varepsilon, \quad \text{coordOf}_\Gamma(\langle d \rangle \cdot ch) = \Gamma.\text{enc}(d) \cdot \text{coordOf}_\Gamma(ch),$$

*a homomorphism from chain concatenation to string concatenation (**coordOf_append**). The mint of an insert $(t, \text{ins}(x \leftarrow a))$ carrying prefix π appends one codeword: $\text{mint}_\Gamma(\pi, t, a) = \pi \cdot \Gamma.\text{enc}(t - a)$ (**mint**; instance form **eCoord**). The three framing bits of the interval realization below (the leading 1 and the quarter-selecting 01) buy positivity and subtree interiority of the dyadic cells alone and are dropped in the stored form (the carrier paragraph below); nothing in the maintenance or faithfulness story changes.*

coordOf, mint

Theorem 2.3 (unique decodability). *For positive chains ch_1, ch_2 : $\text{coordOf}_\Gamma(ch_1) = \text{coordOf}_\Gamma(ch_2)$ implies $ch_1 = ch_2$. Prefix-free concatenations decode uniquely, so distinct birth chains mint distinct coordinates and coordinates are honest names.*

coordOf_inj

Step 1: differential coding. By causality $\delta = x - a \geq 1$. A chain-lex first difference always compares two entries with the *same* anchor (the shared prefix), and for a fixed anchor, ordering by x and by δ coincide, so each chain level may store the differential instead of the timestamp. Call δ the *anchor gap*: the events elapsed between the anchor's mint and the insert's. Continuation typing (each character anchored at its predecessor) has $\delta = 1$ at every level regardless of how long the document has lived. Two things widen the gap, and they pay the same bill: a genuine race with timestamp gap g costs $\log_2 g$ bits, and so does a *cursor jump*, the first character typed at an old anchor; its successors anchor at fresh characters and are back to $\delta = 1$. The differential is an encoding choice, not semantics: the mathematical object remains the timestamp.

Step 2: entropy coding. For $\delta \geq 1$ let L be the bit-length of δ and

$$C(\delta) = \underbrace{1 \cdots 1}_{L-1} 0 \text{ } (\delta \text{ with its leading bit removed}) \quad (|C(\delta)| = 2L - 1),$$

an order-preserving prefix-free code: exactly the contract. The length accounting: prefix-freedom makes each codeword self-delimiting, and a self-delimiting code over unbounded δ must spend bits twice, once on the value and once announcing its own length. C pays each bill at $\sim \log_2 \delta$. The header $1^{L-1}0$ is a unary announcement “ $L-1$ payload bits follow” (L bits); the payload is δ ’s trailing $L-1$ bits, since the leading bit of an L -bit number is always 1: once the header has said L it carries no information and is dropped. Hence

$$|C(\delta)| = \underbrace{(L-1) + 1}_{\text{header}} + \underbrace{(L-1)}_{\text{payload}} = 2L - 1 = 2\lfloor \log_2 \delta \rfloor + 1,$$

e.g. $C(1) = 0$, $C(2) = 100$, $C(3) = 101$, $C(5) = 11001$. This is Elias γ with its header flipped: γ announces the length in *zeros*, which is prefix-free but order-reversing across length classes: at the first divergence the longer, hence larger, number shows a header 0 against the shorter’s leading payload bit 1. Writing the header in ones makes that same position decide in favor of the longer code, so lexicographic order on codewords is numeric order on δ . The γ -flip is the building block, not the last word: only the *value* bill is forced at $\log_2 \delta$, and the *length* bill can itself be entropy-coded. Applying C to the length field (then the payload) gives the order-preserving flip of **Elias** δ , prefix-free and monotone by the same across-length-class argument, at $\log_2 \delta + O(\log \log \delta)$ per level; *this is the canonical mint*. Any order-preserving prefix-free code works here, and the artifact exists in three proved encodings of one datatype (**embed-tree** unary, **embed-code** the γ -flip, **eliasDeltaCode** the δ -flip); the capstone theorem is inherited at each verbatim, with zero new proof content (§12): the code-parametricity claim, machine-validated. Two cost qualifications apply: per level the δ -flip wins only from $\delta \geq 32$ and loses at $\delta \in \{2, 3\} \cup [8, 15]$ (kernel-checked length comparisons), and measured editing tails are thin, so on real traces the two flips are a wash; the dominance shows in race-heavy shapes (the entropy-law table below) and in the asymptotics.

Proposition 2.4 (three proved instances). *Three inhabitants of Definition 2.1 are proved and packaged; every parametric theorem of this part is inherited at each verbatim.*

- *Unary (**unaryCode**):* $\text{enc}(d) = 1^d 0$; *monotonicity and prefix-freedom by two short inductions (**unaryEnc_mono**, **unaryEnc_not_prefix**);* $\Theta(d)$ bits per level, the cost-control arm.
- *The γ -flip (**binaryCode**):* $\text{enc}(d) = 1^{L-1} 0$ (d minus its leading bit) with L the bit-length of d ; $|\text{enc}(d)| = 2L - 1$ at $1 \leq d$ (**binEnc_length**); *monotone at* $1 \leq d$, $d < e$ (**binEnc_mono**) *and prefix-free at* $1 \leq d$, $1 \leq e$, $d \neq e$ (**binEnc_prefixFree**).
- *The δ -flip (**eliasDeltaCode**):* $\text{enc}(d) = \text{binEnc}(L) \cdot (d \text{ minus its leading bit})$, the length field itself γ -flip-coded; $|\text{enc}(d)| = L + 2\text{len}(L) - 2$ at $1 \leq d$ (**dEnc_length**); **dEnc_mono**, **dEnc_prefixFree**, the same hypothesis shapes. The per-level crossover described above is kernel spot-checked at $\delta \in \{2, 8, 16, 32, 1024\}$ in the same file.

The γ - and δ -flips are cross-validated against the Python codes by kernel **decide** on concrete codewords (4 and 10 codeword pins respectively; the unary code has no pins, its **enc** being definitional); per code, this is Theorem 2.5(i). **unaryCode**, **binaryCode**, **eliasDeltaCode**

The mint. Writing $b = 1 \cdot C(\delta)$ (a leading 1 keeps coordinates positive) and $w = 2^{-|b|-2}$, the mint is the second quarter of b 's dyadic cell:

$$M(\delta) = (0.b + w, 0.b + 2w) \subset [0.b, 0.b + 4w).$$

The quarter placement leaves a gap below and headroom above inside the cell, and the open space $(0.b + 2w, 0.b + 4w)$ is never minted: it exists so that the interval has room to *contain* the node's own subtree without touching the next cell. This also pins down the per-level *block* promised in the vocabulary: $0.b + w$ written as a string is $b01$, so the block is $1C(\delta)01$, the codeword in its three framing bits (the leading 1 for positivity, the 01 that selects the second quarter). The block is what a node stores; read as a binary fraction it is the lower endpoint of $M(\delta)$, and flipping the trailing 01 to 10 gives the upper. Worked out for small deltas:

δ	$C(\delta)$	block = $1C(\delta)01$	$M(\delta)$	bits/level
1	0	1001	(9/16, 10/16)	4
2	100	110001	(49/64, 50/64)	6
3	101	110101	(53/64, 54/64)	6
4	11000	11100001	(225/256, 226/256)	8
5	11001	11100101	(229/256, 230/256)	8
10	1110010	1111001001	(969/1024, 970/1024)	10
100	1111110100100	1111111010010001	(65169/65536, 65170/65536)	16

Three things to read off. The endpoints are consecutive dyadics at scale $2^{-(|b|+2)}$ (numerators n and $n+1$): the mint is one tick wide at its own scale, with the tick below it (the gap) and the two ticks above it (the subtree headroom) never minted. The rows are strictly increasing as fractions: monotonicity made visible. And the bit count is $2L + 2$ for an L -bit δ : four bits at $\delta = 1$ (the sequential-typing constant of the entropy law), sixteen at $\delta = 100$.

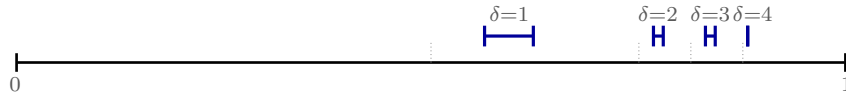


Figure 3: Mints $M(\delta)$ for $\delta = 1..4$ inside the parent's unit interval (cell boundaries dotted). Larger deltas sit strictly higher; distinct deltas are disjoint with gaps; each mint is a quarter of its cell, leaving room for the node's own subtree. Drawn for the γ -flip C ; the construction reads verbatim over any order-preserving prefix-free code, in particular the canonical Elias- δ mint (step 2).

The stored state. The carrier stores each live node parent-relatively:

$$\sigma : t \mapsto (\text{par} \in \mathbb{N}, \quad s \in \{0,1\}^*, \quad \text{char}),$$

where s is a block ($1C(\delta)01$ at birth; after folds, below, a concatenation of blocks) *relative to the parent*, and $\text{par} = 0$ denotes the root sentinel (which owns $(0,1)$ and is never stored). The record holds no interval; the interval is the string's decoding,

$$(lo, hi) = (0.s, 0.s + 2^{-|s|}) \subset (0,1),$$

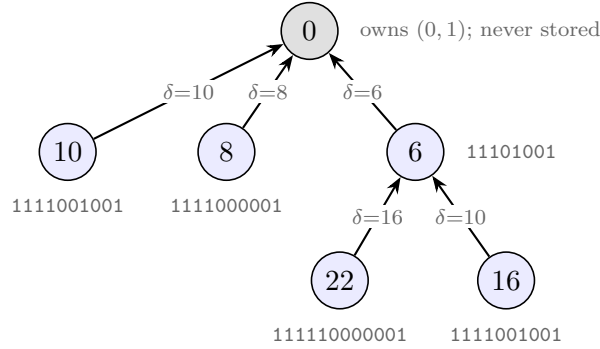
consecutive dyadics at s 's own scale, one block at a time exactly as in the mint table. The maintenance below and Theorem 2.5 are stated on the intervals, and every operation they perform on an interval is a string operation on s : composition is concatenation, comparison is lexicographic. (The validation model stores the decoded pair directly, as exact fractions.) A node's *absolute* interval is the affine composition of the intervals along its parent chain. For a node v with mint $M(\delta_v) = (lo_v, lo_v + w_v)$, let

$$\varphi_v : (0, 1) \rightarrow M(\delta_v), \quad \varphi_v(x) = lo_v + w_v \cdot x,$$

the unique increasing affine bijection of the unit interval onto v 's mint; it maps a subinterval (a, b) to $(lo_v + w_v a, lo_v + w_v b)$, and on strings it prefixes v 's block: $\varphi_v(0.y) = 0.(s_v y)$. Write

$$\alpha(u) = \varphi_{c_1} \circ \cdots \circ \varphi_{c_k}(M(\delta_u))$$

for the composition along u 's birth chain $c_1 \cdots c_k u$ (δ_v is v 's delta to its birth parent): $\alpha(u)$ is the *encoding of* chain(u), and at the string level it decodes the concatenation of the blocks along the chain. Worked on the tree of Figure 2, redrawn before $\text{del}(6)$ with the delta (child minus anchor) on each edge and the stored block under each node:



Writing $(n, n+1)/2^k$ for the interval $(n/2^k, (n+1)/2^k)$ and $\approx$ for its lower endpoint:

node	δ	stores (par, s)	decode of s	α (absolute)	$\approx$
10	10	(0, 1111001001)	$(969, 970)/2^{10}$	$(969, 970)/2^{10}$	0.94629
8	8	(0, 1111000001)	$(961, 962)/2^{10}$	$(961, 962)/2^{10}$	0.93848
6	6	(0, 11101001)	$(233, 234)/2^8$	$(233, 234)/2^8$	0.91016
22	16	(6, 111110000001)	$(3969, 3970)/2^{12}$	$(958337, 958338)/2^{20}$	0.91394
16	10	(6, 1111001001)	$(969, 970)/2^{10}$	$(239561, 239562)/2^{18}$	0.91385

The root's children store their absolute intervals already (par = 0). The grandchildren compose: 22's absolute string is its parent's block followed by its own, 11101001111110000001 (20 bits, hence the 2^{20}), and likewise 16's (18 bits). Both absolute intervals sit inside $M(6) = (0.91016, 0.91406)$ while 8's sits above it, so the descending- α read is [10, 8, 22, 16] and 8 cannot land between the orphans. Nodes 16 and 10 store the same string, both being $\delta = 10$ births; records are parent-relative, so this is no collision. After $\text{del}(6)$ the fold rewrites the two records to (par 0, the concatenated s) and the α column is unchanged bit for bit. Figure 4 draws these intervals (widths schematic).

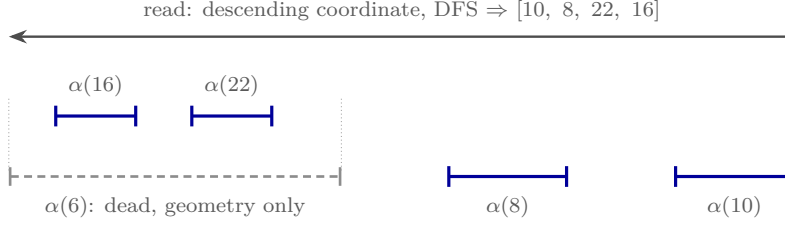


Figure 4: Figure 2 downstairs (widths schematic; true mints are the dyadic cells of Figure 3). Deleting 6 removes its record, but $\alpha(16)$ and $\alpha(22)$ are compositions *through* $M(6)$: the dead node survives as the dashed container, the tombstone compressed into geometry. 8's interval can never land inside $\alpha(6)$'s span: distinct deltas occupy disjoint cells (Theorem 2.5(i)), which is the credential verdict, now a fact about intervals.

Maintenance. How the stored form tracks the datatype:

- $\text{ins}(x \leftarrow a)$ mints $M(x - a)$ relative to a : the new record depends only on the two timestamps.
- $\text{del}(d)$ *isometrically folds* d into its children: each child c is re-parented to d 's parent and d 's record is substituted into c 's. On the stored strings this is concatenation, $c.s := d.s.c.s$; read as intervals, $c.\text{frac} := d.\text{frac} \circ c.\text{frac}$, where $I \circ J$ is the image of J under the increasing affine bijection of $(0, 1)$ onto I (the φ of the stored state, with I in place of a mint): for $I = (lo, lo + w)$,

$$I \circ J = (lo + w \cdot lo_J, lo + w \cdot hi_J).$$

The substitution is exact. Upstairs, deletion changes nobody's sort key (§1); the fold is the parent-relative storage scheme *re-normalizing* while every absolute interval stays fixed (Figure 5): representation maintenance, not datatype semantics. On the running example, $\text{del}(6)$ removes 6's record and substitutes it into both children: 22's record $(6, 111110000001)$ becomes $(0, 1110100111110000001)$, whose decode is

$$\begin{aligned} (233, 234)/2^8 \circ (3969, 3970)/2^{12} &= (233 \cdot 2^{12} + 3969, 233 \cdot 2^{12} + 3970)/2^{20} \\ &= (958337, 958338)/2^{20}, \end{aligned}$$

and likewise 16's becomes $(0, 111010011111001001)$, decoding $(239561, 239562)/2^{18}$. Both are exactly the α column of the worked table: the absolute intervals did not move, only the parent-relative bookkeeping did.

- The merge *refolds*: for each survivor, compute its absolute interval by folding its record to the root inside the input that holds it (a record's stored parent is always live in its own input, so the chain stays in one input; by Theorem 2.5(iii) the choice of input cannot matter, asserted at runtime and never fired); re-parent to the nearest *surviving* ancestor; re-relativize.
- The read is depth-first from the root, children in descending order of lo . No tie rule is needed (Corollary 2.6).

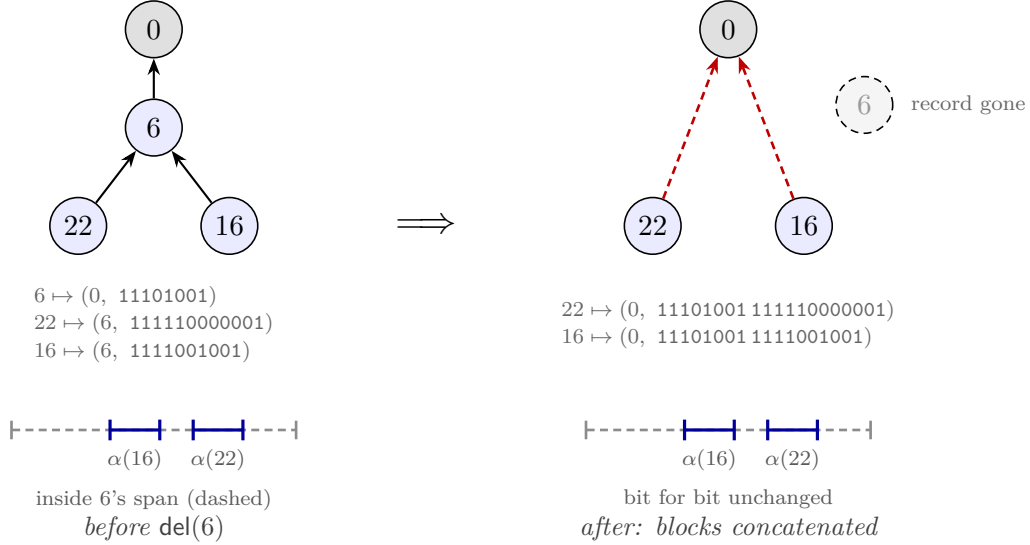


Figure 5: The isometric fold on the running example. `del(6)` deletes 6’s record and concatenates its block `11101001` onto the front of each child’s, re-parenting both to the root (red: the re-parenting, a storage move only). The stored states change: 22 and 16 now carry two-block strings and par 0. The absolute intervals $\alpha(22) = (958337, 958338)/2^{20}$ and $\alpha(16) = (239561, 239562)/2^{18}$, and with them the display, are bit-for-bit unchanged: upstairs, the sort keys still read $\langle 6, 22 \rangle$ and $\langle 6, 16 \rangle$. The dashed span is 6’s old interval, surviving as pure geometry.

Why the codeword must announce its own end. Before the theorem, the countermodel that motivates it. A single codeword in a field of known extent needs no header, and the length looks recoverable from the string itself; plain binary shows what breaks, once per purchase in the contract. *Geometry:* $B(2) = 10$ prefixes $B(5) = 101$, so $B(5)$ ’s cell sits inside $B(2)$ ’s; containment means ancestry, so the $\delta=5$ sibling lands amid the $\delta=2$ sibling’s descendants and non-interleaving dies with everything alive. *Parsing:* once a fold erases the structural boundary between blocks, `101100...` reads as $B(2)$ then `11...` or as $B(5)$ then `00...`, and nothing outside the string can say. *Ties:* a binary fraction cannot see trailing zeros, so `10` and `100` both denote $\frac{1}{2}$; distinct siblings mint touching intervals, and the id tiebreak wakes into an arbiter that orders by id, not chain-lex. Nor can the header bits be dodged: prefix-freedom demands $\sum_{\delta} 2^{-\ell(\delta)} \leq 1$, and plain binary puts 2^{L-1} values at every length L , so every prefix-free code pays a length announcement of order $\log L$ somewhere (out-of-band boundary tables cost the same bits and forfeit the string as a self-contained key; database key encodings inline it for the same reason). And the fold names where the boundary went: the tree node that carried it structurally is the deleted tombstone, whose ordering verdict becomes the payload and whose boundary becomes the header. “Tombstone compressed into coordinate entropy” is literal at the bit level.

The one obligation. The four theorems of the earlier drafts collapse into a single statement about the encoding.

Theorem 2.5 (encoding faithfulness). *(i) Order-embedding at one level: for $\delta \neq \delta'$, $M(\delta)$ and $M(\delta')$ are disjoint with a gap; $\delta < \delta' \Rightarrow M(\delta)$ lies entirely below $M(\delta')$.*

(ii) *Homomorphism*: α realizes chain concatenation as affine composition and embeds chain-lex in the interval order: for distinct u, v , if $\text{chain}(u)$ is a proper prefix of $\text{chain}(v)$ then $\alpha(v) \subset \alpha(u)$ (and, if both are live, the two are never read-time siblings); otherwise $\alpha(u) \cap \alpha(v) = \emptyset$, with the chain-lex-greater element's interval strictly higher.

(iii) *Exact maintenance*: in every reachable state, every live node's stored records compose to exactly $\alpha(u)$.

(Paper-level statement, proved below; its mechanized counterparts are the code laws of Definition 2.1 with Theorem 2.3, and the stored form is Theorem 2.7.)

Proof. (i) C is prefix-free: codes of lengths $2L-1 < 2M-1$ differ at position L (the shorter's header terminator 0 against the longer's header 1); equal-length codes differ in the payload. Prefix-free codes have pairwise-disjoint dyadic cells. C is monotone as a binary fraction: within a length class the payload is δ 's low bits in order; across classes the position- L bit decides in favor of the longer code. Disjoint cells ordered by their codes, and mints interior quarters of their cells, give both claims. (ii) If neither chain is a prefix of the other, they first differ on distinct deltas relative to the *same* birth ancestor (§2, step 1); (i) separates them there, and affine images of disjoint intervals are disjoint and order-preserved (each φ is increasing). If $\text{chain}(u)$ prefixes $\text{chain}(v)$, then $\alpha(v) \subseteq \varphi_{c_1} \circ \dots \circ \varphi_{c_k}(\varphi_u((0,1))) = \alpha(u)$ by construction; and if both are live, v 's nearest live ancestor is u or deeper, so they never share a current parent, and sibling groups are pairwise disjoint. (iii) By induction over transitions. `ins` establishes it by definition. `del`'s fold replaces (`par`, `frac`) pairs by their exact composition, leaving α unchanged. The merge recomputes precisely $\alpha(u)$ (folding a record chain to the root) and re-relativizes against another α ; ties cannot exist by (i) and (ii), so no step of any transition ever rewrites an interval by anything but exact composition. $\square$

Corollary 2.6 (display $\equiv$ chain-lex; no tie rule). *The read (DFS by descending lo) enumerates survivors in exactly chain-lex order: per level, larger delta first, so among same-parent siblings, exactly newest first; survivors rank by their own timestamps, folded heirs by their dead founder's. Sibling lo-values never tie; the read's id tiebreak is dead code. Machine checks: pairwise sibling disjointness asserted at every read of 320 randomized executions, all clean; over 19,531 multi-member sibling groups, own-timestamp order violated 3,481 times (exactly the post-fold heir groups; forcing own-ts there is the refuted dissolution behavior), **chain-lex violated 0 times**. (Paper-level corollary of Theorem 2.5; the mechanized form of the display-order claim is Theorem 2.7.)*

Theorem 2.7 (the chain-lex theorem, stored form). *For distinct positive chains ch_1, ch_2 :*

$$\text{keyLt}(\text{key}(\text{coordOf}_\Gamma(ch_2)), \text{key}(\text{coordOf}_\Gamma(ch_1))) = \text{true} \iff \text{chainBefore } ch_1 \text{ } ch_2 :$$

the display comparator on stored coordinates (keys descending) computes exactly the chain order of Definition 1.1, for every ordered prefix code. This is Corollary 2.6 as one flat comparison of immutable bit strings, the form the read executes. **display_iff_chainBefore**

The entropy law. A coordinate is the concatenation of the codes along the birth chain: $\Theta(\sum_i \log \delta_i)$ bits, the entropy of the chain. Continuation typing ($\delta = 1$ at every level) costs ~ 4 bits per level under every code here; a level minted across an event gap g , by a race or by the first character after a cursor jump, costs $\log_2 g + O(\log \log g)$ bits under the canonical Elias- δ mint

($2\log_2 g + 1$ under the γ -flip). *The state pays for the magnitude of edit non-locality, concurrent or not, and for nothing else.* Measured (1000 elements; the numbers are *order metadata alone*, survivors’ coordinate bits read off the dyadic denominators; characters are excluded, since data costs are identical across all designs):

scenario	quarter carve (refuted)	unary mint	γ -flip mint	Elias-δ mint
chain then delete-all ($\delta=1$)	2^{2001}	2^{502501} (~ 61 KB)	2^{4001} (~ 0.5 KB)	2^{4001} (~ 0.5 KB)
1000 root siblings ($\delta=t$)	— (ties)	$\max 2^{1003}, 125$ KB	$\max 2^{23}, 5$ KB	$\max 2^{20}, 4$ KB

The carrier. The natural representation is a **parent-relative bit string per node**: the fold is concatenation, comparison is lexicographic ($O(1)$ -ish with structure sharing); the rationals are the semantics of the strings, kept for the proofs. Nothing here computes $M(\delta)$: the endpoints $0.b + w$ and $0.b + 2w$ are the strings $b01$ and $b10$, so the mint emits a block and the fold concatenates, and *hi* need not be stored at all. The Lean mechanization drops even the framing bits: its coordinate is the bare codeword concatenation on `List Bool` (`mint` appends $\Gamma.\text{enc}(t - a)$ to the anchor’s coordinate), with the prefix rule stated directly on strings; the quarter framing serves the interval realization alone, buying positivity and subtree interiority. The rational $M(\delta)$ is computed in exactly one place, the validation model `embed_tree.py`, as exact fractions; the entropy tables read their bit counts off those denominators. IEEE 754 binary64 is machine-refuted as the carrier, a faithfulness failure rather than a design failure: mints degenerate at $t = 52$, depths underflow at 44, and rounding voids Theorem 2.5(iii) (6/120 PBT executions die). The obstruction is a pigeonhole (chains carry $\Theta(\sum \log \delta)$ bits; a fixed-width word holds 64), but binary64 is sound as a *comparison cache* with exact fallback on float equality.

Timestamps in practice. The unique stamps in $\mathbb{N}^+$ are Lamport’s 1978 construction, flattened: each replica ticks a counter per local operation, advances it to the max on merge, and breaks ties by replica id, packed as $t \cdot 2^k + r$ with k id bits. Uniqueness and causality ($t_a < t_x$ gives $\text{packed}_a < \text{packed}_x$ regardless of ids, so $\delta \geq 1$) both survive the packing; the proof layer only *assumes* uniqueness (the Lamport-structural part of honesty), and this construction is the witness that the assumption is realizable. Two entropy caveats. (i) Packing inflates every delta by 2^k , costing $\sim 2k$ extra bits per chain level, which turns the 0.5 KB sequential run above into ~ 3 KB at $k = 10$. Cheaper: extend the mint’s code to the pair, $C(\Delta t)$ followed by a k -bit id. The pair code is still order-preserving prefix-free (lex on $(\Delta t, r)$), and costs k bits only where the level is minted. A further option, unvalidated against the battery, is to order sibling ids *relative to the anchor’s* (any deterministic per-anchor id order is a legitimate tiebreak; convergence needs agreement, not a uniform order), making “same author as my anchor”, i.e. sequential typing, cost ~ 1 bit, so only genuine cross-author races pay $\log R$. (ii) The clock must be *dense logical time, never physical*: wall-clock or HLC stamps make δ measure elapsed time rather than elapsed events: a keystroke every 100 ms over μs -resolution stamps mints $\delta \approx 10^5$, ~ 34 bits per level for a purely sequential edit. The entropy law holds precisely because the counter ticks once per event, making δ an event-count gap; if wall-clock order is ever wanted, it belongs in a display-layer tiebreak, never in the minted coordinate.

What the storage layer now delivers. Dead chain entries concatenated into survivors’ strings are the conservation law’s irreducible content (§1); reclaiming them under *causal stability*

(renumber once every replica has seen the delete, the only condition under which rewriting geometry is safe, since no verdict involving the dead can be contested again) was the deepest of the open cost items. That item is now closed, and the answer occupies its own section (§8): a verified re-coding theorem cluster at settled cuts (rank renumbering and spine fusion, gated by §’s SettledAt), and, one level further down, the run table, a lossless re-representation of the live chains that collapses the per-record cost by one to two orders of magnitude with *no* stability gate at all. The residual headroom (per-anchor context codes, Steiner-sharing dead prefixes across co-heirs, the Elias- δ constant of step 2) remains catalogued in the companion assessment `whiteboard/order-coding-compression-notes.md`.

3 The insert prefix: for the proof, and the proof alone

The insert payload π exists for verification, not execution. This section says why it must exist, and why an implementation may drop it from the wire.

Why rc must be Either. The verification framework (Part I) resolves a concurrent pair of non-commuting operations by consulting a declared conflict-resolution order rc ; any *oriented* entry for an insert/delete pair, however, incurs a base commutation obligation that is machine-refuted for this design space: ordering $\text{ins}(x \leftarrow a)$ against $\text{del}(a)$ (in either direction) is undischageable as a conflict entry, because rehomings is non-transitive across chains of deletions. So $rc = \text{Either}$: the proof must show the two *commute*, in both orders, on all applicable states.

The commutation, and where the prefix earns its keep. Order 1, $\text{ins}(x \leftarrow a)$ then $\text{del}(a)$: x is born with chain $\pi \cdot x$, and the delete touches no chain (downstairs: the fold re-homes x ’s record with composed coordinates; by exactness $\alpha(x) = \alpha(a) \circ M(x - a)$). Order 2, $\text{del}(a)$ then $\text{ins}(x \leftarrow a)$: a is gone; application must still be *total* and land x on the same sort key. This is what the carried prefix $\pi = \text{chain}(a)$ (equivalently, the anchor’s coordinate) provides: apply sets $\text{chain}(x) = \pi \cdot x$ (downstairs: $\alpha(x) = \pi \circ M(x - a)$, attached to the nearest live ancestor). The two orders produce *equal* states: commutation to equality, not merely up to $\approx$, courtesy of canonicity (Theorem 1.3). Figure 6 draws the square on the smallest instance.

The commutation kernel is stated with its guards displayed in full: this is a *conditioned* commutation, and per the framework’s standing post-mortem, a commutation whose quantifier was edited is exactly where a verification effort can fool itself, so the guard list is the content.

Definition 3.1 (conditioned commutation, embed form). *For ops o_1, o_2 , $\text{commutesWith}'_\Gamma(o_1, o_2)$ demands, for every state s satisfying all five guards:*

- *0 is not a stored key of s ;*
- *o_1 and o_2 are each accurate at s (**accurate**: an insert’s anchor precedes it, $a < t$, and the carried prefix is the anchor’s stored coordinate, or the anchor is the root and the prefix empty; a delete’s target is live);*
- *o_1 and o_2 each have fresh timestamps (**fresh_ts**: an insert’s t is nonzero and unstored; vacuous for deletes),*

*that $o_2(o_1(s)) \equiv o_1(o_2(s))$, where $\equiv$ is extensional map equality (equal domains, equal records: the carrier’s literal equality, not a quotient). *commutes_with'**

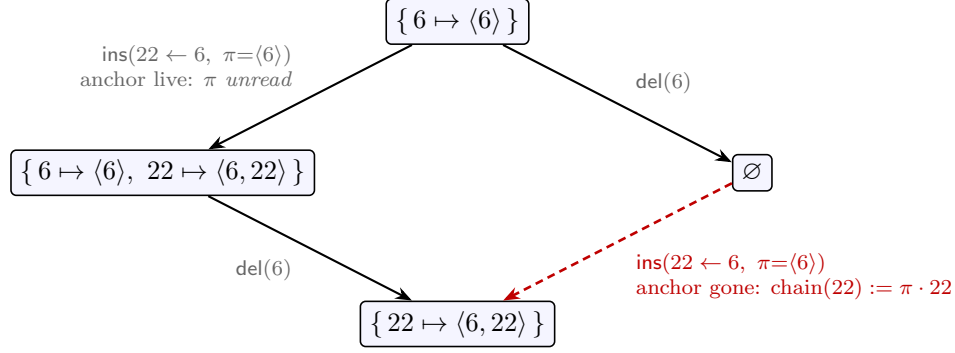


Figure 6: $\text{rc} = \text{Either}$ discharged: $\text{ins}(22 \leftarrow 6)$ and $\text{del}(6)$ commute *to equality*. Down the left, the anchor is live and application never consults π (the reachable, honest order). Down the right, 6 is already gone: the state cannot supply the anchor’s chain, so the carried $\pi = \text{chain}(6)$ does, landing 22 on the same sort key. The right-hand state is unreachable under honesty; the verification condition quantifies over it anyway, and π exists exactly to make that corner total.

Theorem 3.2 ($\text{rc} = \text{Either}$ is honest). *For all o_1, o_2 with distinct timestamps and distinct origin replicas:*

$$\text{rc}(o_1, o_2) = \text{Either} \iff \text{commutesWith}'_{\Gamma}(o_1, o_2).$$

*Since rc is **Either** everywhere, the content is the forward direction: every distinct cross-replica pair commutes on guarded states, to equality. The one clause with teeth is that a delete’s live target is never a fresh insert’s timestamp (accuracy plus freshness), ruling out the causally impossible insert-then-delete-it shape.* **rc_non_comm'** (*Sal/MRDTs/RGA_Embed/RGA_Embed_MRDT.lean*)

Ghost, in the strict sense. On every reachable state the anchor is live at application (honesty), and the application reads only the state and the two timestamps: π is never consulted. The Lean form is exact:

Theorem 3.3 (the prefix is ghost). *If 0 is not a stored key of s and o is accurate at s , then $\text{do}_{\Gamma}(s, o) = \text{do}_{\Gamma}^{\text{run}}(s, o)$: the always- π transition and the state-reading transition (**do_run**, which mints from $\text{pos}_s(a)$, or from ε at the root) are literally the same function on accurate ops.* **ins_prefix_ghost**

An implementation may therefore drop π from the wire entirely; the *proof* may not, because the verification conditions quantify over reorderings that visit unreachable states, and there application without π is partial. The prefix costs $O(\sum \log \delta)$ bits, the same entropy as the coordinate itself, and exists at the proof layer alone.

4 Validation

All numbers reproduce from `whiteboard/litmus/` in the accompanying Sal repository (`embed_tree.py`; harnesses `litmus.py`, `pbt.py`). Notation: $\text{RGA}_{\dagger}$ is the published tombstoned RGA (Roh et al.; §11); the *rehoming* RGA is the convergence-verified flat tombstone-free design of §1 (refuted repo-wide: the L1 row below generalizes into the kernel-checked sequential refutation of Part III, §18, and the embedded-chain design holds the canonical seat). The L1–L25

battery is the litmus suite accumulated over this project’s design sweep; each entry is the named countermodel that killed at least one of 16 prior candidate designs. The one exception below, L19, is discharged by the sided generalization of §5.

check	verdict
litmus battery L1–L25 (incl. every countermodel that killed 16 prior designs: ceiling escapes, tie inheritance, three-branch topology, frame mixing, fold-verdict inheritance)	clean, except one-sided L19 (backward-run interleaving, the published RGA’s own anomaly)
credential countermodel (tie heirs vs mid-ts third party, death-before vs death-after topologies)	converges, no flips; = $\text{RGA}_\dagger$ ’s reads
subordination-escape and retroactive-subordination CEs	survive
randomized version-DAG PBT (FLIP / CONV / LIVE / DUP)	120/120 and 300/300 clean
per-read sibling-disjointness assertion sweep	320/320 clean
chain-lex instrumentation (19,531 sibling groups)	0 violations
lockstep vs the published tombstoned RGA	read-equal at every apply/merge/read, 120/120
L1 delete-reorder vs the verified rehoming RGA	embed $[2, 3]$ ✓ vs rehoming $[3, 2]$ ×

The lockstep row is the identification: *the embedded-chain RGA is observationally the published tombstoned RGA with the tombstone set compressed into coordinate entropy*, machine-checked on 120 executions and now a kernel-checked theorem (the compaction theorem, §12): on every honest event set, any enumeration of the tombstoned RGA’s visible ids in its own visible order *equals* the embed read, element for element. The L1 row is the separation from the verified rehoming RGA, and it is exactly the one-line difference of §1: that design’s climb re-sorts a rehomed node by its own timestamp (the $[b, a, c] \rightarrow [c, b]$ anomaly), i.e. its sort key is the birth chain *truncated at each rehoming*; this design keeps the chain whole.

5 The sided generalization: two-sidedness as a parameter

The one blemish in the battery is L19: two replicas that each type a *backward* run at the same place (repeatedly “insert before the current head”) interleave, because every insert-before anchors at the same left neighbor and both runs land in one sibling group. The fan is not a traversal artifact: no read order of the same birth tree un-fans a fan, so any fix must change where insert-before *anchors*. The Fugue family’s answer is a second side: insert-before anchors at the *right* neighbor as an L-entry, so a backward run is a chain, exactly as a forward run already is.

The L19 example, concretely. It is the battery’s own trace (Figure 7). The LCA document is $\langle 1 \rangle$ (one insert, stamp 1, anchored at the start marker $\diamond$). Then, concurrently, replica A types a backward run at the head, three inserts each anchored “before the current first element”:

ins(10 at head), ins(30 at head), ins(50 at head),

so A locally reads $\langle 50, 30, 10, 1 \rangle$; replica B does the same with stamps 21, 41, 61 and locally reads $\langle 61, 41, 21, 1 \rangle$. (The stamps interleave numerically, $10 < 21 < 30 < 41 < 50 < 61$, because the two replicas’ Lamport clocks advance together.) In the one-sided datatype, “insert at the head” can only anchor at the left neighbor, which is $\diamond$ for every one of the six inserts: all six become

R-children of $\diamond$, one sibling group, and the sibling tiebreak (newest first) merges the two runs by stamp:

$$\langle 61, 50, 41, 30, 21, 10, 1 \rangle$$

after the merge, the two sentences shuffled together. No traversal of that tree can do better; the information “these three form one run” is simply not in a sibling fan whose stamps interleave.

The sided datatype, and why it prevents the interleaving. The generalization does not build a second datatype. An insert names an anchor *and a side*; chains become sequences of (side, timestamp delta) entries; and the display rule generalizes from the prefix rule (“a prefix precedes its extensions”) to the in-order rule (“L-extensions, then the node, then R-extensions”). One marker trick makes this ordinary lexicographic comparison again: a node’s sort key is its coordinate followed by a virtual terminator, and the sided symbol alphabet is ordered

$$(R, 1) < (R, 2) < \dots < \text{marker} < \dots < (L, 2) < (L, 1),$$

with the L band *mirrored* (among L-siblings the newer sits adjacent to the node). Marker above the R band puts a node before its R-extensions; marker below the L band puts L-extensions before the node. The three displays that carry this section:

Definition 5.1 (the sided chain and its coordinate). *A sided chain is a list of entries (side, δ), side $\in \{R, L\}$, deltas positive ($SEntry$, $SChain$, $PosSChain$). Its stored coordinate concatenates one symbol block per entry ($sBlock$, $sidedCoordOf$):*

$$\text{block}(R, \delta) = \Gamma.\text{enc}(\delta) \text{ in the } R \text{ band}, \quad \text{block}(L, \delta) = \overline{\Gamma.\text{enc}(\delta)} \text{ in the } L \text{ band},$$

the R band mapping $0 \mapsto 1$, $1 \mapsto 2$, the L band $0 \mapsto 4$, $1 \mapsto 5$, and $\bar{\cdot}$ the bitwise complement (**compl**: the order-reversing twin at identical lengths, prefix-freedom preserved); equivalently, one band-symbol map per side, L pre-composing the complement ($sBlock_eq$, $sideSym$). The sort key appends the marker, $sKey(c) = c \cdot \langle 3 \rangle$, and the bands straddle it: $\{1, 2\} < 3 < \{4, 5\}$. $sidedCoordOf$, $sKey$

Definition 5.2 (the in-order rule). *Entry divergence at a shared anchor ($sEntryBefore$): among R-siblings the newer first ((R, d) before (R, e) iff $e < d$); among L-siblings the older first ((L, d) before (L, e) iff $d < e$, the mirror); every L entry before every R entry. The display order on sided chains is the least relation closed under three rules:*

$$ch \cdot \langle (L, d) \rangle \cdot \text{rest} \prec ch, \quad ch \prec ch \cdot \langle (R, d) \rangle \cdot \text{rest}, \quad q \cdot \langle e_1 \rangle \cdot c_1 \prec q \cdot \langle e_2 \rangle \cdot c_2 \text{ (} e_1 \text{ before } e_2 \text{)} :$$

L-extensions precede the node, the node precedes its R-extensions, divergences by the entry order. Distinct chains are always comparable ($schainBefore_total$). $schainBefore$

Theorem 5.3 (the sided marker theorem). *For distinct positive sided chains c_1, c_2 :*

$$\text{keyLt}(sKey(sidedCoordOf_\Gamma(c_2)), sKey(sidedCoordOf_\Gamma(c_1))) = \text{true} \iff schainBefore\ c_1\ c_2 :$$

two-sidedness is plain lexicographic comparison over the marker alphabet; no bespoke three-way prefix clause survives into the read. Unique decodability holds alongside ($sidedCoordOf_inj$: the side is recoverable from the first symbol’s band, the delta by per-band prefix-freedom). $sdisplay_iff_schainBefore$

On the example, the Fugue anchoring rule (the generation-policy paragraph below) sends each “insert at head” to the *successor* as an L-entry, so each run is an L-chain (Figure 7, right). Writing chains as entry sequences with deltas relative to the parent’s stamp ($10 = 1 + 9$, $30 = 10 + 20$, ...):

$$\begin{array}{ll} \text{chain}(1) = (R, 1) & \text{chain}(10) = (R, 1) (L, 9) \\ \text{chain}(30) = (R, 1) (L, 9) (L, 20) & \text{chain}(21) = (R, 1) (L, 20) \\ \text{chain}(50) = (R, 1) (L, 9) (L, 20) (L, 20) & \text{chain}(41) = (R, 1) (L, 20) (L, 20) \end{array}$$

and the marker-lex comparisons (keys descending in display order) work out as follows, writing M for the marker:

- *L-extensions precede the node*: $\text{key}(10) = (R, 1)(L, 9) M$ vs $\text{key}(1) = (R, 1) M$ first differ at position 2, $(L, 9)$ vs M , and the L band sits above the marker: so 10 displays before 1. Likewise 30 before 10, 50 before 30: each backward keystroke lands immediately left of the previous one, which is the typed intent.
- *The two runs are decided once, not per pair*: $\text{key}(10)$ vs $\text{key}(21)$ first differ at position 2, $(L, 9)$ vs $(L, 20)$, and the L band is mirrored, so $(L, 20) < (L, 9)$: A’s run displays before B’s. Now *every* node of A’s chain carries the prefix $(R, 1)(L, 9)$ and every node of B’s carries $(R, 1)(L, 20)$, so *every* cross-pair first-differs at that same position 2 with that same verdict. The blocks cannot interleave:

$$\langle 50, 30, 10, 61, 41, 21, 1 \rangle.$$

This is the datatype-level subtree convexity (Theorem 5.4): *any* policy that keeps a run inside one subtree gets non-interleaving from the datatype, for free.

Theorem 5.4 (sided subtree convexity). *For every prefix p and sided chains c_x, c_y, c_z : if c_x and c_y extend p , $\text{schainBefore } c_x c_z$, and $\text{schainBefore } c_z c_y$, then c_z extends p . With the empty extension allowed, the node itself is a member: a subtree’s display interval is L-descendants, the node, R-descendants, with nothing foreign between. A pure chain lemma: no state, no honesty, no property of the code.*

schain_subtree_convex

The current datatype is the all-R fragment of this order, *exactly*: with L uninhabited the in-order rule degenerates to the prefix rule, and the existing key of §2 (the sym-mapped bits plus terminator) is literally this construction with the L band empty. That is a theorem, not a slogan, twice: once on chains, once on folds.

Theorem 5.5 (the all-R fragment theorem). *For distinct positive one-sided chains c_1, c_2 , with liftR tagging every entry R :*

$$\text{schainBefore } (\text{liftR } c_1) (\text{liftR } c_2) \iff \text{chainBefore } c_1 c_2,$$

*via the key identity $\text{sKey}(\text{sidedCoordOf}_\Gamma(\text{liftR } c)) = \text{key}(\text{coordOf}_\Gamma(c))$ (*sKey_liftR*): with L uninhabited, the in-order rule is the prefix rule, symbol for symbol.*

schainBefore_liftR

Theorem 5.6 (the fragment at the instance). *For every operation list ρ of the one-sided instance,*

$$\text{sFold}_\Gamma(\text{map liftOp } \rho) = \text{map liftRec } (\text{eFold}_\Gamma \rho),$$

*record for record. Unconditional: no honesty, no sortedness hypotheses, a pure simulation; the reads agree as a corollary (*sided_allR_read_eq*). Chained with the compaction theorem (§12), the all-R sided instance reads as the published tombstoned RGA.*

sFold_liftOp

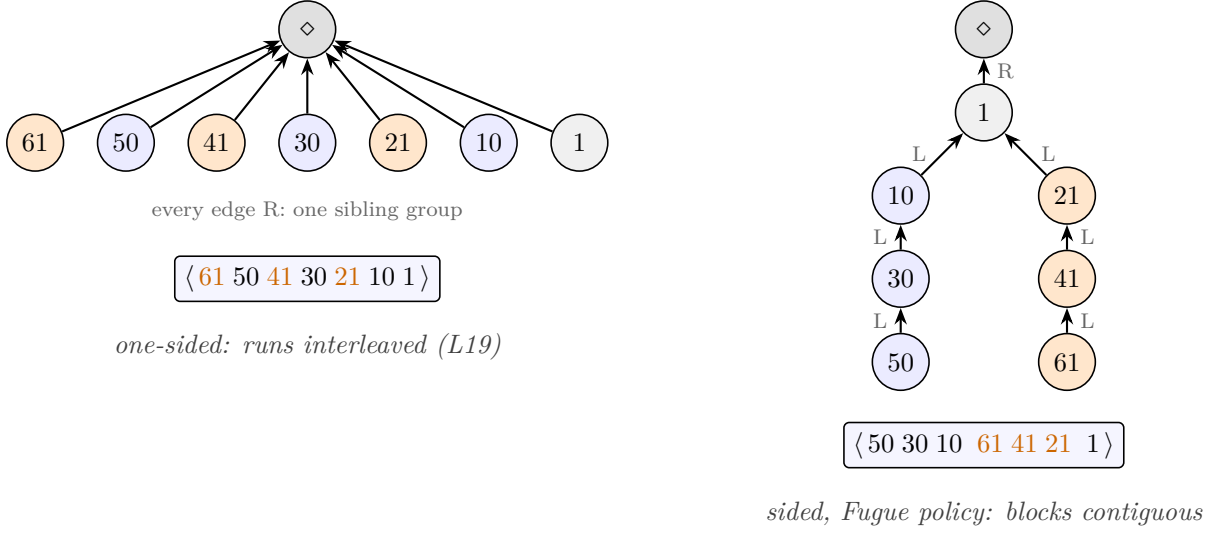


Figure 7: L19 under the two anchoring rules, the battery’s own trace. LCA document $\langle 1 \rangle$; replica A (blue) types backward 10, 30, 50 before the head while replica B (orange) concurrently types backward 21, 41, 61. One-sided (left): every insert-before anchors at the same left neighbor $\diamond$, both runs land in one R-sibling group, and the timestamp-sorted fan interleaves them. Sided under the Fugue policy (right): insert-before anchors at the *successor* as an L-entry, each run is an L-chain hanging off 1, and the in-order read keeps the blocks contiguous: L-extensions precede the node (50, 30, 10 before their parents’ line ending at 1), and among the L-siblings 10, 21 of node 1 the older run displays first (the L band is mirrored). Both display strips are outputs of the sided model harness, `embed_sided.py`.

How the encoding realizes it, for zero extra bits. The one-sided block of §2 spends a constant leading 1 per level whose only job is keeping coordinates nonempty; repurpose it as the side bit. The L band needs an order-reversing prefix-free code, and the bit complement $\overline{C}$ is exactly that at identical lengths (flipping every bit flips every first-difference verdict and preserves both lengths and prefix relations), so the block becomes

$$\text{block}(\text{side}, \delta) = \text{sideBit} \parallel (C(\delta) \text{ if } R, \overline{C}(\delta) \text{ if } L),$$

prefix-free per band, with no framing change and no new bits. On the example, with the binary delta code ($C(1) = 0$, $C(9) = 1110001$, $C(20) = 111100100$, so $\overline{C}(9) = 0001110$, $\overline{C}(20) = 000011011$), the stored coordinates are the concatenated blocks:

node	chain	stored coordinate (side bits marked)
1	$(R, 1)$	<u>1</u> 0
10	$(R, 1)(L, 9)$	<u>1</u> 0 00001110
21	$(R, 1)(L, 20)$	<u>1</u> 0 0000011011
30	$(R, 1)(L, 9)(L, 20)$	<u>1</u> 0 00001110 0000011011
50	$(R, 1)(L, 9)(L, 20)(L, 20)$	<u>1</u> 0 00001110 0000011011 0000011011

To compare two coordinates, parse: read a side bit; if 1, decode one C codeword as an R-band symbol (R, δ) ; if 0, decode one $\overline{C}$ codeword (decode C on the flipped bits) as an L-band symbol (L, δ) ; repeat, then terminate with the marker. Prefix-freeness per band delimits every block,

so no length fields or framing are needed, exactly as one-sided. The complement is what makes the mirror free: within the L band, plain descending comparison of the *stored* bits is already the mirrored delta order ($\overline{C(20)} = 000011011 < \overline{C(9)} = 0001110$ bitwise, matching $(L, 20) < (L, 9)$), so the carrier stores entropy payload only and the marker-lex of the previous paragraph is read off the bits. Faithfulness of this parse (unique decodability plus order embedding of the sided alphabet) is machine-checked by the chain-model/code-model lockstep below.

Two honest cost notes. Runs absorb the side bit like every other run-constant (paid once at the head). On *this* trace the sided coordinates are longer than the one-sided fan’s (the runs’ deltas are 20 because the two replicas’ Lamport clocks interleave); the compression claim is about the common case, continuation-stamped backward typing ($\delta = 1$ per keystroke, an $0\|\overline{C(1)} = 01$ block per level), where today’s design pays a growing- δ fan. Two-sidedness buys the semantics always and the compression exactly when backward runs are locally stamped.

Side selection is a generation policy, not a datatype choice. If insert merely “takes a side,” convergence and RA-linearizability hold for any sides whatsoever: those proofs never consult the order’s intent. What depends on *which* side is picked is the ordering intent. Fugue’s non-interleaving needs the rule “right child of the left neighbor when it has no right children, else left child of the successor,” and that rule lives at generation time, where the framework already has a slot: honesty, the same layer that polices the mint. The architecture is therefore one sided datatype, verified once, policy-free (canonicity, fold exactness, the Join, RA-linearizability), with intent theorems per generation policy: under always-R, read-equivalence to the published RGA (the fragment theorem composed with the compaction theorem); under the Fugue rule, non-interleaving (a subtree is an in-order interval). RGA and Fugue become two disciplines over one kernel rather than two datatypes.

Validation. `whiteboard/litmus/embed_sided.py`: the chain semantics and the bit carrier as lockstep twin models, both run under both policies. Every design prediction survived the first full run:

check	verdict
full gauntlet under the Fugue policy (L1–L25, credential family, subordination CEs, stale forks)	clean, including L19 (out = $\langle 50, 30, 10, 61, 41, 21, 1 \rangle$)
same gauntlet under always-R	clean except L19: the one-sided fan, reproduced
chain semantics $\sim$ bit carrier, both policies	lockstep-EXACT (unique decodability + order embedding)
all-R fragment $\sim$ the one-sided embed model	lockstep-EXACT, every scenario
randomized version-DAG PBT, all four models	120 + 300 clean

The Lean mechanization of both layers (the sided chain-lex kernel and the sided conditioned instance) is kernel-clean; its capstone is the policy/datatype split as a quantifier.

Theorem 5.7 (the sided capstone). *Call a configuration C of the sided instance honest ($SHonest$) when (a) every delete’s target has a visible prior insert, and (b) one global chain assignment exists: some $\text{chainOf} : \mathbb{N} \rightarrow \text{SChain}$ such that every insert event o of C has $\text{chainOf}(t_o)$ positive, stored coordinate $\text{sidedCoordOf}_\Gamma(\text{chainOf}(t_o))$, and deltas telescoping to*

the id: $\sum \text{chainOf}(t_o) = t_o$. Let *SReach* be honest reachability at this contract (Definition). Then every *SReach*-reachable configuration is *RA*-linearizable, per version, for every ordered prefix code and for every side assignment: sides are payload to convergence. The discharge is the direct-join route of § (*s_join_at*); the generation discipline supplies the contract (*sHonest_of_applicable*) and fold canonicity mirrors the one-sided *e_fold_canon* as *s_fold_canon*. *sided_embed_ra_linearizable3*

§12 records the surrounding mechanization inventory.

6 Maximal non-interleaving, closed: the traversal theory and Theorem 9

The sided kernel of §5 hosts the Fugue family, and the Weidner–Kleppmann paper (TPDS’25) supplies that family’s ordering specification. This section states the specification natively, presents the theory that discharges it, and records the closure: all three conditions of the paper’s Definition 4 are kernel-checked for the FugueMax variant, which is, to our knowledge, the first fully mechanized maximal non-interleaving theorem for a sequence datatype. Along the way three findings recur on one theme: **the dead are load-bearing**, in stating the specification, in achieving it, and in proving it.

The specification, natively. Fix an execution. The *left origin* of an element is the element directly preceding its insertion position at the time of insertion (or a start sentinel); its *right origin* is the element directly following the left origin *in the list including tombstones* at that moment (or an end sentinel). The paper’s Definition 4 (*maximal non-interleaving*) demands, over all reachable list states:

- (1) *forward*: if *A* is the left origin of *B* and *B* displays earliest among elements with left origin *A*, then *A* and *B* are consecutive;
- (2) *backward, with exceptions*: if *B* is the right origin of *A* and *A* displays latest among elements with right origin *B*, then *A* and *B* are consecutive, unless the exception holds: *A* and *B* have different left origins and some *C* in the list state sits strictly between *lo(A)* and *B* without being a descendant of *lo(A)* in the left-origin tree;
- (3) *arbitration*: same left and right origins order by lower identifier first.

Both origin definitions and both “earliest/latest among” quantifiers range over the list *including tombstones*: that is the first of the three places the dead carry the statement. The conditions are formalized over reachable configurations of the FugueMax kernel variant (the recorded origins are the generation policy’s, the display order the kernel’s), with the conclusion “consecutive” read over the *visible* document, which is the property an editor’s user sees.

The knowledge model (the generation-layer fold of §1, declared here once). A global configuration is a map *G* from replica to its list of *generation records*; a record carries (ts, rep, op, lo, ro, chain): the Lamport id, the issuing replica, the op, the recorded left origin (0 = start), the recorded right origin (the tombstone-visible successor at mint time; none = end), and the minted birth chain. For the plain Fugue policy the record list is **Know** with reachability **FugueReach** (local positional inserts with globally fresh, locally dominant Lamport

ids; positional deletes; pairwise whole-knowledge sync); for the FugueMax variant, **KnowM** and **MaxReach**, same shape over the tagged records. The readers, per knowledge state K : $\text{lo}(x)$ and $\text{ro}(x)$ read the minted record (**mLoOf**, **mRoOf**); $x \prec y$ is “displays before, tombstones included,” the display comparator on the strong-list keys (**mBefore**); $\text{live}(x)$ is membership in the visible document (**mLive**); $\text{minted}(K)$ the minted ids, tombstones included (**mMintedIds**); left-origin ancestry is the iterated-lo relation (**mLoDesc**). Plain-Fugue twins carry g-names (**gLoOf**, **gBefore**, ...).

Definition 6.1 (adapted Definition 4, the quantified form). *Over a knowledge state K (FugueMax readers shown; the plain-Fugue twins are verbatim over g-readers):*

- (1) *forward (**ForwardNIM**): for all minted A, B , both live, with $\text{lo}(B) = A$ and B display-earliest among minted elements with left origin A (every minted $D \neq B$ with $\text{lo}(D) = A$ has $B \prec D$): A and B are consecutive, i.e. $A \prec B$ with no live c strictly between (**mConsecutive**).*
- (2) *backward, with the exception (**BackwardNIExcM**): for all minted live A, B with $\text{ro}(A) = B$ and A display-latest among minted elements with right origin B : if the exception fails, A and B are consecutive. The exception (**BackwardExceptionM**): $\text{lo}(A) \neq \text{lo}(B)$, and some minted C (tombstones included) has $\text{lo}(A) \prec C \prec B$ and is not a left-origin descendant of $\text{lo}(A)$.*
- (3) *arbitration (**SameOriginLowFirstM**): minted live A, B with equal left origins, equal right origins, and $A < B$ display $A \prec B$.*

$\text{MaxNonInterleaving}(K)$ is the conjunction of the three; the quantified statement $\text{FugueMaxMaximallyNonInterleaving}(\Gamma)$ demands it at every replica of every **MaxReach**-reachable G . The premise quantifiers and both origins range over minted elements, tombstones included; the conclusion is live (the visible document). The plain-Fugue twin of the exception (**BackwardException**) carries one extra conjunct, a live witness: that reading is refuted below and is retained as the negative control.

MaxNonInterleavingM,
FugueMaxMaximallyNonInterleaving

The traversal theory. The paper’s proofs run through a tree traversal (its Lemma 7: the list order of a forward non-interleaving algorithm is a depth-first traversal of the left-origin tree; its Lemma 8 computes origins by walking the algorithm’s own tree). On the embedded-chain family the algorithm’s tree is implicit in the chains, and the marker theorems of §5 already state that the display order is the in-order rule on chains. What the discharges consume is therefore not an emission function but an *interval* form of traversal theory (**Sided_Traversal.lean**): subtree convexity (a chain prefix’s extensions display as one contiguous block around it), the side lemmas (an extension displays after the node iff its first added entry is R-headed, before iff L-headed), prefix strip-and-lift, and transitivity of the chain order. A recursive emission function was designed and deliberately not built: every consumer needs only the interval lemmas, and the emission would add a fueled recursion with no downstream client; the executable emission lives in the Python twin (**traversal_check.py**), checked against the descending-key sort on both policies.

The bridge from recorded origins to chain geometry is one shape invariant, the chain form of the paper’s walk-up rule. *loShape*: every minted element x satisfies

$$\text{chain}(x) = \text{chain}(\text{lo}(x)) \cdot \langle \text{R entry} \rangle \cdot \ell s, \quad \ell s \text{ all-L.}$$

R mints satisfy it with ℓ s empty; for L mints the content is that the tombstone-visible successor of an anchor with an R-child is itself an all-L descendant of that R-child, proved by an argmax argument (an R entry inside the rest would exhibit a minted element displaying between anchor and successor, beating the successor’s maximality; the invariant and its key lemma are `loShape` and `succ_R_desc_allL`, `SidedRGA_NonInterleaving.lean`). `loShape` is a *generation-discipline* fact, not a kernel fact: a forged state that stores an L entry under the wrong parent displays fine (the total order does not care) but breaks the walk-up rule, and condition (1) then genuinely fails; the honesty clause excluding it is precisely that side and parent come from the policy at mint time, never from op payload.

The forward discharge. With `loShape` in hand, the paper’s “first node the traversal visits among A ’s right descendants” is replaced by a *descent*: if a minted c is an R-headed extension of A , then some minted D with $\text{lo}(D) = A$ displays at or before c (strong induction on chain length, comparing the two prefix decompositions of $\text{chain}(c)$). Condition (1) follows: any live c strictly between A and its earliest left-origin child B lands in A ’s subtree by convexity, is R-headed by the side lemma, and the descent produces a D with $\text{lo}(D) = A$ displaying before B , contradicting B ’s minimality. The one new invariant clause, the all-L successor shape for L mints, transports through insert, delete, and sync like the main bundle. Condition (3) was already the positive twin of the Fugue refutation (`fuguemax_same_origin_low_first`: at every replica of every `MaxReach`-reachable G , same-origin pairs display lower id first).

Theorem 6.2 (forward non-interleaving, FugueMax). *For every ordered prefix code Γ : every replica of every `MaxReach`-reachable configuration satisfies condition (1) of Definition 6.1. No hypotheses beyond reachability; every invariant consumed is internal. Kernel axioms only. `fuguemax_forward_ni`*

Everything in the forward proof never consults the sibling order, and the same argument discharges the *plain Fugue* policy’s forward condition as stated: the link layer was rebuilt as an external invariant over an untouched `FugueReach`, with roughly 85 percent of the FugueMax machinery transferring verbatim.

Theorem 6.3 (forward non-interleaving, plain Fugue). *For every ordered prefix code Γ : every replica of every `FugueReach`-reachable configuration satisfies condition (1) in its g-reader form (`ForwardNI`). Kernel axioms only. `fugue_forward_ni` (`SidedRGA_Fugue_ForwardNI.lean`)*

Plain Fugue’s conditions (2) and (3) remain false, as the paper itself proves, and the separation is machine-checked at reachable configurations:

Proposition 6.4 (the plain-Fugue refutations). *At the unary code:*

- *Condition (3) fails: on the reachable two-concurrent-front-inserts configuration (`GTie`), ids 1 and 2 share both origins and display [2, 1]: the kernel’s newest-first sibling order is the exact reverse of the paper’s (by its Theorem 10, not optional) lowest-id-first arbitration. Hence $\neg \text{FugueMaximallyNonInterleaving}(\text{unary})$. `fugue_not_maximally_noninterleaving`*
- *Condition (2) fails, on the paper’s own Figure-7 execution (`KFig`): the pair $(A, B) = (7, 4)$ satisfies the backward premises, the live 6 sits strictly between, and every candidate exception witness is refuted pointwise, so $\neg \text{BackwardNIExc}(\text{unary}, K_{\text{Fig}})$*

(*fugue_backward_gap*); lifted to the full statement at the reachable configuration as *fugue_not_maximally_noninterleaving_backward*. This is the paper’s own *Fugue-versus-FugueMax* gap, so the separation does not hinge on the tiebreak clause.

Both close by *native_decide* on the real fold, under the *SPOT* convention (§1).

The backward condition: the live-witness refutation. The Lean statement of the exception, as first adapted, required the witness C to be *live*. That reading is false, and the countermodel is the paper’s own Figure-7 execution followed by one delete. In that execution (elements here named by their ids) the display is $[3, 6, 7, 4, 5]$ with $\text{ro}(6) = 5$ and $\text{lo}(6) = 3 \neq \text{lo}(5)$; delete 4, so the visible document is $[3, 6, 7, 5]$. The premise pair $(A, B) = (6, 5)$ holds: 6 is the only minted element with right origin 5, hence trivially display-latest among them. The pair is not consecutive: 7 sits between. The exception’s first half holds. But no *live* witness exists: the only elements strictly between $\text{lo}(6) = 3$ and 5 are 6 and 7 (both descendants of 3 in the left-origin tree) and the *dead* 4, which is exactly the C the paper’s own proof produces for this execution. The paper’s Lemma 5 quantifies C over the list state including tombstones, so the corrected definition drops the one liveness conjunct and nothing else. Second load-bearing role of the dead: without them the specification itself is not a theorem of *FugueMax* (the refutation fired on the directed families and on 3 of 4500 randomized delete-enabled states; the earlier insert-only sweeps could never see it). Two remarks. The corrected reading is strictly paper-faithful and strictly weaker as a guarantee: a pair excused by a dead witness is a pair the visible document may interleave; that is forced, not chosen. And the *conclusion* keeps its live reading (no live element strictly between): the visible-document property is the one proved. Per this project’s standing rule, the goal was corrected in the open, forced by a countermodel and matching the paper’s own quantification, never silently renamed.

Replacing causal minimality. The paper’s condition-(2) proof exhibits its exception witness by choosing an in-between element “not causally later than any other,” then consumes that minimality three times. A reachability-invariant formulation cannot replay this: the invariant carries per-record facts about the current knowledge, not a well-founded causal order to minimize over. The replacement is geometric. Three mint-time clauses on R-mint records (a supplementary invariant bundle, transported exactly like the existing ones): a root-anchored R mint records no successor; a recorded successor displays after the anchor; and the two *RSucc* clauses pin, for an R mint whose anchor is itself an L or R mint, a stable witness (the anchor’s parent, a later L-sibling, or the anchor’s own recorded successor) whose key bounds the recorded successor’s (the *RBk* bundle, *SidedRGA_Backward.lean*). With those, a deterministic, state-definable witness function replaces the causal choice: it inspects the in-between element’s chain, walks at most two steps (four routes, exercised as α 253 times, β_1 22, β_0 -direct 2, β_0 -ladder 2 across 4500 validation states, the β_0 routes reachable only by directed construction), and returns a minted element strictly between $\text{lo}(A)$ and B that the left-origin walk can never reach. The methodological point: a *universal* fact about the minter’s knowledge does not transport to later states, but an *existential* witness does, because minted records, chains, and keys are stable under knowledge growth. Third load-bearing role of the dead: the constructed witness may itself be a tombstone (it is, in the Figure-7 family), and the intermediate lemmas are deliberately stated over minted, not live, elements so the ladder case can call them.

The capstone. The backward discharge (`SidedRGA_Backward.lean`, `fuguemax_backward_ni`) closes the arc.

Theorem 6.5 (Theorem 9, mechanized). *For every ordered prefix code Γ : `FugueMaxMaximallyNonInterleaving`(Γ), that is, all three conditions of Definition 6.1 hold at every replica of every `MaxReach`-reachable `FugueMax` configuration. No hypotheses beyond reachability and the code; kernel-clean on $\{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$. Assembled from conditions (1) and (3) (Theorem 6.2, `fuguemax_same_origin_low_first`) and the backward condition (`fuguemax_backward_ni`) by the reduction theorem `fuguemax_theorem9_of_backward.fuguemax_maximally_noninterleaving`*

The reduction theorem and the forward file recompiled with zero edits after the definition correction, as the design phase predicted. SPOTs pin the witness values on all four delete-bearing directed variants, PASS and FAIL shaped.

The convergence capstone, and the TagsOK finding. Ordering intent is only half the variant's bill. The `FugueMax` realization writes records over a *different block format* (R entries carry the mint-time successor's key as an order-reversing tag before the complemented delta code), so RA-linearizability for the tagged datatype was owed separately and is paid: same canonical sorted-list state, same mergeable-queue route, fold canonicity mirrored as `f_fold_canon`. The honesty contract needs one clause the sided contract did not:

Definition 6.6 (wellformed tags). *A right-origin tag t is wellformed when it is the end tag $\langle 0 \rangle$, or a real key: $t = (h \cdot c) \cdot \langle 3 \rangle$ with head $h \notin \{0, 3\}$, $h \leq 5$, and every symbol of c marker-free and bounded ($\neq 3, \leq 5$). The obligation applies to R entries (L entries carry no tag); `TagsOK(ch)` demands it of every entry of the chain.* `TagOK, TagsOK`

Theorem 6.7 (`FugueMax` convergence). *Call a configuration C of the tagged instance honest (`FMHonest`) when (a) every delete's target has a visible prior insert, and (b) one global chain assignment exists: some `chainOf` with, for every insert event o of C , the chain positive, its tags wellformed (`TagsOK(chainOf(to))`, the new clause), the stored coordinate equal to the chain's tagged coordinate (`fmCoordOf`), and deltas telescoping to the id. With `FMReach` honest reachability at this contract (Definition): every `FMReach`-reachable configuration is RA-linearizable, per version, for every ordered prefix code. The right-origin tags ops carry are payload to this theorem, exactly as sides were to Theorem 5.7.* `fuguemax_ra_linearizable3`
(`SidedRGA_FugueMax_RA_Lin.lean`)

The finding worth a name: **the right-origin tag is convergence-load-bearing**, not merely intent-load-bearing. Unique decodability of the variant coordinate format (`fmCoordOf_inj`), which key injectivity and hence fold canonicity run through, is only unambiguous on wellformed tags: the generation layer supplies `TagsOK` for free, but a hand-forged tag would break convergence itself, not just the display intent. Convergence is otherwise insensitive to what the coordinates contain, exactly as the policy/datatype split predicts: coordinates are injectively minted, immutable, and consumed only through key comparisons.

6.1 Where the family sits in the published table

The Weidner–Kleppmann paper’s Table 1 is the nearest published designs $\times$ anomalies matrix: roughly ten list CRDTs classified by hand against the interleaving conditions. The placement this repository contributes is not a new row but a change in how rows are occupied: **one kernel datatype, verified once, whose generation policy selects the row, each placement by theorem.**

policy / variant	row occupied	by which theorem
always-R	RGA	fragment theorem + compaction theorem: read-equal to the published RGA, anomalies <i>included</i> (L19 inherited by theorem)
Fugue policy	Fugue	forward NI as stated (<code>fugue_forward_ni</code>); <i>not</i> maximally non-interleaving: the paper’s own separation, machine-checked on its Figure-7 execution
FugueMax variant	FugueMax	Theorem 9 mechanized (<code>fuguemax_maximally_noninterleaving</code>); convergence separately (<code>fuguemax_ra_linearizable3</code>)

Two readings of the table. First, the one-sided embedded-chain RGA occupies the RGA row *by theorem*: the compaction theorem (§12) makes it the published RGA’s reads verbatim, so every property Table 1 records for RGA, favorable or not, transfers; the design does not get to keep the good rows and disown L19. Second, the convergence column is policy-independent (sides and tags are payload to the merge; the sided capstone quantifies over every side assignment) while the interleaving column is per-policy, which is the policy/datatype split of §5 rendered as table structure. What no policy can do is leave the family’s *delete-insensitive* rows altogether: display keys are immutable and deletion is pure removal for every policy, so a policy selects among the retention-faithful orders and nothing else. The row a splice-based design would occupy is unreachable by policy, and the next section proves that unreachability is not an accident of this kernel but an information-theoretic fact.

7 The chain price: what delete must remember

The conservation law of §1 (the sort key must retain dead ancestors’ timestamps) was stated as an empirical regularity of the design sweep: every design that forgets them has a named countermodel. This section upgrades it to a characterization. The question that forces it: the embed RGA works, but its proof rests on keeping a deleted node’s memory alive inside live descendants’ coordinates; could a *sibling-edge* implementation, which splices a dead node out and lifts its children a level up, be shown equivalent to the embed RGA with no such retention? The answer is no, by a four-event fooling pair (`Refutations/SiblingSplice_Fooling.lean`), and the counterexample names exactly the information the splice destroys.

The mechanism. In the embed order, a lifted child displays at its dead ancestor’s rank: the subtree keyed by the ancestor’s stamp holds its *slot*, and the child sits in that slot. Splice-and-lift erases the ancestor and re-ranks the child among its new siblings by the child’s *own* stamp.

Whenever a concurrent sibling's stamp lies strictly between the dead ancestor's and the child's, the two rankings disagree, and no trace of the disagreement survives in the spliced state.

The fooling pair, walked. Timestamps only; slot keys shown. Both worlds share the LCA and the concurrent branch B ; only branch A differs, and its two variants land on *equal* spliced states.

	world 1	world 2
shared LCA	ins a (stamp 1) at the root	same
shared branch B	ins c (stamp 2) at the root, concurrent with all of A	same
branch A	ins b (5) <i>under</i> a ; del a (6)	del a (4); ins b (5) at the root
spliced A -state	the one-node forest $[b(5)]$	$[b(5)]$: equal , machine-checked
b 's slot key (embed)	1 (dead a 's slot)	5 (its own)
owed read after merge	$[c, b]$ ($1 < 2$)	$[b, c]$ ($5 > 2$)

The knob is the stamp crossing $a < c < b$ with b and c on *different* branches, and it rides on concurrency: B saw only the LCA, so its stamp 2 is Lamport-honest, and every stamp on A exceeds everything its issuer observed ($1 < 5 < 6$; $1 < 4 < 5$). Sequentially the crossing is impossible (a later insert has a larger stamp than everything visible, the dead anchor included), which is why no sequential test can see this and why the pair had to be built on a version DAG. The impossibility is then three lines: a ternary merge function on the splice representation receives *identical* arguments in the two worlds (the LCA and branch B verbatim, the A -states by the machine-checked equality) and owes *different* answers, so there is none.

Proposition 7.1 (the fooling pair: no merge function over spliced states). *Let w_L and w_B be the spliced LCA and branch- B states, w_{1A} , w_{2A} the two worlds' branch- A states, and ρ_1 , ρ_2 the two worlds' embed histories (unary code; payloads 101/105/102 for $a/b/c$; read is the element projection of the fold). The supporting pins, expected values hand-derived:*

$$w_{1A} = w_{2A}, \quad \text{read}(\text{eFold } \rho_1) = [102, 105], \quad \text{read}(\text{eFold } \rho_2) = [105, 102]$$

(*states_equal_A*, *embed_read_w1*, *embed_read_w2*; the FAIL pin *embed_reads_differ* is the inequality of the two reads). The impossibility, quantified over every merge function on the splice representation: for all $f : \text{St} \rightarrow \text{St} \rightarrow \text{St} \rightarrow \text{List } \mathbb{N}$,

$$\neg (f \ w_L \ w_{1A} \ w_B = \text{read}(\text{eFold } \rho_1) \ \wedge \ f \ w_L \ w_{2A} \ w_B = \text{read}(\text{eFold } \rho_2)).$$

The proof is the three lines above; the pins close under the SPOT convention (fold (iii), §1). *sibling_splice_no_merge_function*

Finding (the retention characterization). *The fooling pair (Proposition 7.1) is a lower bound: any state representation that forgets a lifted child's dead slot key admits the pair, so no merge over it can reproduce the embed's reads. The capstone is the matching upper bound: keeping the whole dead chain suffices for everything (embed_ra_linearizable3, the compaction theorem, the sequential spec, delete-order preservation; §12). Between them: the chain of dead slot keys is exactly the information content of delete. Deletion's semantic content in this family is "the subtree keeps its slot," and the slot chain is both necessary and sufficient to honor it. What*

remains negotiable is only where the memory lives (coordinate prefixes here, carried spine paths in the validated sibling-edge design, tombstone records in the published RGA: three encodings of the same bits) and how many bits it costs (§2, §8); whether it lives is not. This is the pairing of two mechanized bounds, not a single Lean name.

The two-by-two. Retention crossed with display rank sorts the four named designs of this document’s arc:

design	retains the dead chain?	lifted child ranks by	verdict
embed RGA	yes: coordinate prefix	dead ancestor’s slot key	capstone + seq. spec
RGA _† (tombstoned)	yes: tombstone records	dead ancestor’s slot key	read-equal to embed (compaction theorem)
rehoming (flat TF)	truncated at each rehomings	own stamp, after the climb	RA-lin; seq. spec <i>refuted</i> (Part III)
Shesha (splice)	no	own stamp, after the splice	no merge function (the fooling pair)

Reading down the rows: full retention with slot-key display gives the two read-equal designs (tombstone-free and tombstoned differ only in encoding); truncating the chain at each rehomings preserves convergence but breaks the sequential specification; discarding it altogether leaves nothing for any merge function to work with. The verdicts degrade exactly as the retained information does.

The delete-sensitivity separation. §6.1 showed the generation policy selecting rows of the interleaving table. Here is what no policy can select.

Theorem 7.2 (delete never moves survivors). *For every ordered prefix code Γ , every state s , and every delete operation: $\text{eUpdate } \Gamma \ s \ (t, r, \text{del } x)$ is a sublist of s (the survivors, in unchanged order). No hypotheses: no honesty, no reachability, no policy. The proof is one line (`List.filter_sublist`); the theorem carries the separation because it is unconditional.*
`eUpdate_del_sublist`

For every policy over the retaining kernel, display keys are immutable and deletion is record removal, so a delete never moves survivors (Theorem 7.2 is policy-free): the whole family is *delete-insensitive*. Shesha’s row is *delete-sensitive* (the splice moves survivors), and the fooling pair shows the sensitivity is not a stylistic difference but an information-theoretic one: the splice state cannot support the retention family’s reads at all. Policies span the interleaving axis; nothing spans the retention axis. The two degrees of freedom are orthogonal, and only one of them is free.

No coding escapes. Could a cleverer coordinate code rescue the splice? No, in both directions. Downward: the impossibility theorem quantifies over *every* merge function on the splice states, and an encoding is a function of the state, so any coded splice inherits the fooling pair verbatim (the two worlds’ encodings are equal bit for bit, being encodings of equal states). Upward: the positive theorems are parametric in the code (§12: one proof, three instantiated codes), so the entire coding freedom provably lives in what retention *costs*, never in whether it happens. Coding moves bits; it cannot manufacture information the state has discarded. That

is the chain price: the storage layer of the next section spends considerable effort making the price small, and none trying to evade it.

8 The storage layer: re-coding, compaction, and the run table

The datatype’s costs were settled upstairs (§2: pay the entropy of the anchor gaps at mint time) and its obligations downstairs (§7: the dead chain must be retained). This section is the layer between: what a running system does about accumulated weight. The arc has three rungs, and stating them up front keeps the bookkeeping honest. *Chains carry the semantics*: birth constants, never edited, the objects of every correctness theorem. *Runs carry the representation*: an invertible re-encoding of the current state, valid with no hypotheses at all. *Stability is needed only for forgetting*: epoch maps that genuinely discard dead history rewrite geometry, and rewriting is safe only at a settled cut (§). Every measurement below reproduces from the cited harnesses, and each table’s accounting model is stated with it.

8.1 Re-coding at a settled cut: the theorem cluster

The epoch problem. Coordinates are birth constants and never shrink: a live record whose birth chain passed through nine deleted ancestors carries all nine dead codewords as a prefix forever. Tombstone-freedom removed the dead *records*; their *codewords* survive inside every descendant, so long editing accumulates coordinate weight proportional to total history, not to the live document. A GC epoch should re-code: assign live records fresh, short coordinates inducing the same display order, and let new mints extend the fresh coordinates.

Lazy translation, and why it is the only nontrivial formulation. Replicas cannot switch codes in lockstep: a message minted against old coordinates may be in flight while the receiver has already compacted. The model is therefore *lazy translation*: the re-map is a pure function on coordinates, applied once to the state at rest and to each incoming record or op minted in the old code (the op’s carried anchor prefix is translated; its delta codeword is not touched). Formally, fix a set S of *stable* coordinates, downward closed under the ancestor relation. Downward closure is not a design choice but a fact about cuts: a coordinate’s proper codeword-prefixes are exactly its ancestors’ coordinates, ancestors causally precede their descendants, and any causally downward-closed notion of stable therefore contains the ancestors of everything it contains. Given a compaction ρ on S , the induced re-map factors every coordinate at the cut:

$$\hat{\rho}(c) = \rho(\text{stab } c) \parallel \text{rest } c,$$

where $\text{stab } c$ is the longest stable prefix (it is exactly one chain prefix, again by downward closure) and the unstable tail is kept verbatim. The theorems consume two hypotheses, both relativized to the coordinates *at hand* (convention (v), §1), never to all bit strings; injectivity is free. The Lean bundle (`EmbedRGA_Recoding.lean`, payload- and code-generic, kernel-clean):

Definition 8.1 (the stable-prefix re-map bundle). *A stable-prefix map for Γ is $F = (f, \text{Rest}, \text{MintAt}, \text{ext}, \text{ord})$: a coordinate map f , the at-rest domain Rest (the cut state’s records), the declared beyond-cut mints MintAt , and two laws:*

(H3) *extension commuting, guarded by MintAt : $\text{MintAt } \pi \ d$ implies $f(\pi \cdot \Gamma.\text{enc}(d)) = f(\pi) \cdot \Gamma.\text{enc}(d)$;*

(H2) order preservation on the domain: for c, c' at hand, $\text{keyLt}(\text{key}(f c))(\text{key}(f c')) = \text{keyLt}(\text{key } c)(\text{key } c')$, a Boolean equation carrying both directions of the display comparator.

“At hand” ($F.\text{Dom } c$) is $\text{Rest } c \vee \exists \pi d, \text{MintAt } \pi d \wedge c = \pi \cdot \Gamma.\text{enc}(d)$. There is no injectivity field. *StablePrefixMap, StablePrefixMap.Dom*

Lemma 8.2 (H1 is derived). For c, c' at hand with $f c = f c' : c = c'$. (H2 at the pair, plus totality and irreflexivity of keyLt and injectivity of key : a strict first-difference order admits no two-sided tie except equality.) *StablePrefixMap.injOn*

Writing remap_F for the translation of records, states, and ops by $F.f$ (eRemapSt , eRemapOp : an op’s carried anchor prefix is translated; its delta codeword and deletes are untouched):

Theorem 8.3 (T1: cut-parametric fold congruence). For a stable-prefix map F , cut state s , and continuation τ with s at hand and τ declared at F :

$$\text{applySeq } (\text{remap}_F s) (\tau.\text{map } \text{remap}_F) = \text{remap}_F (\text{applySeq } s \tau) :$$

folding the translated continuation over the translated cut state is the translation of the untranslated fold. *eRecode_applySeq*

Proposition 8.4 (the global-collapse negative). Stating T1 over the whole history (mapping every operation ever issued and re-folding from the initial state) forces H3 at every historical mint. If a coordinate map f satisfies H3 at every positive mint ($\forall \pi d, 1 \leq d$ implies $f(\pi \cdot \Gamma.\text{enc}(d)) = f(\pi) \cdot \Gamma.\text{enc}(d)$), then for every positive chain ch

$$f(\text{coordOf}_\Gamma ch) = f(\varepsilon) \cdot \text{coordOf}_\Gamma ch :$$

a constant-prefix prepend, which compacts nothing. Re-coding is inherently an epoch operation on states, not a history rewrite; the lazy, cut-parametric formulation is the only nontrivial one. A finding, mechanized; kernel-clean (a theorem over all such maps, not a SPOT). *eRecode_ext_global_collapse*

Theorem 8.5 (T2: reads identical). Under exactly the hypotheses of T1 (s at hand, τ declared at F), the query of the translated run equals the untranslated one:

$$\text{query}(\text{applySeq } (\text{remap}_F s) (\tau.\text{map } \text{remap}_F)) = \text{query}(\text{applySeq } s \tau) :$$

compression is invisible to every observer. *eRecode_reads_identical*

The contentful half of T2 hides in T1: the translated side re-sorts by the *new* keys, so equality of reads is exactly the statement that the new keys induce the old order, H2 doing its job. The negative control earns its keep here: an injective, H3-satisfying but order-breaking map genuinely flips the read (the **badMap** FAIL companion of the file’s SPOT), pinning H2 as the load-bearing hypothesis.

Theorem 8.6 (T3: the capstone’s witness statement transports). *Hypothesis*: $\text{EReach } \Gamma C$ (honest reachability, §12); no at-hand hypothesis. For every registered version $C.\text{ver } v = (s, E_v)$ there is a witness enumeration π with: π a permutation of E_v (*listPermOf*); π respecting the delivery order of C ’s core; and

$$\text{remap}_F s = \text{remap}_F (\text{eFold}_\Gamma \pi), \quad \text{query}(\text{remap}_F s) = \text{query}(\text{eFold}_\Gamma \pi).$$

Canonicity, sortedness, and version-level sortedness transport too, each under its own side conditions: canonicity needs EWf of both enumerations, a membership condition, and declared mints (`eRecode_fold_canon`); sortedness needs the at-hand hypothesis of convention (v) (`eRecode_sorted`); version-level sortedness needs EHonest in addition to EReach (honesty at the endpoint, which honest reachability alone does not supply) plus at-hand (`eRecode_version_sorted`). `eRecode_ra_transport`

Honest scope note. The transport is at the level of states and witnesses, not configurations: the re-coded history is *not* itself an honest configuration of the same datatype (compacted coordinates deliberately forget dead chain segments, so the chain-generation honesty clause fails for them). Re-basing honesty across the epoch, so that the capstone re-invokes natively in the new epoch, is closed by the $(*)$ cluster of §8.2 (Definition 8.21 through Corollary 8.25); what stays open downstream is the multi-replica half (agreement on a common cut, Open Question 25). The cut hypothesis itself is discharged by §’s SettledAt gate: `eRecode_settled_bridge` (in `EmbedRGA_Stability_Bridge.lean`, with the stability import, not in the re-coding file) packages T1+T2 at a settled cut against the named residue `AnchorsFactorBeyond`, and is superseded by the residue-free honest form (Theorem 8.11).

Measurement 1: the microbenchmark. Directed, hand-computed worked example (unary code; history: insert a at the root, b under a , c under b , delete a and b ; then, beyond the cut, insert d under c and e at the root). Accounting: the numbers are *coordinate bit lengths of live records*, order metadata alone, identical convention to the entropy tables of §2; characters are excluded.

live record	original coordinate	re-coded
c (two dead codewords carried)	6 bits	2 bits
d (minted beyond the cut, under c)	10 bits	6 bits
e (minted beyond the cut, at the root)	8 bits	8 bits
total	24 bits	16 bits

Reads are identical on both sides (hand-checked and pinned as SPOTs); the dead-chain share of c and d ’s bits is gone entirely, and e , which never passed through the dead region, is untouched. The same harness’s delete-heavy directed case (a 200-deep chain, binary code) drops live coordinate weight from 200 bits to 1 bit, the **200×** figure quoted in the arc’s motivation; 500 randomized histories (random cut, two replicas forked beyond the cut with concurrent ops and a merge at the end) pass reads-identical and the T1 record-for-record correspondence at every step, and the order-breaking negative control flips the read exactly as predicted.

8.2 The verified concrete compaction, and what real traces say

compactEliasDelta. The theorem cluster is parametric in the re-map; the concrete v1 compaction instantiates it (`EmbedRGA_CompactEliasDelta.lean`): (a) *drop dead ranges* (a stable chain is carried forward iff a live descendant or a declared in-flight coordinate passes through it; everything else is dropped and its sibling rank reclaimed) and (b) *rank-renumber* (in each sibling group the surviving deltas are renumbered to $1..k$ order-isomorphically and re-encoded with the same ordered prefix code). Its H2 discharge runs the per-group order isomorphism through

the chain-lex theorem, with renumbered ranks dominated by Lamport-fresh future deltas; the theorem discharges every obligation at a settled cut of an honest configuration.

Theorem 8.7 (the concrete compaction preserves every read at a settled cut). *Let C be a configuration at the Elias- δ code with (i) EReach C (honest reachability, §12), (ii) GenHonest at eApplicable (the generation discipline), and (iii) CausalPastEnumerable C ; let (iv) $C.\text{ver } v = (s, E_v)$ be a registered version with (v) SettledAtOn C E_v S a settled cut (§). Let tree, live, inflight be the compactor's inputs with (vi) every tree chain positive (h_{tpos}); (vii) tree exactly the settled inserts' chains, in both directions (h_{Tree} : every settled insert's coordinate is some tree chain's coordinate; h_{TreeS} : every tree chain is some settled insert's); (viii) h_{Live} : a tree chain whose insert no delete of S targets carries live = true; and (ix) h_{Inflight} : every insert of E_v escaping S is declared in inflight with its (positive) chain. Then there is a stable-prefix map F with*

$$F.f = \text{fStab}(\text{rED}(\text{ePruneKeep tree live inflight}) \text{ inflight}) (\text{ePruneKeep tree live inflight})$$

(drop dead ranges, rank-renumber outside **skipAt**-guarded groups, re-encode) such that every continuation τ of events existing beyond the cut ($o \in C.\text{events}$, $o \notin E_v$) satisfies both the T1 fold congruence and T2 reads-identical of §8.1. **compactEliasDelta_settled_reads**

Remark 8.8. Display compressions, for the record: the mechanized statement phrases (vii)–(ix) per operation, over eIsIns $o = \text{true}$ with coordinate matching coordOf $k = \text{eCoord } o$; each per-operation quantifier there additionally ranges over $o \in C.\text{events}$ (a conjunct the display drops: it is redundant under hypothesis (i), which bounds every registered version's events by $C.\text{events}$ through GoodConfig3, but it is present in the mechanized statement); and (viii)'s undeleted clause compares payloads, $\forall dl \in S, dl \neq \text{del}(o.1)$ read at the payload of dl . The theorem is the Elias- δ instantiation of the code-parametric engine **compactRanked_settled_reads**, whose hypothesis list is identical.

The in-flight wrinkle, pinned both ways. Renumbering a sibling group is only an order isomorphism against the deltas it can see. If an op minted in the old code is still in flight with a delta in that group, dense renumbering can slide a surviving sibling past it: the FAIL SPOT pins exactly this (an in-flight root insert; the unguarded renumbering sends a surviving delta $9 \mapsto 2$, and the delivered op lands on the wrong side: the pinned read flips). The v1 resolution is the guard **skipAt**: sibling groups with a known in-flight child delta are skipped this epoch. Both pins are hand-derived, PASS and FAIL shaped.

Proposition 8.9 (the in-flight wrinkle, pinned both ways). *On the file's directed cut (root sibling group $\{1, 2, 3, 4, 9\}$, survivor 4, dead-but-kept 9; an epoch-0 in-flight root insert with chain [6], payload 110; hand-derived control read [106, 110, 104]):*

*FAIL the unguarded dense renumbering (the tempting degenerate, **rED** fed an empty in-flight list) yields*

$$\text{read}(\text{unguarded}) = [110, 106, 104] \neq [106, 110, 104] = \text{read}(\text{control}) :$$

*9 $\mapsto$ 2 slides the compacted survivor under the in-flight 6 and the late op jumps to the front (**unguarded_flips_order**);*

PASS the guarded construction skips exactly the root group (`skip_guard`: `skipAt = true` there, false at the in-flight-free [9] group), keeps the read pinned at [106,110,104], and still compresses: coordinate weight 19 against the control's 23 (`guarded_skips_group`).

unguarded_flips_order, guarded_skips_group

AnchorsFactorBeyond: the residue is a theorem. The bridge from SettledAt to the cluster left one named residue: every op minted with the cut in its causal past must carry an anchor prefix that factors at the map. This is *not* dischargeable from bare honest reachability (countermodel: the chain-generation clause constrains the minted coordinate, not the carried split, so a dishonest split satisfies it while defeating the factorization). It *is* a theorem of the generation discipline: under the framework's honest generation (the same hypotheses the capstone's honesty discharge consumes), every insert's carried prefix is the stored coordinate of an actual anchor record in the fold of its causal past, so the factorization sits on a codeword boundary by construction, and the bridge is restated residue-free.

Theorem 8.10 (the residue is a theorem of the discipline). *Call (π, δ) a canonical chain-aligned mint beyond the cut S (**ChainMintBeyond**) when $1 \leq \delta$ and $\pi = \text{coordOf}_\Gamma(\text{ch}_A)$ for a positive chain ch_A , witnessed by an actual insert event $o \in C.\text{events}$ with S entirely in its causal past ($\forall a \in S$, $\text{vis } a \ o$) and $\text{chainOf}(o.1) = \text{ch}_A \cdot \delta$. Hypotheses: (i) $\text{EAnchored } \Gamma \ C \ \text{chainOf}$ (the anchor certificate the honesty discharge produces); (ii) $F.\text{MintAt}$ contains every canonical chain-aligned mint beyond S . Conclusion: $\text{AnchorsFactorBeyond } \Gamma \ F \ C \ S$, i.e. every op minted with the whole cut in its causal past carries an anchor prefix that factors at the map. The containment (ii) is definitional for the stage-2 construction; the residue is a theorem of the anchored geometry, not an extra hypothesis.*

anchorsFactorBeyond_of_anchored

Theorem 8.11 (the bridge, residue-free). *Hypotheses: a stable-prefix map F ; GenHonest at eApplicable ; $\text{CausalPastEnumerable } C$; $C.\text{ver } v = (s, E_v)$; $\text{SettledAtOn } C \ E_v \ S$; s at hand for F ; and the construction-checkable containment: for every anchored assignment chainOf certified by EAnchored , the canonical chain-aligned mints beyond S lie in $F.\text{MintAt}$. Conclusion: every continuation τ of events existing beyond the cut folds and reads identically under the lazy translation ($T1 \wedge T2$). The **AnchorsFactorBeyond** hypothesis of **eRecode_settled_bridge** is gone; no vis-side condition survives in the interface.*

eRecode_settled_bridge_honest

Scope, honestly delimited: the single-epoch statement, with cut-side data addressed by coordinates and chains, never by event ids resolved through prefix sums (the runtime twin's erratum: renumbered coordinates no longer telescope to ids, so id-addressed cuts break after epoch one). Concurrent divergent compactions (an epoch DAG with common-refinement translation) remain the deferred protocol half.

Spine fusion, mechanized. Rank renumbering alone leaves the *spine depth*: typing runs mint $\delta=1$ chains, and every level costs at least one codeword even at ordinal 1 (Measurement 2 quantifies this). Fusion removes the dead levels themselves. A *fusible spine* is a maximal chain of settled dead nodes, each with exactly one child branch (counting every known coordinate, in-flight prefixes included) and no in-flight op anchored on it; it collapses to a single level at the spine head's codeword, so a typing-run-then-delete chain of depth k costs one codeword instead of k . Composed after the rank pass this is the full iteration-two map, and reads survive it (`EmbedRGA_Fusion.lean`, `kernel-clean`).

Theorem 8.12 (fusion composes with the rank pass, reads intact). *Let F be the rank-pass map. Fusion data: a positive spine Q ($h_{Q_{\text{pos}}}$), its surviving head Q' a prefix of Q ($h_{Q'}$), and a chain domain $\mathcal{D}$ with (i) every $\mathcal{D}$ chain positive ($h_{\mathcal{D}_{\text{pos}}}$) and (ii) through-traffic passes the whole spine (h_{thru} : a $\mathcal{D}$ chain extending Q' extends Q). Let the two maps be compatible at a surviving domain (Rest' , MintAt') (CompatOn , Definition 8.14), and let the cut state s be at hand and the continuation τ declared at the composite $F.\text{comp}$ ($\text{fuseSPM} \Gamma Q Q' \mathcal{D} \dots$). Then the reads of any beyond-cut continuation survive the two-pass translation $\text{fuseBits} \circ F.f$:*

$$\text{query}(\text{applySeq}(\text{remap } s) (\tau \text{ translated})) = \text{query}(\text{applySeq } s \tau).$$

(*fuseSPM packages the fusion as a stable-prefix bundle over fuseBits , its bit-level map; the fusion-alone form is $\text{fuse_reads_identical}$; both are direct re-inocations of $T2$.)*
 $\text{compactThenFuse_reads}$

Order preservation is a three-class H2 argument: (1) within a fused block the common prefix is replaced wholesale, relative order untouched; (2) a block against the spine head’s siblings is decided at the head’s level, whose codeword fusion keeps; (3) the anchor-versus-extension corner is vacuous, since dead spine nodes hold no records and no key ends at a fused-away level. The mechanization sharpens class 2 into triviality: because fusion runs *after* the rank pass, the head codeword it sees is already the ordinal, so the fusion map keeps the head verbatim and drops only the strict interior, and the class-2 comparison is unchanged by construction rather than by a separate “block inherits the rank” argument. The guard is conservative (any known in-flight branch or anchor blocks its node from every spine); through-traffic below a blocked node still fuses.

The merge clause, discharged (VC-S4 for the remap species). The stability bundle’s argumentwise merge congruence (§, VC-S4) is, for the coordinate re-map, the statement that lazy translation commutes with the ternary merge ($\text{EmbedRGA_MergeCongr.lean}$).

Theorem 8.13 (the merge clause). *For a stable-prefix map F and states l, a, b with a and b at hand for F (no hypothesis on the LCA l):*

$$\text{eMergeL}(\text{remap}_F l) (\text{remap}_F a) (\text{remap}_F b) = \text{remap}_F (\text{eMergeL } l a b).$$

*The LCA enters the survival rule solely through its id set, which the re-map preserves; this is what lets a compacted head merge against a full LCA payload, the mixed case § calls the normal one. The VC-S4 packaging: the simulation relation eRemapRel (compacted = translation of full, full at hand) is preserved by the ternary merge (eRemapRel_merge), literally the vc_merge conclusion for the remap species; $\text{embed_merge_remap_congr}$ restates the congruence at $(E \Gamma \alpha).\text{mergeL}.$
 merge_remap_congr*

Multi-epoch composition. A single re-coded configuration is not a native honest configuration (T3’s gap), so a *second* compaction cannot naively re-invoke the cluster. The data-plane half is now closed ($\text{EmbedRGA_MultiEpoch.lean}$, kernel-clean): the semantic residue of one epoch is a stable-prefix map, and maps compose.

Definition 8.14 (epoch composition at the surviving domain). *For maps F (earlier) and G (later), $\text{CompatOn } F \ G \ \text{Rest}' \ \text{MintAt}'$ carries the composite's own at-hand domain, the coordinates surviving to G 's epoch, and demands four boundary clauses: restF ($\text{Rest}' \ c$ implies $F.\text{Dom } c$), restG ($\text{Rest}' \ c$ implies $G.\text{Dom } (F.f \ c)$), and $\text{mintF}/\text{mintG}$ (the same two clauses for declared mints, the later mint at the translated prefix $F.f \ \pi$). Under them $F.\text{comp } G$ is again a stable-prefix map: $f = G.f \circ F.f$ over $(\text{Rest}', \text{MintAt}')$; $H3$ chains through both maps' ext (an epoch-1 mint (π, d) once remapped is an epoch-2 mint $(F.f \ \pi, d)$, the delta codeword untouched); $H2$ chains because order-isomorphisms compose; $H1$ is derived as always. The no-reclaim special case Compat (every F -at-hand coordinate stays G -at-hand after remapping, every F -mint stays a G -mint) collapses the four clauses to two at F 's full domain. The n -fold closure chainSPM composes a whole CompatChain of maps into one, carrying the earliest map's domain. ***CompatOn***, ***StablePrefixMap.comp***, ***chainSPM****

Theorem 8.15 (n -epoch observational preservation). *For a compatibility chain $\text{CompatChain } (F :: F_s)$, cut state s at hand for the first map F , and continuation τ declared at F :*

$$\begin{aligned} & \text{query}(\text{applySeq } (\text{remap}_{\text{chainSPM } (F::F_s)} \ s) \ (\tau \text{ translated through every epoch's map})) \\ &= \text{query}(\text{applySeq } s \ \tau) : \end{aligned}$$

*after arbitrarily many settled-cut compactions, a beyond-all-cuts continuation, its lagging ops translated through every epoch map, reads exactly as the uncompacted run. (The fold-congruence twin is ***multiEpoch_applySeq***; both are direct re-inocations of $T1/T2$ on the composite.) ***multiEpoch_settled_reads****

The subtlety that makes this non-trivial is *rank reclaim*, pinned by the first of two companion negatives; the second records the runtime twin's erratum.

Proposition 8.16 (rank reclaim defeats naive composition). *Unary code. With the epoch-1 map gc_1 ($\text{enc } 3 \mapsto \text{enc } 1$, $\text{enc } 7 \mapsto \text{enc } 2$) and the reclaiming epoch-2 map gc_2 ($\text{enc } 2 \mapsto \text{enc } 1$):*

$$(gc_2 \circ gc_1)(\text{enc } 3) = (gc_2 \circ gc_1)(\text{enc } 7) \quad \text{while} \quad \text{enc } 3 \neq \text{enc } 7 :$$

*a dead coordinate and a live one collide on the reclaimed rank, $H1$ fails, and the composite is not a stable-prefix map on the full first-epoch domain. This is exactly why comp carries the surviving domain. ***naive_composition_collides****

Proposition 8.17 (the id-addressing erratum). *After epoch one a renumbered coordinate no longer telescopes to its event id: with the unary telescope decode unaryId (the count of 1-bits), $\text{unaryId}(\text{enc } 7) = 7$ but $\text{unaryId}(gc_1(\text{enc } 7)) \neq 7$, and an id-addressed second cut at $\{7\}$ resolves to no record of the renumbered state while the coordinate-addressed cut resolves the record exactly. Second cuts must be coordinate- or record-addressed. ***id_addressing_breaks****

The diamond: incomparable cuts, and the barrier removed. Everything above composes cuts that are *comparable* (each a refinement of the last). The distributed question is the incomparable case: two replicas compact at settled cuts S_1 and S_2 that neither refines, then merge. The result is that they merge with *no* coordination, and the finding is that this is not a new algebraic obstruction at all. Since $S_1 \subseteq W$ and $S_2 \subseteq W$ for the join $W = S_1 \cup S_2$, each

leg $S_i \rightarrow W$ is a comparable (nested) step, hence a genuine StablePrefixMap by the same comp closure; the only new content is that the two legs and the one-shot W -compaction *agree*, and agreement is determinism of the W -map in its certificate. The join map is produced by a builder that is a function of a JoinCert alone (surviving coordinates, settled-dead, declared mints), so two replicas that computed the same certificate install the same map:

$$c_1 = c_2 \Rightarrow \mathcal{C}(c_1) = \mathcal{C}(c_2) \quad (\text{certMap_deterministic}).$$

This `congrArg` is the “without coordination” content: equal certificates give equal maps by definition, not by reconciliation. Agreement then upgrades to bit-identical states.

Theorem 8.18 (the epoch diamond at $s1$). *For maps $\text{spm}_{S_1}, \text{spm}_{S_2}, \text{rel}_A, \text{rel}_B, \text{spm}_W$ and a carried domain D , if every record of the cut state s has its coordinate in D , and both relative composites agree with the one-shot on D ($\text{rel}_A(\text{spm}_{S_1} c) = \text{spm}_W c$ and $\text{rel}_B(\text{spm}_{S_2} c) = \text{spm}_W c$ for all $c \in D$), then the two-leg compactations and the one-shot compaction produce bit-identical states, not merely equal reads:*

$$\text{remap}_{\text{rel}_A \circ \text{spm}_{S_1}} s = \text{remap}_{\text{spm}_W} s = \text{remap}_{\text{rel}_B \circ \text{spm}_{S_2}} s,$$

and their reads coincide.

diamond_confluence (via diamond_confluence_comp through comp)

The one genuinely new obstruction is the carried domain. It must be the join cut’s kept id-set transported to epoch-0 coordinates *by record identity*, $\text{transportedDom } s \text{ kept}_W = \{c : \exists x \in s, \text{kept}_W(x.\text{id}) \wedge x.\text{coord} = c\}$ (`transportedDom`, coordinate-nodup on a sorted state by `transportedDom_coords_nodup`, so it never decodes a renumbered coordinate and the id-addressing erratum does not apply). The naive membership pullback aliases, the incomparable-cut twin of Proposition 8.16:

Proposition 8.19 (naive pullback aliases). *Unary code. A dropped coordinate falls through a leg’s map verbatim onto a kept later coordinate: $(\text{rel} \circ \text{gc})(\text{enc } 3) = (\text{rel} \circ \text{gc})(\text{enc } 4)$ while $\text{enc } 3 \neq \text{enc } 4$. Intersecting the two legs’ kept sets would admit the aliased coordinate; the record-identity transported domain excludes it.* *naive_pullback_aliases*

Two more obligations close the protocol. Dropping a superseded epoch’s map is a fold no-op once that map fixes every live coordinate and every continuation anchor (the `ack-plus-AllHeardSince` certificate supplies exactly these hypotheses; the `ack-only` shortcut is machine-refuted as its `FAIL` companion):

$$\text{applySeq} (\text{remap}_F s) (\tau.\text{map } \text{remap}_F) = \text{applySeq } s \tau \quad (\text{mapDrop_sound}),$$

and continuation freshness survives the join epoch (dense renumbering never grows a delta, so a fresh delta stays key-distinct from every renumbered ordinal, `contOK_root_key_fresh`, `rED_fresh_dominates_join`) — the theorem face of the 11029/0 `ContOK` observation in the model harness. The model (`whiteboard/epoch_diamond_check.py`) validated the whole diamond at $s1$ across 2400 trials before any of this was stated.

Remark 8.20 (what the runtime realizes, and an honest gap). *The shipped runtime (`runtime/src/epoch.js`) turns the per-replica epoch integer into a cut-indexed DAG and replaces the old cross-epoch throw with the certificate-determined join. The model’s “relative-compact each side to W , merge in epoch W ” is unsound under partial straggler visibility: a*

settled record's group freezes on the side holding a straggler and rennumbers on the side without it, so the two sides disagree on shared coordinates. The sound protocol lifts both heads down to their common base (the greatest lower bound) via inverse coordinate maps and merges there; this is read- and fold-equivalent to the W-merge by Theorem 8.18, and coordination-free (both sides lift deterministically to the same frame). The merged head therefore de-compacts to the base frame, recovering compactness only on the next certified compaction: a compaction frame is minted solely by the content-addressed `compactStable`, never re-derived at a merge from divergent local holdings, which is arguably the correct discipline rather than a compromise. The twin property-based test is clean across the 2000-trial validation sweep, of which 600 are committed to CI (`runtime/test/epoch.test.js`).

Freshness across epochs is representation-agnostic (`rED_le_self`, `rED_fresh_dominates`: a delta fresh against original sibling deltas stays fresh against renumbered ordinals without inspecting their meaning), so the order-iso half composes whether stored deltas are timestamp differences or epoch- k ordinals. The `StablePrefixMap` bundle is not text-specific: the rich-text marks layer instantiates the same interface a second time as its keep-set relabeling, and composes through the same route (§9).

The honesty-rebasing lemma (*), closed. Two residues above — the single-epoch honesty re-basing (T3's gap: the re-coded history is honest only *modulo* the coordinate order-embedding) and the multi-epoch `CompatChain` residue — are the *same* obligation. Its causal-stability half (`nested_cuts_settled`, from `settledAt_of_allHeard` and cut-antitonicity of the frontier, `EmbedRGA_CompatChain.lean`) and its order-iso freshness half (`rED_hocc_fresh`) closed first; what remained was one lemma, which we called (*): exhibit an epoch-2 configuration honest *modulo* the order-embedding F_1 , so that anchored-existence re-derives at the epoch boundary. (*) is now discharged (`EmbedRGA_HonestyRebase.lean`, kernel-clean), and the shape of the discharge is itself the finding: honesty is not *re-derived* at epoch 2, it is *re-based*. The invariant that crosses the boundary is not `EHonest` (false of a compacted state, whose coordinates are no longer birth-chain telescopes) but a history-free *coded-forest* invariant I over the live coordinate set: honest configurations satisfy I (S1), every chain-aligned stable-prefix map preserves I (S2), and I alone discharges the epoch-boundary obligations of the next epoch's map (S4), with no epoch-2 honesty assumption anywhere.

Definition 8.21 (the coded anchored forest invariant I). *For a code Γ and a list L of live coordinates, $I(L)$ (`CodedAnchoredForest  $\Gamma$   $L$`) has two fields: (nodup) L is duplicate-free; (decode) every $c \in L$ is `coordOf $_{\Gamma}$ (ch)` for some nonempty positive birth chain ch . History-free: a predicate on the coordinate set alone, which is the whole point — a re-coded configuration has no honest history but still carries I . (The nearest-live-ancestor forest is reconstructed from I by prefix-freeness alone: `caf_ancestor_factor`, `caf_ancestor_shorter`, `caf_chain_inj`.)*
`CodedAnchoredForest`

Theorem 8.22 (S1: honest configurations satisfy I). *Hypotheses: `GenHonest` at `eApplicable`; `CausalPastEnumerable` C ; a well-formed enumeration `EWf  $\Gamma$   $\rho$` ; every `op` of ρ in C .events. Conclusion: I of the live coordinate set of `eFold $_{\Gamma}$   $\rho$` . (The anchor certificate gives each live insert's coordinate as its chain's; nodup is sortedness; nonemptiness is the nonzero id.)*
`ehonest_implies_I`

Call a map F *chain-aligned* via φ (**ChainAligned**) when $F.f(\text{coordOf}_\Gamma \text{ } ch) = \text{coordOf}_\Gamma(\varphi \text{ } ch)$ on positive chains and φ preserves positivity and nonemptiness: the stable/unstable split lands on a codeword boundary and the tail rides verbatim. Both shipped passes are chain-aligned (**chainAligned_compactSPM**, **chainAligned_fuseSPM**), and chain-alignment composes (**chainAligned_comp**).

Theorem 8.23 (S2, load-bearing: stable-prefix maps preserve I). *Hypotheses: F chain-aligned via φ ; $I(L)$; every $c \in L$ at hand for F . Conclusion: $I(L.\text{map } F.f)$. Decodability survives through φ ; distinctness survives because F is injective on its domain (H1, Lemma 8.2). Honesty rebases: no re-derivation of EHonest on the re-coded set. **stablePrefixMap_preserves_I***

Theorem 8.24 (S4: the epoch boundary discharged from I). *Hypotheses: epoch-1's map F_1 chain-aligned via φ_1 ; a domain $\mathcal{D}_1$ of positive chains; and the two construction-shape clauses, true of the shipped compaction and fusion maps: $h_{\text{RestShape}}$ (every $F_1.\text{Rest}$ coordinate is coordOf of a $\mathcal{D}_1$ chain) and $h_{\text{MintShape}}$ (every declared mint (π, d) has $\pi = \text{coordOf}(ch_A)$ with ch_A positive, $ch_A \cdot d \in \mathcal{D}_1$, $1 \leq d$). Conclusion:*

$$\text{CompatOn } F_1 \text{ (rebaseSPM } \Gamma \mathcal{D}_1 \varphi_1 \dots) F_1.\text{Rest } F_1.\text{MintAt} :$$

*the epoch-boundary obligations restG/mintG of Definition 8.14 are met by epoch 2 built over the rebased image domain (**rebaseSPM**, **rebasedDom**: the φ_1 -images of epoch-1's at-hand chains), a survivor's image at hand by chain-alignment and a beyond-cut mint's image a declared mint by the snoc-commute (**chainAligned_snoc**). No epoch-2 honesty assumption; this is $(*)$, closed. (I is also exactly the at-hand certificate a next-epoch compaction's domain needs: $I_implies_atHand$, S3.) **twoEpoch_compatOn_of_I***

Corollary 8.25 (I rebases across two epochs). *For F_1, F_2 chain-aligned via φ_1, φ_2 , $I(L)$, and the two domain coverages: $I((L.\text{map } F_1.f).\text{map } F_2.f)$ — the invariant survives both compactions, so the second epoch operates on the I -carried rebased domain, not by transport through the first. **$I_preserved_two_epoch$***

What is left downstream is the protocol layer: deriving the per-continuation hypotheses from a delivery protocol, and everything multi-replica (agreement on a common cut, concurrent divergent compaction), Open Questions 25, 25.

Measurement 2: real traces, and where the bits actually sit. The full epoch stack (rank renumbering plus *spine fusion*: collapse each maximal settled dead chain with exactly one child branch and no in-flight anchor into a single level) was run over the standard collaborative-editing replay corpus (the josephg traces). Accounting: per record, the sum over its kept-chain levels of $|D(\delta)|$ bits, D the flipped Elias- δ code of §2; totals divided by live characters; character payloads excluded; “fused” is after the renumber-plus-fusion epoch at a settled cut.

trace (one-sided)	chain b/ch, raw	chain b/ch, fused	ratio
friendsforever	1622.5	856.5	1.9×
clownschool	2488.7	1471.0	1.7×
seph-blog1	2974.9	917.2	3.2×
automerge-paper	2304.5	1279.4	1.8×

Three findings, in causal order. (i) Rank renumbering alone recovers only 1.1–1.8×: typing runs mint $\delta=1$ chains, and every level costs at least one bit even at ordinal 1, so the residual cost is *spine depth*, which is what fusion owns. (ii) Fusion then removes essentially everything removable: surviving dead levels cost 3 to 76 bits per character in total, and the fused column equals the code cost of the *live tree shape* alone, verified by a per-level attribution that accounts for every bit. (iii) The few-bits-per-character hope fails for a reason worth the number: the live shape is itself expensive. Real traces anchor typing runs in *live* chains of mean depth 835 to 1468 levels at about one bit per level, and 97 to 99.8 percent of the post-fusion bits sit on live ancestor levels that no epoch map may touch (they are the retained semantics, §7). History independence holds three ways (staggered cuts, one final cut, and from-scratch compaction land on bit-identical totals), and the fused display spells the trace’s final content on every sequential trace. Conclusion: the epoch stack removes the dead-history share; what survives is a *representation* cost, which is the run table’s brief (§8.3).

Measurement 3: the sided instantiation. The same stack, band-aware (renumber within each parent-band group; fusion carries the head’s band and codeword), replayed under the Fugue policy. Same accounting, plus one side bit per level, which is precisely the point:

trace (sided, Fugue)	chain b/ch, raw	chain b/ch, fused	ratio
friendsforever	2582.0	1735.7	1.5×
clownschool	4058.5	2988.1	1.4×
seph-blog1	5559.1	1919.8	2.9×
automerge-paper	4448.0	2583.7	1.7×

The ratios sit *systematically below* the one-sided 1.7–3.2×, and the naive expectation that the ratio would be preserved is falsified by its own arithmetic: adding the same flat per-level side cost to numerator and denominator pulls the quotient toward the *level-count* ratio, which is below the payload ratio. Where the sides pay: the fused sided state costs about twice the fused one-sided state, and the premium is *entirely the flat side flag*; the sided delta payload exceeds the one-sided payload by only 1.0 to 4.7 percent (on these traces the Fugue tree is payload-equivalent to RGA’s). The flag costs a flat 1.000 bit per level against a true side entropy of 0.19 to 0.27 bits (L-mints are 3.0 to 4.6 percent of mints under Fugue on real traces): a **4–5× overpay** on the side channel, load-bearing for lexicographic band separation, and 94 to 98 percent of the side bits sit on live levels no epoch map can touch. Both residues (live depth, flat flags) point at the same lever: a different representation of the same immutable chain data.

8.3 The run table: the representation lever

Runs. Work over the *kept tree*: the live records plus their dead ancestors (every other node has been removed outright; tombstone-freedom is unchanged). Call the edge from node p to child c *fusible* when (1) c is the unique kept child of p , (2) $\delta(c) = 1$, (3) c ’s side is R (vacuous one-sided), and (4) c and p have the same liveness. A *run* is a maximal chain of fusible edges. Fusibility is a per-edge predicate and condition (1) gives each node at most one fusible out-edge, so the runs are a canonical partition of the kept tree into chains: a function of the state, independent of arrival order. A table entry is one run, with header (liveness flag, parent entry, offset in parent, side, head delta, length); members after the head implicitly have delta 1 and side R, and liveness is uniform along a run. A record’s address is (*run id*, *offset*). Dead entries carry no records,

only the structure live descendants' coordinates pass through. The design's starting observation: what typing mints *is* exactly a fusible chain, so real documents decompose into few long runs.

Definition 8.26 (the run table). *Over the kept tree of a live chain set L (**keptL**: the live chains plus their nonempty prefixes, deduplicated), the edge into c is fusible (**fusible**) when c and its parent are kept, c 's label is 1, c is the parent's unique kept child, and c and the parent have equal liveness ($c \in L \leftrightarrow c.\text{dropLast} \in L$); a head (**isHead**) is a kept node whose incoming edge is not fusible. Runs are the fibers of the climb **headOf**; every member is its head extended by a block of 1 labels. An entry (**RTEEntry**) is the header $\langle \text{par} : \text{Option}(\text{parent index} \times \text{offset}), \text{live}, \delta, \text{len} \rangle$; the table (**tableOf**) maps the canonical head enumeration (**headsList**, sorted by the shortlex **hLt**: shorter first, so parents precede children, **keyLt** within a length) through the entry compiler (**entryOfHead**); the abstraction (**stateOf**) recovers each live entry's members from headers alone. **Canonical** recognizes the image: positive lengths, backward parent references, tail attachment (**par_tail**), maximal fusion (**no_fuse**), dead leaves pruned (**leaves_live**), sorted head enumeration. **RTEEntry, tableOf, stateOf, Canonical***

Two properties before any numbers. *No stability gate*: the run table is an invertible representation of the current state, whatever that state is, unsettled suffixes included, and in particular the freshly typed hot end of the document where the depth cost is being minted; the PBT exploits this by rebuilding the table mid-flight between merges after every event. *Lossless in the strong sense*: from the table alone one recovers every kept node's delta (head delta from the header, 1 elsewhere), side, liveness, and pre-epoch timestamp (deltas summed along the contracted path), hence every coordinate, hence the state; the harness gates exactly this, with zero tolerance, on every record of every trace.

Theorem 8.27 (tail attachment). *In the canonical table, every entry attaches at its parent entry's last member. Kernel form: if h is a head, the edge into $h.\text{dropLast} \cdot 1$ is not fusible,*

$$\text{isHead } L \ h = \text{true} \implies \text{fusible } L \ (h.\text{dropLast} \cdot 1) = \text{false}.$$

*(If an entry attached at an interior member m , then m 's run successor and the entry's head would be two kept children of m , contradicting fusibility of m 's outgoing run edge.) No honesty, no stability, no reachability hypothesis. **tail_attachment***

Three consequences: the "offset in parent" header field is derivable (always parent length minus one; the accounting still counts it and reports it recoverable); the cross-run comparator only ever diverges at a tail or inside a shared run, so it consults entry chains and header fields only and *never materializes a coordinate*; and it is cheap to assert, which the harness does on every table built.

The comparator. Within a run, display order is offset order (each member is the R delta-1 child of its predecessor, hence displays directly after it). Across runs, compare the two entry chains from the root; by the lemma every step exits at a tail, and at the first difference either one chain is a prefix of the other (the deeper record is inside the shallower's continuation subtree; one-sided the shallower displays first, sided an L branch displays first) or two branch entries hang at the same tail and the branch headers decide (one-sided newest first; sided by the banded order). The (ts, agent) concurrency tiebreak is *not* encoded in either representation's counted bits; the tie oracle rides outside the representation on both sides and is charged to neither, preserving parity with the chain measurements. The cost of a comparison is the *contracted* depth: one entry per run on the path instead of one level per node.

Mutation, and the forced coalesce. Splits mutate the table, never the semantics: stored chain data is immutable, only the partition changes. Insert at anchor a : if a is interior, split its entry after a ; attach the new record as a singleton; then *coalesce* (a fusible singleton that is the tail’s unique attachment fuses into the parent entry), which is how typing extends a run in $O(1)$ table work. Delete of x : split to make x a tail; if x still has kept children, carve it out as a dead singleton (a liveness split); childless, it simply vanishes, dropping newly unkept dead tails upward and re-coalescing the parent with a now-unique fusible child. Coalesce is the inverse of split *and it is not optional*: without it, deleting a concurrent interloper leaves the two halves of the old run split forever and the table drifts from the canonical partition (found during design, pinned by a directed case and by the PBT’s incremental-equals-canonical-rebuild gate after every event). Delivery under a vanished anchor (the anchor chain passed through nodes deleted while childless here) re-materializes the missing levels as dead entries from the op’s carried coordinate: the run-table cost of tombstone-freedom, paid with information the embed op already carries. Merges deliver the other side’s events through the same two rules; epochs rebuild the table over the compacted tree (sound because the table is a function of the state).

The accounting model. Applied identically to both representations; every pointer and every header bit counted, no hidden structure. D is the flipped Elias- δ code; $W = \lceil \log_2(N_{\text{entries}} + 1) \rceil$ is the id width.

per record: $W + |D(\text{offset} + 1)|$

per entry: $1 + W + |D(\text{poff} + 1)| + |D(\text{head } \delta)| + |D(\text{len})|$ (+ 1 side bit, sided).

Totals are per live character, reported with the full per-field breakdown so every number is auditable back to a field. Not charged on either side: the tie oracle (above). Two fields are counted although the canonical table makes them recoverable, and the breakdown lets a reader subtract them: the parent offset (the tail-attachment lemma) and the per-record run id when records are stored grouped by run (it is then positional).

Measurement 4: the run table. Same traces and replay as measurements 2 and 3. Columns: the raw chain representation (“chain”), the chain after its best epoch (“fused,” the previous subsection’s current best), the run table over the *raw uncompact*ed chains with no epoch and no settled cut (“rt”), and the run table rebuilt after the epoch (“rtc”).

trace	family	chain	fused	rt	rtc	fused/rtc
friendsforever_flat	1-sided	1622.5	856.5	22.13	22.30	38.4×
friendsforever_flat	sided/F	2582.0	1735.7	22.54	20.98	82.7×
clownschool_flat	1-sided	2488.7	1471.0	22.42	22.64	65.0×
clownschool_flat	sided/F	4058.5	2988.1	24.11	21.20	140.9×
seph-blog1	1-sided	2974.9	917.2	26.00	23.16	39.6×
seph-blog1	sided/F	5559.1	1919.8	27.21	24.99	76.8×
automerge-paper	1-sided	2304.5	1279.4	23.76	23.92	53.5×
automerge-paper	sided/F	4448.0	2583.7	25.21	24.58	105.1×
friendsforever	1-sided	1243.2	856.5	20.92	22.30	38.4×
friendsforever	sided/F	2189.2	1735.6	22.36	20.98	82.7×
clownschool	1-sided	1863.1	1471.0	20.85	22.64	65.0×
clownschool	sided/F	3421.3	2988.1	22.29	21.20	140.9×

All values in bits per live character under the accounting model above; the `_flat` traces are the concurrent traces replayed single-author, their unsuffixed twins the concurrent originals. The verdict: **20.9 to 27.2 bits per character, 38 to 141 times below the fused chains**, consistent with the design prediction (per-record cost collapses from $\Theta(\text{depth})$ to $O(1)$ amortized plus a log-sized offset). The audited breakdown locates the remaining bits:

- The per-record *run id* now dominates (10 to 14 of the 21 to 27 bits, the flat id width): the deep-chain cost did not shrink to the offset code, it shrank to *one pointer*. An implementation storing records grouped by run makes this field positional and free, putting totals at 9 to 14 bits per character; the model charges it because the pre-registered accounting does.
- Offsets cost 5.2 to 12.1 bits per character; all header fields together amortize to 0.6 to 6.4.
- The side channel collapses from one flat bit per live level (867.8 to 1494.1 bits per character after fusion) to one bit per *entry*: **0.061 to 0.174 bits per character**, three to four orders of magnitude, and below even an entropy-coded per-level flag ($H(\text{side})$ times 850 to 1500 levels); the explicit L-entry-list alternative is computable from the reported entry counts and is dearer at these sizes.
- The epoch stack becomes *bit-wise marginal* under this representation: the composed column beats raw by at most 2.9 bits per character (sided) and loses to raw by up to 1.8 (one-sided, where longer post-renumber runs make offsets dearer than the headers they save). The run table alone, with no settled cut at all, already delivers the collapse. What the epoch still buys is *structure*, not bits: entries drop up to $7\times$, the id width shrinks, raw cursor-jump boundaries vanish under renumbering, and the contracted depth drops up to $11.8\times$.
- The honest limit: the *bit* cost per record is $O(1)$ amortized plus the offset code, but the comparator's *time* is still $\Theta(\text{contracted depth})$ per cross-run comparison (mean 19.6 to 261 runs per path against chain depths of 850 to 1500). Collapsing comparison time further (an order index over the contracted tree) is a separate, orthogonal layer; nothing here needs it for the metadata claim.

Gates, all passing: display identity on all six traces, both families, raw and composed (walk order equals the chain display; text equals the final content); about 198k sampled pairwise comparator verdicts matching display positions with antisymmetry checked; losslessness exact on every record; the tail-attachment invariant on every entry of every table; and a 150-trial PBT (8,186 events, three replicas, concurrent edits and merges) gating walk identity, the all-pairs comparator, and incremental-equals-canonical after every event, while exercising 872 merges, 315 mid-run splits, 438 liveness splits, 551 coalesces, and 26 vanished-anchor materializations.

The Lean mechanization of this subsection is in place (`RunTable.lean`, state-level and hypothesis-free except where noted); the load-bearing kernel is Theorem 8.27. Only the epoch-composition clause (`runTable_epoch`: sorted state, at-hand domain, chain representation; walk and read identity after the epoch) carries a stability hypothesis. The remaining theorem bill is closed: the four mutation rules are promoted from SPOT-pinned perturbation lemmas (canonicalization under mutation: `tableOf_congr`) to *closed tableOf* equations, and the walk is promoted to a literal, table-direct reading procedure. The sided run table has its own subsection (§8.4); the one face still open is the wire format (Open Question 25).

Theorem 8.28 (the representation iso, over labels and liveness). *Losslessness: for $\square \notin L$, $\text{stateOf}(\text{tableOf } L)$ is duplicate-free and membership-exact,*

$$(\text{stateOf}(\text{tableOf } L)).\text{Nodup} \wedge \forall c, c \in \text{stateOf}(\text{tableOf } L) \leftrightarrow c \in L$$

(stateOf_tableOf; this membership form is the load-bearing declaration, its unconditional sibling stateOf_tableOf_eq being the flatMap computation the proof runs on). Canonicity: for Canonical T , $\text{tableOf}(\text{stateOf } T) = T$, field for field (`tableOf_stateOf`); without `no_fuse` this direction is false (a run split in two re-canonicalizes to one entry, the drift the coalesce rule exists to prevent). Image: every $\text{tableOf } L$ is canonical (`canonical_tableOf`), so the pair is a genuine bijection between live-chain sets and canonical tables. Stated over labels, sides (vacuously R), and liveness, not timestamps: the exact correction the implementation forced, rank ordinals forgetting time after the first epoch. `stateOf_tableOf`, `tableOf_stateOf`, `canonical_tableOf`

Theorem 8.29 (the contracted comparator equals the key order). *For every ordered prefix code Γ : if every chain of L is positive, and x, y are kept nodes with $x \neq y$, then*

$$\text{cmpTable } L \ x \ y = \text{keyLt } (\text{key}(\text{coordOf}_\Gamma \ y)) (\text{key}(\text{coordOf}_\Gamma \ x)),$$

the display comparator of the coded fold. The chain form (`cmpTable_iff_chainBefore`) is the two-case rule of the design prose; the label order stands where the (ts, agent) tie oracle rides, on both sides identically. `cmpTable_eq_keyLt`

Theorem 8.30 (the structural walk is the display). *For $\square \notin L$: walk L is duplicate-free, contains exactly the live chains, and is pairwise in chainBefore order (via `walkE_sound/walkE_complete`), which pins it to the display sequence, strict orders having unique sorted enumerations. Key form: descending in `keyLt` on the coded coordinates for every ordered prefix code (`walk_pairwise_keyLt`, adding the positivity hypothesis). No stability hypothesis. `walk_display`*

The mutation rules, as closed equations. Each rule is now an equation: `tableOf` of the mutated state equals an explicit rewrite of pre-mutation data. New heads enter the enumeration by sorted insertion, removed heads leave by filtration, surviving entries keep their fields except the announced parent rehoming and length arithmetic, and parent references are re-indexed into the new enumeration (itself closed, being the equation's own head list). No stability or honesty hypothesis appears in any of the four. The engine is a pair of characterization lemmas that force the right-hand sides rather than check them: any strictly sorted enumeration of the heads is the canonical head list (`headsList_eq_of_iff`), and a run's length follows from an explicit offset enumeration of its members (`lenOf_eq_of_iff`). One equation displayed, the other three grouped.

Theorem 8.31 (D1: the mid-run split, closed). *Hypotheses, exactly two: fusible L ($a \cdot 1 = \text{true}$ (the anchor a is run-interior: its run successor is fused) and $a \cdot d \notin \text{keptL } L$ (the delivered record is fresh); $d \neq 1$ is derived (`splitc_d_ne_one`), not assumed. Writing $g := \text{headOf } L \ a$, $j := |a| - |g|$ (the anchor's offset), $\ell := \text{lenOf } L \ g$: the new table $\text{tableOf}(L + [a \cdot d])$ is the old*

one with

$g :$ $\text{len} := j + 1$ (the anchor run cut at the offset),
 $a \cdot 1 :$ $\langle \text{par} = (g, j), \delta = 1, \text{len} = \ell - j - 1 \rangle$ (the suffix, re-headed),
 $a \cdot d :$ $\langle \text{par} = (g, j), \delta = d, \text{len} = 1 \rangle$ (the fresh live singleton),
 h attached in g 's run : $\text{par} := (a \cdot 1, \text{off} - (j+1))$ (rehomed to the suffix),
every other $h :$ unchanged,

the two new heads entering at their sorted positions and the suffix keeping the run's (uniform) liveness; by tail attachment the rehomed entries are exactly those at the old tail. (The childless-anchor happy path stays the $O(1)$ typing equation, grouped below.) **tableOf_insert_split**

Remark 8.32. Display dictionary: in the mechanized equation the new enumeration is literally (headsList $L + [a \cdot 1, a \cdot d]$) re-sorted, parent references re-indexed by index-of into it, and the suffix liveness is $a \cdot 1 \in L$; the table above is that term with g, j, ℓ substituted. Nothing else is elided.

Theorem 8.33 (the remaining closed equations: typing, D2, D4, D5). Each is an equation with all right-hand data pre-mutation; none carries a stability or honesty hypothesis.

- Typing (**tableOf_insert_extend**). Hypotheses: $a \in L, [] \notin L$, a childless in the kept tree ($\forall d, a \cdot d \notin \text{keptL } L$). Appending the delta-1 child re-canonicalizes to the same table with ONE len field bumped, at headOf L a : $O(1)$ table work per keystroke.
- D2, the delete liveness split (**tableOf_delete_split**). Hypotheses: $x \in L$, both edges of x fused (fusible L x , fusible L $(x \cdot 1)$), and $L' = L$ minus x (membership-exact). The kept tree is untouched; the run cuts in three at the liveness break: the live prefix keeps the old head with length cut to x 's offset, x survives as a dead singleton entry, the live suffix re-heads at $x \cdot 1$, and old-tail attachments rehome to the suffix. The boundary shapes (x a run head, x the tail) split in two rather than three and are the same construction with one empty side.
- D4, the coalesce (**tableOf_coalesce**), the closed delete-inverse of D1. Hypotheses: $a \neq []$, $d \neq 1$, the interloper $a \cdot d \in L$ and childless in the kept tree, the suffix $a \cdot 1$ kept with a 's liveness ($a \cdot 1 \in L \leftrightarrow a \in L$), a 's only kept children $a \cdot 1$ and $a \cdot d$, and $L' = L$ minus $a \cdot d$. Both the interloper's entry and the suffix entry are removed, the anchor's run absorbs the suffix (len the sum of the two), and suffix-tail attachments rehome into the merged run.
- D5, directed materialization (**tableOf_materialize**). Hypotheses: $[] \notin L$, the vanished anchor's parent $m.\text{dropLast} \in L$ and childless in the kept tree. Delivering $m \cdot d$ inserts two singleton entries at their sorted head positions, the dead materialized anchor m and the live record below it, parent references re-indexed. This is the directed single-anchor shape; the general vanished-path cascade (a whole dead spine re-materializing, with cuts at non-unit labels and a possible mid-run split at the re-attachment point) is deliberately presented as the composite of this rule, applied once per vanished level, with D1-shaped splits: a genuine cascade, not forced into one equation.

The necessity of D4 is pinned by **spot_d4_coalesce_necessity**. **tableOf_insert_extend**, **tableOf_delete_split**, **tableOf_coalesce**, **tableOf_materialize**

The literal walk. The structural walk behind Theorem 8.30 still consults the state. Its promoted form, `tableWalk`, consults nothing but the table: entry headers, parent references, and address arithmetic (each entry’s head recovered by summing deltas along the contracted parent path, each run emitted as that head followed by its offset extensions, recursion into the entries attached at the run’s tail, newest label first). `stateOf` is never materialized, and the recursion fuel (one more than the total of the length fields) is proved sufficient.

Theorem 8.34 (the literal walk is faithful). *For $\square \notin L$:*

$$\text{tableWalk}(\text{tableOf } L) = \text{walk } L :$$

build the table, read it back table-directly, obtain the display of the live-chain set verbatim. The simulation at matched fuel is `walkT_eq_walkE`; the canonical-table form is `tableWalk_eq_walk`. No stability hypothesis. `tableWalk_tableOf`

This is the formal face of the reading claim the measurement bullets priced: a full read is one pass over the entries in walk order, emission per record with no coordinate materialized. The Θ (contracted depth) comparator time of the honest-limit bullet above is a *random-access* cost; a sequential read never pays it, because the table itself is the order index. The instance file pins the generic procedure on the D1 state (`spot_d1_tableWalk_generic`: the concrete SPOT and the faithfulness theorem name the same reading procedure).

8.4 The sided run table: the same lever under Fugue

Measurements 3 and 4 already ran the sided (Fugue) family through the same accounting and reported the decisive number: the side channel collapses from one flat bit per live level to one bit per entry, 0.061 to 0.174 bits per character. This subsection is that family’s verified counterpart (`SidedRunTable.lean`): the full mirror of the one-sided development of §8.3, over the sided chain kernel of §5, state-level and hypothesis-free throughout (no stability gate, no honest-delivery hypothesis).

Entries, runs, and where the L data lives. A sided entry adds one field to the one-sided header: (parent entry and offset, liveness, *side*, head delta, length). Fusibility condition (3) of §8.3 (the child’s side is R), vacuous one-sided, is now live.

Theorem 8.35 (runs are uniformly R; L nodes are heads). *A fusible sided edge’s last step is $(R, 1)$ with the parent’s liveness: `sfusible L c = true` implies the side of c ’s last element is R (`sfusible_side_R`), so runs are uniformly R and member sides cost nothing. And every kept node whose last element is L-sided fails condition (3) (`Lentry_not_fusible`), hence is a head with its own explicit entry: `Lentry_isHead`, with hypotheses $c \in \text{skeptL } L$ and c ’s last side = L. No further hypotheses.* `sfusible_side_R, Lentry_isHead`

The entire L content of a state is therefore the side bits of the entry headers, which is the verified shape of the measured collapse: the representation stores L data nowhere else.

The canonical order and the iso. The head enumeration is sorted by `shLt`, short-lex over an *injective* flattening of the sided word stream: each head chain flattens to its alternating (side, delta) word, injectivity makes the order strict and total, parents strictly precede children (flattened lengths double), and the tie-break is inherited wholesale from the one-sided head order. Canonical tables (`SCanonical`) carry the side-aware `no_fuse` condition: only an R, delta-1, same-liveness *lone* attachment counts as a missed fuse, so an L singleton hanging alone at a tail is canonical.

Theorem 8.36 (the sided representation iso). *Losslessness: for $\square \notin L$, $\text{sStateOf}(\text{sTableOf } L)$ is duplicate-free and membership-exact for L (sStateOf_sTableOf). Canonicity: for `SCanonical` T , $\text{sTableOf}(\text{sStateOf } T) = T$, field for field, the side field included (sTableOf_sStateOf). Image: every sided table of a state is canonical ($\text{scanonical_sTableOf}$). Over labels, sides, and liveness; deliberately not over timestamps, for the same reason as one-sided: rank ordinals forget time, so a timestamp-preserving iso is false after the first epoch, and the iso is stated over exactly what survives one. sStateOf_sTableOf , sTableOf_sStateOf , $\text{scanonical_sTableOf}$*

The comparator: the offset-vs-tail exit-side rule. The two-case structure of §8.3 survives, licensed by the sided tail-attachment lemma (`stail_attachment`), and the divergence case is still a header verdict: two branch entries at the same tail compare by the banded entry order (L before R; among L siblings the older label first, among R the newer first). The new structural content is in the prefix case. One-sided, an ancestor simply displays before its continuation subtree. Sided, the comparator consults the ancestor’s offset against its run length: a strictly interior ancestor precedes its subtree *regardless of the exit side*, while exactly at the run tail the exit side decides (an R exit displays after the ancestor, an L exit before). This is the run-table shadow of the kernel’s in-order display rule: runs are uniformly R, so strictly inside a run the only continuation is the $(R, 1)$ step toward the tail, an R extension, and the ancestor wins; an L edge can hang only at a tail, where every attachment lives, so sidedness surfaces exactly there.

Theorem 8.37 (the sided comparator equals the sided key order). *For every ordered prefix code Γ : if every chain of L is a positive sided chain, and x, y are kept with $x \neq y$, then*

$$\text{scmpTable } L \ x \ y = \text{keyLt } (\text{sKey}(\text{sidedCoordOf}_\Gamma y)) (\text{sKey}(\text{sidedCoordOf}_\Gamma x)),$$

refining the kernel’s in-order marker theorem (§5) to the storage layer; the chain form is $\text{scmpTable_iff_schainBefore}$. $\text{scmpTable_eq_keyLt}$

The cost is unchanged from one-sided: contracted depth, header fields only, no coordinate materialized.

Congruence and the typing equation. `sTableOf_congr`: the sided table is a function of the live sided-chain set, so merges land wherever delivery order put them and the table cannot tell. `sTableOf_insert_extend`: appending an $(R, 1)$ child to a childless live node changes exactly one length field, $O(1)$ sided table work per keystroke, exactly as one-sided.

The in-order walk, and its literal form. The sided walk of one run: interior members in offset order, then the L subtrees at the tail (ascending, older label first), then the tail member itself, then the R subtrees (descending, newer label first); at the virtual root, the L-rooted subtrees ascending, then the R-rooted descending.

Theorem 8.38 (the sided walk is the in-order display). *For $\square \notin L$: `swalk L` is duplicate-free, membership-exact for the live sided chains, and pairwise in `schainBefore` order (`swalk_display`); descending in `keyLt` on the sided coordinates for every ordered prefix code (`swalk_pairwise_keyLt`, adding the positivity hypothesis). The literal form, `stableWalk`, consults entry headers, parent references, and the same address arithmetic only, with `sStateOf` never materialized; its faithfulness theorem is `stableWalk_sTableOf`, mirroring Theorem 8.34: build the sided table, read it back, obtain the in-order display verbatim.* `swalk_display`

The all-R lockstep bridge. On an all-R live set the sided development projects to the one-sided one in lockstep.

Theorem 8.39 (the all-R lockstep bridge). *Table half, no hypotheses:*

$$\text{sTableOf } (L.\text{map liftR}) = (\text{tableOf } L).\text{map liftEntry} :$$

the sided table of a lifted all-R state is the one-sided table with the side bit stamped R on every entry (`sTableOf_liftR`). Walk half, hypotheses every chain of L positive and $\square \notin L$:

$$\text{swalk } (L.\text{map liftR}) = (\text{walk } L).\text{map liftR},$$

consuming the kernel’s fragment theorem `schainBefore_liftR` (§5). `sTableOf_liftR`,
`swalk_liftR`

This is the storage-layer analog of the convergence-layer “sides are payload” theorem (§12): the one-sided datatype is the L-uninhabited fragment of the sided one, so every one-sided storage result of §8.3 transports to the sided table on that fragment with no re-proof.

Per house discipline the file closes with hand-derived PASS+FAIL executions: the L child that does not fuse where the R child does; the band order pinned both in the entry order and in the coordinate keys; the L-band mint displaying L band, then node, then R band; the band *direction* with its FAIL companion (sorting the L band newest-first, like the R band, provably flips the display: the tempting degenerate order is itself refuted); the table-direct walk on the mint case; and the all-R lockstep pinned on the one-sided D1 state, table half and walk half. Every capstone named in this subsection and in §8.3 passes its `#print axioms` audit at `propext/Classical.choice/Quot.sound` or below; `native_decide` is confined to the SPOTs.

8.5 From projection to shipped bytes, and the cross-system question

Two external facts close the representation arc; both are measured, and both are laid out with their full methodology in the performance section (§23), so this section summarizes the results. First, the run-table projection of §8.3 is validated beyond the accounting model: the runtime ships a lossless run-table serializer (`runtime/src/serialize.js`), whose `accountingBits` routine reproduces the projection’s bit totals *bit-for-bit* on every trace, while the shipped bytes land *below* the projection (the model charges the recoverable positional fields — per-record run id and

offset, and the tail-attachment parent offset — that a real encoder drops). Over a settled-cut compacted state the serializer reaches 1.1 to 1.3 bytes per character on the josephg traces, at production save size and an order of magnitude below the packed absolute-chain estimate. Second, placed against production CRDT libraries this closes the save-size gap to rough parity with the smallest state-only production saves. Tombstone-freedom lets the save *shrink* on deletion, unlike the history-carrying systems in the experiment; other reclaiming state formats can also shrink. The full cross-system matrix — per-character apply latency, merge and sync, load, all four save representations, sync payload, and the grows-on-delete reproduction — is developed in §23.

9 The marks layer: document-order rich text, and its GC

Rich text is the family’s first *effect-coupled* extension: marks (bold, links, comments) are anchored in the sequence but are not sequence elements. The repository holds two mark encodings. Part III’s fused Peritext (§16) encodes a mark as a pair of in-sequence boundary inserts; the model of this section is the other encoding, the one the runtime ships as its third datatype (§22): characters live in the embed order, and marks are *separate immutable records* that name their boundary anchors by id. The model has two components. The read model and its theorems (`Peritext_Embed/PeritextEmbed_MarkIntent.lean`); and its garbage collector (design and validation `whiteboard/marks-gc-note.md` with harness `whiteboard/litmus/marks_gc_check.py`; mechanization `PeritextEmbed_MarksGC.lean` and `PeritextEmbed_MarksGC_A3.lean`; runtime `runtime/src/compact-peritext.js`), which moves the runtime’s Peritext state GC from a justified refusal to a certified fire. Verdicts below are labeled by provenance, as everywhere in this document: *proved* is kernel-checked Lean, *measured-in-model* is the checked-in Python harness, *measured* is the shipped JavaScript runtime’s checked-in tests.

9.1 The document-order mark read model

State. Text is an insert-only shadow: a map from character id (a Lamport stamp, globally unique, exceeding the id of the insert’s anchor) to its immutable birth coordinate, plus a *grow-only set of deleted ids*. Text deletes are logical because the resolver below needs the birth positions of *dead* anchors; that retention is exactly what the GC half of this section must then contend with. Marks are a grow-only map from mark id *mid* (the same Lamport space) to an immutable record (*mid, mtype, op, startId, endId, startSide, endSide*) with *op* ∈ {`add`, `remove`}: a `removeMark` is a first-class negative record, and per (character, *mtype*) the covering mark with the highest *mid* wins (LWW), a winning remove suppressing the formatting. A per-mark value (a link url, a colour) rides outside the LWW key. Reading order is the embed order of §1 (birth-chain lexicographic, newer children first); the live order is that order filtered by the deleted set.

Boundary resolution. Each boundary resolves to a live position. For a live anchor the side is gravity, and “newer than the mark” means character id > *mid*; *growth is end-side only*, the one design decision carried in from validation: an end with *endSide* = *after* grows right over the newer-than-mark run (typing at the end of a bold span stays bold), while a *before* start does *not* grow left, else it grabs unrelated newer siblings and diverges from the Peritext paper’s

semantics. For a dead anchor the boundary *rehomes*: the nearest surviving character in reading order, scanning from the anchor’s birth position in the side’s direction. The eight cases:

boundary	anchor	side	resolution
start	live	<i>before</i>	the span starts at the anchor
start	live	<i>after</i>	starts after the anchor, skipping the newer-than-mark run
start	dead	<i>before</i>	rehome right, the span starting at the survivor; scan exhausted: the mark collapses
start	dead	<i>after</i>	rehome left, then as the live case (the skip applies to the survivor); exhausted: document start
end	live	<i>after</i>	ends at the anchor, then <i>grows</i> right over the newer-than-mark run
end	live	<i>before</i>	ends before the anchor, stepping back over the newer-than-mark run
end	dead	<i>after</i>	rehome left, then as the live case (the growth applies to the survivor); exhausted: the mark collapses
end	dead	<i>before</i>	rehome right, then as the live case (the step-back applies to the survivor); exhausted: document end

Definition 9.1 (the mark-record state and the document-order read). *The document is $d = (\text{shadow}, \text{deleted})$ (DocD): an insert-only embed state (every insert ever applied, payload the codepoint) plus the grow-only deleted-id list. The birth order is the shadow’s id sequence in embed reading order (DocD.birthIds); the live order is the birth order filtered by deleted (DocD.liveIds). A mark is the immutable record $(\text{mid}, \text{mtype}, \text{op}, \text{value}, \text{startId}, \text{endId}, \text{startSide}, \text{endSide})$ (MarkD ; two frozen ancestry snapshots ride along for the retracted tree control alone and are ignored by the resolver). The resolver returns option-valued live indices: $\text{startIncl } d \ m$ is the first covered index and $\text{endExcl } d \ m$ the exclusive upper index, each computed by the eight-case table above, in resolve-then-side order: a dead anchor first rehomes by the side’s nearest-survivor scan from its birth position (exhaustion giving the side’s collapse or document sentinel), and the boundary anchor, live or so rehomed, then takes its side’s rule (gravity; the newer-than-mark run — contiguous live ids $> \text{mid}$ — skipped at an after start and grown over at an after end); none is the collapsed span. Coverage is the half-open interval:*

$$\text{markCoversPos } d \ m \ k = (f \leq k \wedge k < e) \\ \text{when } \text{startIncl } d \ m = \text{some } f, \text{ endExcl } d \ m = \text{some } e,$$

and false when either boundary collapses. Per $(\text{position}, \text{mtype})$ the covering mark with the largest mid wins (**bestCover**), and the position is formatted iff that winner is an add (**fmtAt**). The render $\text{renderMarksDoc } d$ marks mt is the live order with each character’s flag, $\text{renderFlagWith } d$ ($\text{markCoversPos } d$); the retracted control renderMarksTree is the same render over treeCoversPos (climb the frozen path to the nearest live ancestor and cover the inclusive interval between the two resolved anchors, sides ignored). $\text{DocD}, \text{MarkD}, \text{renderMarksDoc}$

The positive theorem, and the retraction it replaces. An earlier composed-Peritext read resolved a dead boundary by climbing *tree* ancestry (frozen path snapshots), and carried a `mark*_no_leak` theorem asserting that formatting never leaks under deletion. That claim was **false** and is retracted: a tree ancestor sits *earlier in reading order* than its descendants, so a dead boundary anchor migrates backward in the document and formats text that was never in the span. The document-order model is built so that the replacement is provable: rehoming scans *reading* order, so a single-replica rehome can only shrink or hold a span, never migrate it backward.

Theorem 9.2 (no backward leak, document order). *For every document d , mark m , type mt , and live indices f, k : if $\text{startIncl } d \ m = \text{some } f$ and $k < f$, then*

$$\text{fmtAt } (\text{markCoversPos } d) \ [m] \ mt \ k = \text{false} :$$

no character strictly before the mark's rehomed start carries that mark's formatting. Stated at the singleton mark list $[m]$, which is exactly the per-mark claim: `markCoversPos` is list-independent, so in any larger list m still covers nothing before f , and a position there can be formatted only by another mark's own span. Proved on axioms $\{\text{propext}\}$ alone, below even the document's kernel-clean baseline. **`doc_no_backward_leak`**

Proposition 9.3 (the tree-ancestry read leaks backward). *There exist a document d , a mark m , and a live position k with*

$$\text{markCoversPos } d \ m \ k = \text{false} \quad \wedge \quad \text{treeCoversPos } d \ m \ k = \text{true} :$$

*the retracted claim's own resolver formats a character the document-order read does not, so no analogue of Theorem 9.2 holds for the frozen-path read. The witness is concrete: delete a bold span's start anchor; document order rehomes the start forward to the next survivor (**`leak_doc_shrinks_doc`**), while the tree read climbs to the anchor's parent and formats text that was never bold (**`leak_doc_shrinks_tree`**), with the $\neq$ companion **`leak_doc_shrinks`** pinning that the two renders genuinely differ.* **`tree_backward_leak_refutes_noleak`**

The Litt et al. pins. The model is pinned against the worked examples of the Peritext paper (Litt et al. CSCW'22), each a concrete SPOT with a hand-derived expected render and a $\neq$ companion refuting the tempting degenerate: insertion within a bold span is formatted (Ex1); overlapping bold and italic carry both on the overlap (Ex2); marking a span, deleting the whole span, and reinserting yields plain text under document order, while the tree control leaks the formatting back (Ex3, the leak made concrete); concurrent add versus removeMark resolves LWW by *mid*, and swapping which is newer flips the verdict (Ex5); typing at the end of a bold span is grabbed exactly when the typed text is newer than the mark (Ex7); and the gravity contrast (Ex8): on the same insertion, a bold span (*endSide = after*) expands and a link (*endSide = before*) does not. Convergence is watched at the same grain: the render is a function of the mark *set* (permutation invariance, with a dropped-mark FAIL pin); character order converges by the embed capstone.

The honesty theorem: the atomicity cost, stated. The substrate is tombstone-free and live, so the mark-positioning trilemma (atomic mark placement, tombstone-freedom, live state: pick two; §16) charges this model too, and it pays on the growth rule.

Proposition 9.4 (a delete can re-span: the trilemma horn). *There is a four-character document and a bold mark — concretely: A, B marked ($\text{endSide} = \text{after}$), C older than the mark blocking the growth run, D newer, with the hand-derived render pins respan_before (A, B bold; C, D plain) and respan_after (A, B, D bold) — such that deleting the plain, boundary-untouched C (id 3) changes an untouched survivor’s formatting:*

$\text{renderMarksDocIds } d_{\text{after}} [m] \text{ bold} \neq (\text{renderMarksDocIds } d_{\text{before}} [m] \text{ bold}).\text{filter } (\lambda e. e.1 \neq 3) :$

the post-delete render is not the pre-delete render minus the deleted character’s entry, because C ’s deletion makes D contiguous with the span and the after-end growth run reaches it.

doc_delete_can_respan

This is a membership re-span, not a backward leak (Theorem 9.2 is untouched), and the contrast with the fused encoding is exact: the fused design has no growth rule and proves **renderIds_del** (a character delete re-formats nothing); the marks model trades that theorem for the paper’s expand-at-the-end behaviour, and the price is stated as a theorem rather than hidden.

9.2 Why plain-text compaction is unsound for marks

The state-GC pass of §8.2, as shipped in the runtime (§22), drops settled-dead records and re-codes coordinates. For plain text that is sound; for this datatype, pruning a dead anchor’s record destroys exactly the birth position a mark needs to rehome, and the runtime therefore *refused* to compact Peritext at all. The refusal was correct, and the justification is a directed flip (D6, hand-derived). Geometry, reused throughout this section: R (id 1) and L (id 2) minted concurrently at the root, so L is newer and reads first; d (id 3) a child of L ; reading order $L d R$. Mark [start L before, end d after], $\text{mid} = 5$; delete d ; compact at a settled cut with no retention. The never-compacted twin: the dead end anchor d scans left and finds L ; covered $\{L\}$, so L is bold. The unretained subject: d has no record, the resolver cannot place the scan and degrades to the collapse branch; covered $\{\}$, L is plain. The read flips. The companion pins the cause: the same compactor with all mark anchors *live* reads twin-identical, so the refusal’s reason is precisely dead mark anchors, nothing else. Randomized (measured-in-model): the no-retention control flips 351 of 600 randomized DAG trials.

9.3 Two attacks, falsifiable forms first

The settled-cut contract is the text layer’s (§8.2): every replica has delivered everything at or below the cut, and every op concurrent with the cut that is not yet delivered is *declared* in flight (its id and anchor id). The compaction map is also the text layer’s: drop settled-dead records outside the keep-set, renumber each sibling group of the kept tree order-preservingly except groups receiving a declared in-flight delta (frozen verbatim, the **skipAt** guard), and let beyond-cut mints derive their coordinate from the re-coded anchor record plus the fresh delta (the extension law H3 realized as *derive-at-ingest*). The two attacks differ only in the keep-set and the mark treatment, and each was stated as a falsifiable reads-identical claim before any trial ran.

H-A: retention roots, validated. Keep-set = live ids $\cup$ the boundary anchor ids of every present mark record (add and remove records both) $\cup$ declared in-flight anchors. Dead boundary anchors survive as dead records with re-coded coordinates; the marks map is untouched. Falsifier:

any version, on any beyond-cut continuation (text and mark ops on any replica, merged in any order, declared stragglers delivered late, successive cuts composed), whose render differs from the never-compacted twin. Verdict (measured-in-model): **validated**, 2000/2000 randomized DAG trials (4048 cuts, 2214 with declared stragglers), zero divergences. Cost (measured-in-model): 0.296 retained dead records per mark record, maximum 2.000 against the structural bound of 2 (a mark has two boundaries); 30.6 percent of settled-dead records dropped under a deliberately mark-heavy random workload (real traces mark far fewer characters per edit and would drop a larger share).

H-B: early re-anchoring, refuted. At the cut, rewrite each *dead* mark boundary to the resolver’s own cut-time rehome target (side kept; an exhausted scan becomes the side’s collapse sentinel), then retain nothing for marks. The rewrite commits at the cut to information the resolver would otherwise read at render time (the dead anchor’s position among *later* arrivals), and the minimal countermodel family (D1, all cells hand-derived, then machine-confirmed cell by cell) makes that exact. Same geometry as D6; one continuation character n : (a) a beyond-cut insert, id 10 (newer than the mark), anchored at L ; (b) a declared straggler, id 4 (*older* than the mark), anchored at L , delivered after compaction; (c) the same straggler anchored at d itself. Covered sets, twin versus H-B subject, per boundary combination (the mark’s boundary at d , the other on a survivor):

combo (boundary at d)	(a) id 10 at L	(b) id 4 at L	(c) id 4 at d
start <i>before</i> (inner)	$\{R\} = \{R\}$	$\{R\} = \{R\}$	$\{n, R\}$ vs $\{R\}$: diverges
start <i>after</i> (outer)	$\{R\} = \{R\}$	$\{R\}$ vs $\{n, R\}$: diverges	$\{n, R\} = \{n, R\}$
end <i>after</i> (inner)	$\{L, n\} = \{L, n\}$	$\{L, n\}$ vs $\{L\}$: diverges	$\{L\} = \{L\}$
end <i>before</i> (outer)	$\{L\} = \{L\}$	$\{L, n\} = \{L, n\}$	$\{L\}$ vs $\{L, n\}$: diverges

Worked sample, end *after*, variant (b). Twin: birth order $L n d R$; the dead end anchor d scans left and finds n live, so the span ends at n : covered $\{L, n\}$. H-B rewrote the end at the cut, before n existed, to L ; at render the end sits at L and end-growth does not cross n , whose id 4 is below the mark’s *mid* 5: covered $\{L\}$. The straggler sits inside the twin’s span and outside the subject’s.

The compensation finding. Column (a) is the negative space of the countermodel, and it is exact rather than lucky: with only a beyond-cut *newer* insert, all four combinations agree. Every character that appears between the rewrite target and the dead anchor’s old position after a settled cut is either a beyond-cut mint (id above every settled id, hence above the mark’s *mid*, so the newer-run growth and skip rules cross it on *both* sides of the comparison) or a declared straggler, the only source of ids below the mark’s *mid*. And nothing beyond-cut can land between a settled-dead anchor and its reading-order successor without anchoring in the dead subtree, which a settled cut makes invisible to future minters; only a straggler anchored at the dead node itself can, which is column (c). So the naive recency candidate (a beyond-cut newer insert alone) *provably cannot fire*: the H-B countermodel needs a straggler older than the mark (columns b, c) or the death of the rewrite target. The latter is D1(d), hand-derived: chain L (1) at the root, d (2) a child of L ; mark [start L *before*, end d *after*], *mid* = 5; delete d ; cut (H-B rewrites the end to L); beyond the cut insert n (10) under L , then delete L . Twin: birth $L n d$, live $\{n\}$; the start rehomes right to n , the dead end d scans left and finds n : covered $\{n\}$,

n is bold. Subject: the rewritten end anchor L is now dead too and scans *left* from L 's position, where nothing lives (n sits to L 's right, unreachable by that scan): the mark collapses and n is plain. It diverges with no straggler anywhere: the rewrite also forgot that d sat to the *right* of L , so the re-rehoming scans from the wrong birth position.

The weaker true statement, labeled as what it is. The rewrite is computed once at the common settled cut, so every compacted replica shares it, and all compacted replicas *converge among themselves*: machine-checked separately, 2000/2000 trials. This is **licensed divergence**, a semantic change at a settled cut, and it is never reads-identical: 156 of the 2000 randomized trials diverged from the never-compacted twin, on top of the directed family above. It must not be stated as H-B succeeding, and is recorded here as the price a re-anchoring design would pay, not as a result.

D3: a dead run keeps its internal order. Two boundaries inside one dead run are observably *ordered*. Chars A (1) at the root, x (2) under A , y (3) under x , B (4) under y ; mark m_1 covers $[A..x]$ (*mid* 10), m_2 covers $[A..y]$ (*mid* 11), both inner sides; delete x and y (one dead run); cut; declared straggler g (5) anchored at x , delivered after compaction. g is a newer child of x than y , so birth order is $A x g y B$. Twin: m_1 's end (x , dead) scans left to A : covered $\{A\}$; m_2 's end (y , dead) scans left to the live g : covered $\{A, g\}$. So g carries m_2 and not m_1 : the two boundaries' order inside the dead run is observable. H-B rewrote both ends to A at the cut (the run's left survivor): both marks cover $\{A\}$ and g loses m_2 . H-A retains x and y as dead records whose re-coded coordinates keep their relative order, and reads exactly as the twin; that order preservation on retained-dead pairs is the A1 obligation the mechanization below discharges.

9.4 The growth window: the guarded root-freeing drop

Plain H-A retains a mark's anchors as long as the mark record exists, and mark records are grow-only, so a heavily reformatted document accumulates (add,remove) pairs whose anchors are retained forever. The A3 refinement drops such a pair and frees its retention roots: an add m and a remove r of the same *mtype* with *equal boundary tuples* (ids and sides) and $r.mid > m.mid$ resolve to the same covered interval forever, and r beats m on every covered character, so dropping both looks render-neutral. It is sound only under *three* guards, each with a hand-derived countermodel:

- *Removal settled, declared in-flights included* (the α flip): a concurrent same-type add s with $m.mid < s.mid < r.mid$, still in flight at the drop, is suppressed by r on the overlap while the pair is present and formats it once the pair is dropped.
- *No LWW shadowing*: no other same-type mark with *mid* below $r.mid$ may exist, because spans move under later edits, so a no-overlap check *now* does not persist; dropping r could resurrect the older mark on a future overlap.
- *The growth window* (the β flip, the discovery of this phase): no settled or declared id may lie strictly inside $(m.mid, r.mid)$. A character w with $m.mid < w.id < r.mid$ is grabbed by the add's end-growth ($w.id > m.mid$) but *not* by the remove's ($w.id < r.mid$ fails its newer test), so w is bold with the pair present and plain after the drop: the pair is not render-neutral even though its boundaries are identical. Beyond-cut mints are Lamport-fresh (above $r.mid$), so checking the settled and declared ids at the cut is sufficient.

Both unguarded flips and the guarded positive (with a genuinely dead root freed that plain H-A would have retained) are hand-derived and machine-checked. Guarded, on top of H-A (measured-in-model): 600/600 twin trials with 44 pair-drops fired; each drop removes two mark records and frees their anchors for the next cut.

9.5 The mechanization

The note’s obligations O1–O4 are mechanized in `PeritextEmbed_MarksGC.lean` (O1–O3) and `PeritextEmbed_MarksGC_A3.lean` (O4); O5, the settled-cut protocol half, is deliberately *not* taken on, being the already-tracked residue below.

O1: the keep-set is a stable-prefix map, not a new notion.

Definition 9.5 (the keep-set stable-prefix map). *A keep-set is $K = (\text{live}, \text{markAnchor}, \text{inflight})$ (KeepSpec), three coordinate predicates, with $K.\text{kept } c = \text{live } c \vee \text{markAnchor } c \vee \text{inflight } c$: the live records, the boundary anchors of present and declared in-flight mark records (dead anchors included — the retention roots), and the anchors of declared in-flight text ops. Given a relabeling f and declared mints Mint satisfying*

$(h_{\text{ext}}) \text{ Mint } \pi \ d \text{ implies } f(\pi \cdot \Gamma.\text{enc}(d)) = f(\pi) \cdot \Gamma.\text{enc}(d), \text{ and}$

$(h_{\text{ord}}) \text{ for } c, \ c' \text{ each kept or mint-shaped } (\exists \pi \ d, \ \text{Mint } \pi \ d \wedge c = \pi \cdot \Gamma.\text{enc}(d)):$
 $\text{keyLt}(\text{key}(f \ c)) (\text{key}(f \ c')) = \text{keyLt}(\text{key } c) (\text{key } c'),$

$\text{keepSPM } \Gamma \ K \ f \ \text{Mint}$ is the existing stable-prefix bundle of Definition 8.1 with $\text{Rest} := K.\text{kept}$ and $\text{MintAt} := \text{Mint}$: h_{ext} is H3 and h_{ord} is H2, field for field. H3’s Mint covers fresh mints and declared stragglers, whose frozen sibling groups are exactly the text layer’s skipped-groups side condition, with the FAIL companion *freeze_guard_violation* pinning the flip (deltas $2 < 5$ in-flight < 9 renumber to 1, 2 and the frozen 5 jumps the ex-9). *KeepSpec, keepSPM*

Lemma 9.6 (A1, with H1 derived). *For kept coordinates $c, \ c'$:*

$$\text{keyLt}(\text{key}(f \ c)) (\text{key}(f \ c')) = \text{keyLt}(\text{key } c) (\text{key } c') :$$

*the relabeling preserves the display comparator on the whole kept set, retained-dead pairs included, which is exactly what the D3 observation (§9.3) consumes. Injectivity on kept pairs is derived through the bundle (*keepSPM_injOn_kept*), not assumed. *keepSPM_A1* is proved with no axioms at all.* *keepSPM_A1*

O2: scan factorization, a pure list lemma.

Lemma 9.7 (scan factorization). *For every split $l_1 \cdot a \cdot l_2$ of the birth order, deleted list del , and keep predicate p with $p \ a = \text{true}$, $a \notin l_1$, and every survivor of l_2 kept ($\forall x \in l_2, \ x \notin \text{del} \Rightarrow p \ x = \text{true}$):*

$$\begin{aligned} & \text{scanRight } ((l_1 \cdot a \cdot l_2).\text{filter } p) \ \text{del} \ (\text{idxOf } ((l_1 \cdot a \cdot l_2).\text{filter } p) \ a) \\ & = \text{scanRight } (l_1 \cdot a \cdot l_2) \ \text{del} \ (\text{idxOf } (l_1 \cdot a \cdot l_2) \ a) : \end{aligned}$$

*the first survivor strictly right of the retained element a is the same in the filtered and the full sequence, because every dropped element is a non-survivor the scan skips anyway. Mirror statement *scanLeft_filter*, with the kept-survivors hypothesis on l_1 . No datatype content: *List.filter* and *find?* over an arbitrary split. *scanRight_filter, scanLeft_filter**

O3: the render-congruence capstone. The load-bearing observation: `renderMarksDoc` is a function of the live id sequence, the boundary anchors' positions relative to it, and the id order; it never reads a coordinate. Its hypothesis structure is displayed first:

Definition 9.8 (the continuation discipline). `ContOK` Γ s τ , for a cut state s and continuation τ , has four fields:

(*freshId*) every insert $(t, r, \text{ins}(e, \pi, a)) \in \tau$ has $t \notin \text{ids}(s)$;

(*nodupIns*) τ 's insert ids are duplicate-free;

(*freshKeySt*) every insert of τ mints a key $\text{key}(\pi \cdot \Gamma.\text{enc}(t - a))$ distinct from every stored record's key;

(*freshKeyPair*) distinct inserts of τ mint distinct keys.

This is what any honest beyond-cut continuation satisfies (Lamport freshness plus unique decodability); deriving it, together with the straggler declarations, from honest reachability at a certified cut is nothing marks-specific: it is O5, the per-continuation face of the settled-cut protocol residue. The honesty-rebasing half of that residue, the $(*)$ lemma, is closed for the text layer (Definition 8.21–Theorem 8.24, §8.2); what remains is the protocol half — Open Questions 25 and 25 — and the marks layer adds no new obligation to it. **ContOK**

Theorem 9.9 (O3: render congruence under retention-roots compaction). For a stable-prefix map F ; cut state s ; continuation τ ; keep predicate kp ; deleted list del ; mark list $marks$; type mt ; under the hypotheses:

($h_{\text{sort}}, h_{\text{nd}}$) s is strictly key-sorted and its ids are duplicate-free;

(h_{ok}) `ContOK` Γ s τ (Definition 9.8);

(h_{kp}) every insert of τ is kept (kp true at its id);

(h_{dom}) every kept record of s is at hand ($F.\text{Dom}$ of its coordinate);

(h_{mint}) every insert $(t, r, \text{ins}(e, \pi, a)) \in \tau$ is a declared mint ($F.\text{MintAt } \pi (t - a)$);

(h_{dead}) every dropped record's id is recorded dead (kp false at $r.1$ implies $r.1 \in del$);

(h_{anchor}) every present mark's two boundary ids are kept (the retention roots),

the compacted-and-continued document renders equal to the never-compacted twin:

$$\begin{aligned} & \text{renderMarksDoc } (\text{applySeq } (\text{remap}_F (s.\text{filter } kp)) (\tau.\text{map } \text{remap}_F), del) \text{ marks } mt \\ &= \text{renderMarksDoc } (\text{applySeq } s \tau, del) \text{ marks } mt, \end{aligned}$$

for every mark and every type. Kernel-clean. The proof chains the re-coding cluster's T1 (Theorem 8.3 transports the compacted fold), coordinate invisibility of the render, commutation of the continuation with the drop, and O2 (Lemma 9.7) plus the retention roots. **marksGC_render_congr**

Theorem 9.10 (O3, two epochs). *For maps F_1, F_2 with $\text{CompatOn } F_1 F_2 \text{ Rest}' \text{ MintAt}'$ (Definition 8.14); keep predicates kp_1, kp_2 ; continuations τ_1, τ_2 ; under Theorem 9.9's hypothesis list taken twice: epoch 1's clauses $(h_{ok_1}, h_{kp_1}, h_{dom_1}, h_{mint_1})$ verbatim at (s, τ_1, kp_1, F_1) , and epoch 2's at the once-continued state and the composite: h_{ok_2} at $\text{applySeq } s \tau_1$; $h_{kp_2}, h_{dead}, h_{anchor}$ at the conjunction $kp_1 \wedge kp_2$; h_{domG}, h_{mintG} at $F_1.\text{comp } F_2$'s surviving domain $(\text{Rest}', \text{MintAt}')$. Conclusion: compact at a settled cut, continue, compact again on the already re-coded records, and deliver the twice-lagging τ_2 translated through the composite; the final render equals the never-compacted twin's (the display elides the nested remaps; the mechanized statement carries them in full). Kernel-clean. The composite is the existing **StablePrefixMap.comp** carried at its own surviving domain: a retention root freed between the epochs (by an A3 pair-drop) is exactly the rank-reclaim collision shape **naive_composition_collides** pins (§8.2), and restricting the composite's domain to the survivors is the fix that file already provides. n -epoch towers follow by the same **chainSPM/CompatOn** route. **marksGC_render_congr_twoEpoch***

O4: the guarded drop.

Theorem 9.11 (O4: the guarded pair-drop). *For a document d , mark list marks , and $m, r \in \text{marks}$ with $m.op = \text{add}$, $r.op = \text{remove}$, $r.mtype = m.mtype$, $m.mid < r.mid$, equal boundary tuples $(r.startId = m.startId, r.endId = m.endId, r.startSide = m.startSide, r.endSide = m.endSide)$, and marks 's mids duplicate-free; under the three guards:*

- (G1) *removal settled, declared in-flights included: the statement is over the final delivered document and mark list, so a cut-time drop is licensed exactly when the settled marks plus the declared in-flight marks exhaust the final list's candidates — beyond-cut Lamport freshness (every later mid above $r.mid$) covers the rest, which is the α flip's discharge;*
- (G2) *no LWW shadowing (h_{others}): every other same-type mark $o \in \text{marks}$ (with $o.mid \neq m.mid$, $o.mid \neq r.mid$) has $r.mid < o.mid$;*
- (G3) *the growth window (h_{window} , the β clause, the discovery of this phase): every birth id c of d satisfies*

$$c \leq m.mid \vee r.mid < c :$$

no id in the half-open window $(m.mid, r.mid]$ (the formula also excludes $c = r.mid$): such a character is grabbed by the add's end-growth but not the remove's, so the pair is not render-neutral even with identical boundaries,

dropping the pair preserves the render:

$$\begin{aligned} & \text{renderMarksDoc } d \text{ (marks.filter } (\lambda o. o.mid \notin \{m.mid, r.mid\})) \text{) } mt \\ &= \text{renderMarksDoc } d \text{ marks } mt. \end{aligned}$$

*Only $r.op = \text{remove}$ is load-bearing for the verdict (the add hypothesis names the shape, the proof never consults it). **a3_guarded_drop***

The SPOT block is PASS+FAIL shaped throughout: the guarded positive fired *through the theorem*, the freed-root payoff (the next cut drops the ex-anchor's record and still matches the twin that holds both marks), a surviving-rival-mark variant, and both flips (α, β) each with a guard-refuses companion.

Model–harness discrepancy. The Lean model and the Python harness diverge on one degenerate branch: a boundary anchor with *no record at all*. The Lean resolver’s scan then starts past the end of the birth order (finding the last, respectively first, survivor); the Python harness degrades to the side’s collapse sentinel. The two disagree on the degenerate *value*, but D6 flips against the twin either way, which is the only load-bearing fact, pinned on both sides (`d6_no_retention_flip`; the harness’s D6), and the positive obligations never touch the branch: retained anchors always have records. The general lemmas and both capstones are kernel-clean; the SPOTs use `native_decide` under the document’s SPOT convention.

9.6 The runtime: refuse becomes fire-with-certificate

The shipped compactor. `compact-peritext.js` transliterates the harness semantics. The keep-set is as above; the re-coding, the order-preserving rank renumbering, and the in-flight freeze guard reuse the text layer’s `compactEliasDelta` verbatim over the kept shadow; marks reference *ids*, which re-coding never rewrites, so the mark map itself needs no translation; retained dead anchors stay listed in the deleted set. `compactiblePeritext` plugs the hooks into `DistributedReplica`: `compactStable` for Peritext now *fires* under the same frontier certificate as the text layer (§22), where it previously refused, and the A3 pair-drop runs first so freed roots leave the keep-set in the same epoch.

The delete-must-settle refinement. One contract refinement goes beyond the harness. Peritext deletes are logical, so a record may be freed only once its *delete* is settled, not merely its insert: an id whose delete is still propagating is live somewhere and may yet be marked or anchored on. The harness’s full-exchange barrier settles both at once and cannot see the distinction; the runtime’s certificate separates them (settled insert ids versus settled delete ids, both extracted from the certified meet), and a directed test pins the refusal while a delete propagates and the fire once it settles.

Deviations, all conservative. Under a certified cut every in-flight field is empty: every op concurrent with the cut is already delivered (`SettledAt`), so passing empty declarations is proved sufficient rather than asserted, exactly as for the text layer. A declared in-flight insert whose anchor has no kept record throws rather than guesses. And the justifying countermodels stay executable against the shipped code as negative controls (`noRetention`, `unguardedPairDrop`, the unguarded renumber), so the flips above are reproducible in the artifact, not only in the model.

Measured. With the marks GC wired, the full runtime suite passes 103/103 and the p2p-demo suite 10/10. The dedicated file (`runtime/test/peritext-gc.test.js`) carries the directed PASS+FAIL family (D6, the D1 straggler cells, the frozen-group order flip through the Peritext path, D3, both A3 flips with their guard-refuses companions), certified refuse-then-fire with wire catch-up across the compaction commit and a *new* mark anchored on a retained dead record in the new epoch, the delete-must-settle directed pair, and a 150-trial multi-epoch twin PBT with declared stragglers (300 cuts, 179 with stragglers, 217 of 730 settled-dead records dropped, 10 guarded pair-drops): zero divergences from the never-compacted twin. Measured cost on this workload: 444 retained dead records over 1780 mark records, 0.249 per mark record, per-cut

maximum 2.000 against the structural bound of 2. The 60-trial no-retention control flips 35 trials: the old refusal stays justified against the shipped code.

The stack, closed. §7 characterized what a tombstone-free sequence must remember: the dead chain’s slot keys, surviving as order ordinals inside live coordinates, with the fooling pair as the lower bound. The marks layer adds one further debt, and pays it counted. With retention-roots GC and the guarded pair-drop, the entire Peritext state is $O(\text{live})$: live characters, present mark records, and dead information surviving *only* as (i) the provably necessary order ordinals of §7 and (ii) at most two mark-anchor retention roots per present mark record (the structural bound; measured 0.25 to 0.30). Everything else, dead text and settled mark pairs alike, is dropped, and the invisibility of the drop is a kernel-checked theorem plus a twin-checked measurement, not a hope.

10 Comparison with Eg-walker

Eg-walker (Gentle & Kleppmann, EuroSys 2025) is the closest system in spirit, and the two designs can be read as the *operational* and *denotational* realizations of one thesis, **pay for concurrency, not for history**:

	Eg-walker	embedded-chain RGA
model	op-based; replicas exchange events	MRDT: three-way merge with an LCA
durable state	full event graph on disk , deletes included, retained while concurrent ops may arrive; document state metadata-free	live nodes only ; no event log anywhere; dead survive as $\Theta(\log \delta)$ bits inside live coordinates
merge mechanism	<i>replay</i> : walk the event graph, rebuild a transient CRDT structure (tombstones reappear transiently), discard at causally-linear points	<i>refold</i> : $O(\text{survivors})$ coordinate recomputation; no replay, no transient structure, no order decisions
sequential-editing cost	$\approx$ zero (plain edits; no metadata)	$\approx$ zero order metadata (~ 4 bits/level for continuation typing; a cursor jump pays log-gap once, at the run head)
concurrency cost	replay time proportional to the concurrent region, at every affected merge	coordinate bits proportional to $\log(\text{ts gap})$, once, in space
ordering semantics	Yjs-variant (FugueMax-adjacent); maximal non-interleaving <i>conjectured</i> , not proved; two-sided (no L19 analogue)	$\equiv$ published RGA (machine lockstep); one-sided: L19 backward-run interleaving inherited; removed by the sided generalization (§5), RGA kept as the all-R fragment
correctness artifact	informal proof (Appendix C) + property testing	L1–L25 battery + DAG PBT + lockstep vs RGA _† ; RA-linearizability kernel-checked in Lean (§12); the design was chosen so the merge proves itself (no order decisions to verify)
op payload	integer origin positions (compact, human-meaningful)	two timestamps + ghost prefix (droppable on the wire; §3)

Four sentences of honest positioning. *(i)* Eg-walker achieves its clean document state by keeping the entire history and re-deriving order on demand; the embedded-chain RGA achieves it by making order derivation unnecessary: the sort keys are written at birth and never edited, and the coordinates are their encoding (§2), precomputed at mint time and exact forever. *(ii)* Eg-walker’s ordering target was stronger where L19 is concerned (Fugue-style two-sidedness removes backward interleaving, while the one-sided design inherits RGA’s anomaly by construction, being read-equal to RGA); the sided generalization of §5 adopts exactly that two-sidedness as a parameter, closing the gap while keeping the one-sided datatype as its all-R fragment; the Fugue-Max variant’s maximal non-interleaving, which Eg-walker’s shipped ordering only conjectures, is here a kernel-checked theorem (§6). *(iii)* The verification gap runs the other way: Eg-walker’s replay-transform-clear loop is algorithmically sophisticated and only informally proved; here the merge performs no arbitration at all, the metatheory is one-line at the chain layer plus a single faithfulness theorem at the encoding layer, and RA-linearizability is a kernel-checked theorem in a framework that had already verified a dozen-plus replicated datatypes end-to-end (§12). *(iv)* The cost models part company at cursor jumps: a single-user history that hops back to type at

an old anchor is free for Eg-walker (nothing concurrent to replay) but mints log-gap bits here, once per jump, at the run head; on measured single-user editing traces the jump tail is thin (code bits ≈ 1.4 – 1.8 per level; see the companion note of §2). In the MRDT setting there is also a model fit: Eg-walker must reconstruct merge context by replay because the op-based model gives it none; the MRDT model hands the merge its LCA, which is exactly the context replay rebuilds.

11 Related work

The verified sweep of nearby systems lives in `whiteboard/litmus/related-work.md` (per-claim verdicts, primary sources); this section positions the datatype.

Retention taxonomy. What nearby systems keep for the dead: tombstone records (RGA with cemetery under causal stability; WOOT; Yjs’s per-deletion GC structs, kept forever; Fugue, “we cannot remove a deleted element’s node entirely”; Peritext, “the positions must be remembered”); the full history (Eg-walker’s event graph, Chronofold/causal trees where the log *is* the text); consensus-gated compaction (Treedoc’s flatten requires Paxos-commit); or **dead identifiers inside live position keys** (Logoot/LSEQ, Weidner’s position-strings/list-positions, the Fugue id space). The embedded-chain RGA is in the last class *by information content* (dead ancestors’ timestamps persist inside survivors’ sort keys, at live-reachable scope), but with three deltas to its classmates: the retention is *delta-coded at the entropy of the anchor gaps* (a sequential history costs a constant per element where Logoot-family keys grow with allocation policy), the *birth tree kept whole* gives forward non-interleaving and delete-order preservation (position-key designs interleave, the PaPoC’19 anomaly, and Figma-style fractional indexing is unsound under concurrency by its own account), and the semantics is pinned to the best-studied sequence CRDT by a machine-checked lockstep rather than being a new point in anomaly space.

The impossibility backdrop. Attiya et al. (PODC 2016) prove $\Omega(D)$ metadata for push-based protocols meeting even the weak list spec: the published form of “ordering requires remembering the dead.” This design does not escape the bound; it *relocates and minimizes* the memory: $\Theta(\log \delta)$ bits per dead-chain element inside live coordinates (plus the MRDT version store, which the model provides). The sharper, machine-witnessed form from our design sweep is the *conservation law* (§1): the sort key must retain dead ancestors’ timestamps (the verdict information survives every encoding change: stamp, spine, tombstone, or denominator bits), and ties force separate *metadata* only under timestamp-oblivious minting; the timestamp-faithful mint carries the same bits inside the coordinate. Eight timestamp-oblivious mutation variants each have a named machine-checked countermodel (fold-verdict erasure; repair non-locality); the credential countermodel separates this design from all of them.

Verified datatypes. Isabelle RGA (Gomes et al. OOPSLA’17, op-based convergence), OpSets (list semantics specified by a total order of identified ops: arbitration-by-identity as specification), VeriFx (SMT, convergence properties), MET (TLA+, list CRDT bugs). In the MRDT line (Kaki et al. OOPSLA’19; Peepul PLDI’22; RA-lin OOPSLA’25, this framework), no verified sequence

with sequence-CRDT-grade ordering guarantees existed; our verified rehoming RGA (§1) is RA-linearizable against its own fold yet fails the sequential delete-order spec, since RA-linearizability certifies convergence to the datatype’s *own* semantics and ordering intent must be specified and proved separately. The embedded-chain RGA delivers the missing artifact: RA-linearizability is a kernel-checked Lean theorem (§12); delete-order preservation, display stability and forward non-interleaving are proved at the model layer; and the RGA read-equivalence (the compaction theorem, §12) closes the intent question against the best-studied sequence semantics.

Treedoc’s lesson, inverted. Treedoc re-balances geometry and must pay consensus for it; the principle that emerged from our ledger-based sibling design study (*reprojection without consensus is safe iff geometry has been demoted to a rendering*) is here taken to its fixed point: geometry is never reprojected at all. It is immutable, so it may safely be the only thing.

12 Mechanization: the conditioned discharge and the compaction theorem

The verification framework is Part I’s metatheory; file paths below are relative to the Sal repository.

The capstone is proved (kernel-clean on {propext, Classical.choice, Quot.sound}). Its honesty hypothesis is displayed in full, never folded:

Definition 12.1 (embed honesty, and honest reachability). *EHonest Γ C has two components. (Delete provenance): for every $e \in C.\text{events}$ with $e = (\cdot, \cdot, \text{del}(x))$ there is an $a \in C.\text{events}$ with $C.\text{vis } a \ e$, $a.1 = x$, and a an insert. (Chain generation): there is a global assignment $\text{chainOf} : \mathbb{N} \rightarrow \text{List } \mathbb{N}$ such that every insert $o \in C.\text{events}$ satisfies*

$$\text{PosChain } (\text{chainOf } o.1), \quad \text{coord}(o) = \text{coordOf}_\Gamma (\text{chainOf } o.1), \quad (\text{chainOf } o.1).\text{sum} = o.1 :$$

the carried coordinate is a positive birth chain’s, and the chain’s deltas telescope to the op’s own id. EReach Γ is HonestReach ($E \Gamma \alpha$) (EHonest Γ): LTS reachability where every step is taken from a configuration with an honest history, instantiating Part I’s generic metatheorem exactly as the mergeable queue does. *EHonest, EReach*

Theorem 12.2 (the embed capstone). *For every ordered prefix code Γ and configuration C :*

$$\text{EReach } \Gamma \ C \rightarrow \text{IsRALinearizable3 } C,$$

RA-linearizability per version at every honestly reachable configuration, parametric in the code Γ : the unary code, the binary delta code, and the flipped Elias- δ code are all proved OrderedPrefixCode instances, three encodings on one proof (the Elias- δ inheritance corollary `embed_ra_linearizable3_eliasDelta` is one line). *embed_ra_linearizable3*

The factoring is exactly this part’s design–encoding split, which is itself evidence for the split:

1. **Model layer** (Sal/MRDTs/RGA_Embed/, six files, 0 sorry, plus the read-equivalence order core and the sided kernel noted below): the code-parametric kernel; commutations to state equality under $\text{rc} = \text{Either } (\text{rc_non_comm'})$; `ins_prefix_ghost`; coherence and

chain-generation closures; display stability, with step stability (S2, delete-order preservation at its `del` instance) proved *without any axioms* and merge pairwise stability (S4, the adopted contract); unique decodability (`coordOf_inj`); **the chain-lex theorem** (`display_iff_chainBefore` = Corollary 2.6); non-interleaving as subtree convexity (`subtree_convex`); Theorem 2.5(i) for all three codes (`binEnc_mono/binEnc_prefixFree`, `dEnc_mono/dEnc_prefixFree`, cross-validated against the Python codes by kernel `decide`); `native_decide` SPOTs where the rehoming RGA’s delete-reorder witness gets the opposite verdict.

2. **Conditioned instance** (`Sal/ConditionedMRDTs/MRDT_Instances/EmbedRGA/`, nine sections, 0 sorry, plus the Elias- δ inheritance corollary `EmbedRGA_EliasDelta.lean`), proved via the *mergeable-queue route* of §: a generic join lemma (`JoinLemma3At`: the merge exhibits its own linearization witness) plus an honest-reachability metatheorem (`HonestReach`), rather than the rehoming RGA’s 55-file bespoke chain. The route applies exactly when states are canonical per event set, which is Theorem 1.3(i). Landed: the canonical sorted-list state; strictly-sorted extensionality (`esorted_ext`: sorted lists with the same members are equal, replacing any canonical-list formula); **fold-canonicity**, the Join, and the honesty discharge (Theorems 12.3, 12.4, 12.5 below); the honesty core and its bridge to well-formedness; the merge membership characterization (`e_mergeL_mem`, OR-set survival with honesty closing the cross-branch-delete corner); the capstone (Theorem 12.2); and the instance intent theorems: delete-order preservation *is* sublist stability (`eUpdate_del_sublist` is literally `List.filter_sublist`: a delete’s successor state is a sublist of its predecessor, so the surviving display order is untouched; the merge preserves each input’s survivor order on both sides).

Theorem 12.3 (fold canonicity: Theorem 1.3(i) mechanized). *Call an enumeration ρ well-formed ($\text{EWf } \Gamma \rho$) when its insert ids are duplicate-free, no id is deleted before its insert, and distinct inserts mint distinct keys — the three consequences honesty supplies on any enumeration of a closed event set. For ρ, ρ' with $\text{EWf } \Gamma \rho$, $\text{EWf } \Gamma \rho'$, and $\forall o, o \in \rho \iff o \in \rho'$:*

$$\text{eFold}_{\Gamma} \rho = \text{eFold}_{\Gamma} \rho',$$

well-formed enumerations of one event set fold to the same state.

`e_fold_canon`

Theorem 12.4 (the Join: the merge is its own linearization witness). *For every configuration whose core history is honest ($\text{EHonestCore } \Gamma C$, the two components of Definition 12.1 stated at the core): `JoinLemma3At` ($E \Gamma \alpha$) C . The witness enumeration for a merged pair is the LCA’s enumeration, then branch one’s delta in branch order, then branch two’s news; its delivery-respect obligation falls to closure (a later event sitting in an earlier block would have been pulled into the earlier event set), the within-block orders transferring verbatim because the local order is event-set independent under $\text{rc} = \text{Either}$.*

`e_join_at`

Theorem 12.5 (the honesty discharge: applicable generation forces honesty). *The issuer-side guard `eApplicable o s`: for an insert $(t, r, \text{ins}(e, \pi, a))$, $a < t$ and either $a = 0 \wedge \pi = \varepsilon$ (the root) or some record $(a, \text{el}, \pi) \in s$ — the anchor stored with exactly the carried prefix as its coordinate; for a delete, the target id is stored. If every $e \in C.\text{events}$ admits an enumeration π of its causal past (`listPermOf` $\pi \{e' \in C.\text{events} \mid C.\text{vis } e' e\}$) with e applicable at*

$\text{eFold}_\Gamma \pi$, then $\text{EHonest} \Gamma C$: *per-op honesty needs no bespoke oracle. The delete component is fold provenance; the chain component builds the global assignment by strong induction on ids (anchors have smaller stamps, and the anchor's record in any fold of the issuer's past is op-determined, forcing the carried prefix to be the anchor's chain's coordinate). The framework form: GenHonest at eApplicable plus CausalPastEnumerable C supply exactly this hypothesis ($\text{eHonest_of_genHonest}$).* $\text{eApplicable, eHonest_of_applicable}$

The compaction theorem is proved (kernel-clean): $\text{read} \circ \text{embed} = \text{read} \circ \text{RGA}_\dagger$ on honest executions.

Theorem 12.6 (the compaction theorem). *Hypotheses: the generation discipline $\text{EAppDiscipline} \Gamma C$ (every op applicable at some fold of its issuer's causal past — the same hypothesis Theorem 12.5 discharges honesty from, named); an enumeration ρ with $\text{listPermOf } \rho C.\text{events}$ and $\text{EWf} \Gamma \rho$; and a list L that meets the published RGA's own relational read specification at the tombstoned fold:*

$$L \text{ pairwise-sorted by } \text{visible_lt} (\text{rgaFold } \rho), \quad \forall t, t \in L \iff \text{visible} (\text{rgaFold } \rho) t.$$

*Conclusion: $L = \text{eIds} (\text{eFold}_\Gamma \rho)$ — any enumeration of the tombstoned fold's visible ids in the published order is the embed fold's id sequence — with element agreement: every record $(t, \text{el}, c) \in \text{eFold}_\Gamma \rho$ satisfies $\text{readSeq_visible} (\text{rgaFold } \rho) t \text{ el}$ (embed_rec_readSeq , hypothesis $\text{EWf} \Gamma \rho$ alone). The consumed relations are the published side's own: visible_lt here is the inductive of *Sal/MRDTs/RGA_with_tombstones/RGA_ReadSide.lean* (parent-child, newest-first siblings, left-descendant, transitive); the file is named because the repository carries four homonymous visible_lt declarations across its CRDT and MRDT trees, and the compaction theorem consumes exactly this one.* $\text{rga_read_eq_embed_read, embed_rec_readSeq}$

The proof is against the published definition: the order core (*Sal/MRDTs/RGA_Embed/RGA_Embed_ReadEquiv.lean*) shows visible_lt coincides with chain-lex on the shared birth tree (soundness by induction on derivations, completeness by realizing every chain prefix as an **afters**-reachable ancestor), and en route proves the published relational order total and asymmetric on the birth tree, facts the published side never had. This converts the design into a theorem *about* the published RGA: it admits a strictly-live compaction preserving every read. The lockstep rows of §4 are this theorem's tested form.

The sequential specification is proved, and the design holds the canonical seat. Part III proves embed_seq_sound : on every sequentially honest single-replica history the canonical sorted state is the naive insert-at-position text buffer, record for record, with the adjacency lemma ($\text{chainBefore_snoc_iff}$: a fresh insert's delta beats every existing child's by sum-telescoping) as the geometric heart (§17). The published $\text{RGA}_\dagger$ inherits the same buffer spec through the compaction theorem ($\text{rga_seq_read_eq_buffer}$), and the same statement is refuted for the rehoming design (§18): the embedded-chain family is the repository's canonical sequence RDT, and the canonical fused Peritext was re-based onto this kernel (Peritext_Embed/ : capstone by pure instantiation of the payload-generic instance; renderIds_del , deleting a character never re-formats another, the theorem the rehoming kernel provably lacks; and the tier-4 editor theorem $\text{peritextEmbed_seq_sound}$). One caveat is recorded as Open Question 25: per-replica soundness and convergence do not compose into “the merged read is the naive buffer on some ordering of the union”; the dead-anchor corner (§20) is where stamp-sorted replay diverges from the fold.

The sided generalization is proved (kernel-clean), in the same two layers as §5 presents it. The sided chain-lex kernel (`Sal/MRDTs/RGA_Embed/Sided_ChainLex.lean`): the sided marker theorem `sdisplay_iff_schainBefore` (display comparison of sided coordinates IS the in-order rule), totality of the display relation *without any axioms*, unique decodability (`sidedCoordOf_inj`: the side is recoverable from the first symbol’s band, the delta by prefix-freedom within the band), the all-R *fragment theorem* `schainBefore_liftR` (all-R chains order exactly as one-sided chains: “RGA is the all-R fragment,” mechanized), and in-order-interval subtree convexity (`schain_subtree_convex`, the datatype half of the L19 discharge). The conditioned instance (`MRDT_Instances/SidedRGA/`, nine sections mirroring the one-sided instance with the sided key): fold-canonicity (`s_fold_canon`), the Join by the same mergeable-queue route, and the capstone:

Theorem 12.7 (the sided capstone, restated from Theorem 5.7). *For every ordered prefix code Γ and configuration C :*

$$\text{SReach } \Gamma \ C \rightarrow \text{IsRALinearizable3 } C,$$

where $\text{SReach } \Gamma = \text{HonestReach } (S \ \Gamma)$ ($\text{SHonest } \Gamma$) and SHonest is the verbatim sided mirror of Definition 12.1 (delete provenance; sided chain generation, the mint a positive sided chain’s coordinate with telescoping deltas). The theorem holds for every side assignment: the guard constrains the anchor, never the side, so sides are payload to convergence — the policy/datatype split of §5 as a theorem — with `sHonest_of_applicable` discharging honesty from the generation guard exactly as one-sided (Theorem 12.5). `sided_embed_ra_linearizable3`

The per-policy intent theorems make the policy story a machine-checked three-act arc. *Act one:*

Theorem 12.8 (act one: the all-R fragment at the instance). *For every history ρ — no honesty, no sortedness hypotheses, a pure simulation:*

$$\text{sFold}_\Gamma (\rho.\text{map liftOp}) = (\text{eFold}_\Gamma \rho).\text{map liftRec},$$

the sided fold of an all-R-lifted history IS the one-sided fold, record for record (the sort keys coincide definitionally), so every one-sided theorem transports; the reads corollary `sided_allR_read_eq` agrees id-and-element for id-and-element, and chained with tier 3 and Theorem 12.6, the all-R sided instance = the naive text buffer = the published tombstoned RGA. `sFold_liftOp`

Also in act one: `sided_fold_subtree_convex`, subtrees display as contiguous blocks in any chain-generated fold, so runs that chain never interleave (`fugue_reachable_runs_no_interleave`). *Act two:* the Weidner–Kleppmann maximal-non-interleaving Definition 4 was formalized over reachable configurations (`SidedRGA_Fugue.lean`) and REFUTED twice for this Fugue policy on reachable traces (the recency tiebreak reverses the paper’s lowest-ID-first; their Figure-7 execution): the paper’s own Fugue-vs-FugueMax separation, landing on this implementation. *Act three:* FugueMax realized as a kernel variant (`SidedMax_ChainLex.lean`): its sibling order depends on the right ORIGIN, not expressible by any re-banding of deltas (the mirror control machine-witnesses this), so R-entries carry the mint-time successor’s key as an order-reversing tag; condition (3) is proved in full (`fuguemax_same_origin_low_first`, the positive twin of the refutation), and full Theorem 9 is **closed** (§6: the traversal theory discharges condition (1), the corrected backward exception

discharges condition (2), and `fuguemax_maximally_noninterleaving` is kernel-clean). The sided sequential spec is also proved (`sided_seq_sound`: a sentinel-rooted two-sided buffer, R-inserts splice after their anchor, L-inserts immediately before). The tag has a COST: at contested inserts the coordinate embeds the successor’s key, $\Theta(|\text{key}|)$ where plain Fugue pays zero, and whether that retention is a lower bound for tombstone-free maximal non-interleaving is an open question with a Myhill–Nerode shape (Open Question 25): the entropy law of §2 meeting the ordering intent.

Provenance. `whiteboard/litmus/embed_tree.py` (the design, the timestamp-oblivious control, the credential countermodel, the IEEE 754 measurements; `python3 embed_tree.py / fp / code`); `litmus.py`, `pbt.py` (battery and DAG harness); `contest_tree.py` (lockstep harness, CE regressions); `whiteboard/retention-countermodel.pdf` (the retention arc that forced this design); `whiteboard/delta-tree-design.pdf` (the ledger-based sibling design and the invariants I1–I3); `whiteboard/litmus/related-work.md` (verified source list: Roh et al. JPDC’11; Attiya et al. PODC’16; Kleppmann et al. PaPoC’19; Weidner & Kleppmann TPDS’25 (Fugue); Gentle & Kleppmann EuroSys’25 (Eg-walker, arXiv:2409.14252); Preguiça et al. ICDCS’09 (Treedoc); Grishchenko & Patrakeev PaPoC’20 (Chronofold); Litt et al. CSCW’22 (Peritext); Gomes et al. OOPSLA’17; OpSets 2018; Kaki et al. OOPSLA’19; Peepul PLDI’22; RA-lin OOPSLA’25; VeriFx ECOOP’23; Figma/Wallace fractional indexing; Weidner position-strings).

Part II

Sequential specifications: convergence is not correctness

Replication-aware linearizability certifies that every version of a replicated data type is the fold of some delivery-respecting linearization of its events. The reference point of that guarantee is the datatype’s *own* fold: a bug in `do` appears identically on every replica, converges perfectly, and passes every convergence theorem. This part records the campaign that closes the gap. For every production RDT in Sal there is now a kernel-checked theorem that the fold of any single-replica history equals the output of a direct sequential program, the one a programmer would write with no replication in mind. The campaign’s unifying observation is the *witness discipline*: sequential interfaces address state positionally (`dequeue()` is nullary, `insert` names an index), replicated operations cannot, and every replicated operation therefore carries the identity of what its issuer *observed* the positional referent to be; the honesty side conditions of the theorems are exactly the faithfulness of those witnesses. The campaign’s negative results carry as much information as its theorems: the rehoming tombstone-free RGA is refuted at this specification on a four-op single-replica trace, its fused Peritext inherits the refutation at the render, and Shesha passes sequentially while failing at the merge, so *do*-faithfulness and merge-faithfulness are independent axes, and both separations are kernel-checked. A final section states what the campaign does *not* give: per-replica soundness and convergence do not compose into “the merged state is some user’s sequential result,” and the precise corner where composition fails is the dead-anchor corner the sequence-CRDT trilemma names. Every claim below names its Lean identifier.

13 The specification gap

The framework’s central guarantee is RA-linearizability at every reachable configuration: each version the store registers is $\text{fold}(\rho)$ for some enumeration ρ of the version’s event set that respects delivery order. This is the right guarantee for *replication*: the merge invented nothing, reordered nothing it was not entitled to reorder, and all replicas agree.

It says nothing about whether the fold is *right*. The reference sequence is the datatype’s own `do`; if `do` is wrong, every replica is wrong in unison (Figure 8). Two incidents made this concrete in this repository. First, a mark-extent defect in an early Peritext render passed every convergence obligation and was caught only by eye. Second, and decisively: the rehoming tombstone-free RGA carries a fully general, kernel-clean RA-linearizability capstone (`rga_ra_linearizable3_eq`), and its `do` nonetheless reorders surviving text when an interior element is deleted, with no concurrency anywhere in the history (§18). Both defects are invisible to convergence because convergence is self-referential.

The remedy is an independent specification. For sequential behaviour there is a canonical one: *the sequential program a programmer would write for the same operations with no replication in mind*. A counter is addition. A queue is a list. A text buffer is insert-at-position and delete. A marked-text editor is a buffer of characters and boundary tokens with one left-to-right walk.

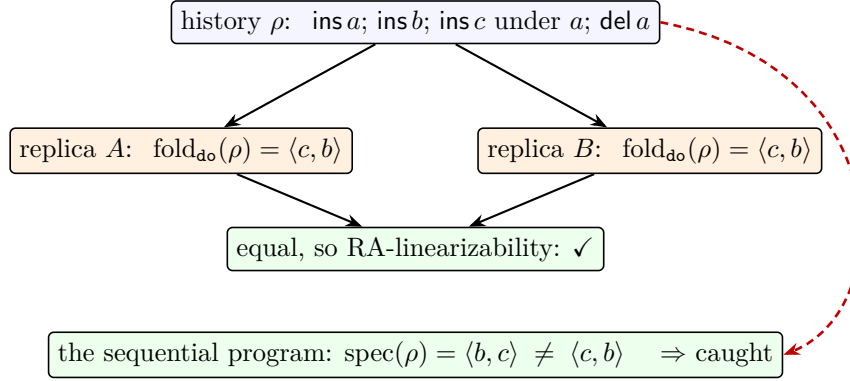


Figure 8: Why convergence cannot see a `do` bug. A defective fold is applied identically everywhere, so all replicas agree and every convergence theorem passes over the shared mistake. Only a reference computed by something *other than* `do` (dashed) can disagree. The history shown is the real refutation trace of §18.

14 The obligation, and the rules of engagement

Convention (Part III plumbing). Three pieces of plumbing are folded out of every Part III display below; Part I’s convention (§) and Part II’s (§1) remain in force. (vi) *The single-replica fold:* histories ρ are plain lists of events (t, r, o) , and the fold is always `seqFold` of Definition 14.1; “the view” of a datatype means its named user-visible projection: the view column of Table 1 for the flat tier and, for the queue and the sequence datatypes, the value map and the record projections `eProj/sProj`. (vii) *The prefix-split honesty shape:* every honesty predicate below quantifies over decompositions $\rho = \sigma ++ o :: \tau$ in the one shape of Definition 15.1, so per-datatype displays state the two clauses at the split and elide the (identical) quantification. (viii) *Snoc/prefix plumbing:* the step lemmas (`seqFold_snoc` and the per-spec `*SpecFold_snoc`) and the prefix-closure lemmas (`*OK_prefix`) are induction plumbing, extending Part I’s `applySeq` fold, and never appear in displays.

Definition 14.1 (the sequential fold, and the campaign obligation). *For a datatype D and a single-replica history ρ , the sequential fold is the datatype’s own replay from its initial state,*

$$\text{seqFold } D \ \rho = \text{fold}(\sigma_0, \rho) \quad (\text{the } \texttt{applySeq} \text{ fold of } \S, \text{ at } D),$$

and the campaign’s obligation, per datatype, is the schema

$$\begin{aligned} &\text{for every sequentially honest single-replica history } \rho : \\ &\quad \text{view}(\text{seqFold } D \ \rho) = \text{spec}(\rho), \end{aligned}$$

with spec a sequential program written from the datatype’s user-facing intent and view its user-visible projection. Every `*_seq_sound` theorem of this part is an instance of the schema. `seqFold`

The obligation is proved by discovering an inductive invariant of the fold, never by weakening the spec to match the implementation. “Sequentially honest” is the datatype’s own issuing discipline specialized to a linear history (fresh, monotone stamps; each operation applicable at the state it executes against), so the theorems attack no strawman: the histories quantified over

are exactly the ones a single well-behaved replica produces, and in every case the side condition reuses a contract the convergence proofs already consume.

Three method rules were fixed in advance and held throughout. *Falsifiable statement first*: each tier states its equality before proof work begins, and a refutation is a first-class outcome (three, §18). *The spec is external*: the sequential program is written from the datatype’s user-facing intent, never read off the implementation; where the two disagree, the disagreement is the result. *Tests carry both polarities*: every concrete test block pairs its expected values (hand-derived, never evaluated out of the code under test) with should-fail companions, so the harness demonstrably discriminates; the refutation files pair each negative with the positive prefix on which the two sides still agree, isolating the departing operation exactly.

15 The observation gap: positional interfaces, witnessed ops

Writing out the specifications exposes one structural pattern. A sequential structure addresses its state *positionally or implicitly*: `dequeue()` takes no argument because “the head” is unambiguous; `insert(i, e)` names an index; `write(v)` implicitly overwrites everything before it. A replicated datatype can afford none of this, because positions and implicit referents are not stable under concurrent edits: by the time an operation is applied elsewhere, “the head” may be a different element and index i a different character. So the replicated operation carries a **witness**: the identity (tag, stamp, id, coordinate, snapshot) of what the issuing replica *observed* the positional referent to be, at its own state, at issue time (Figure 9).

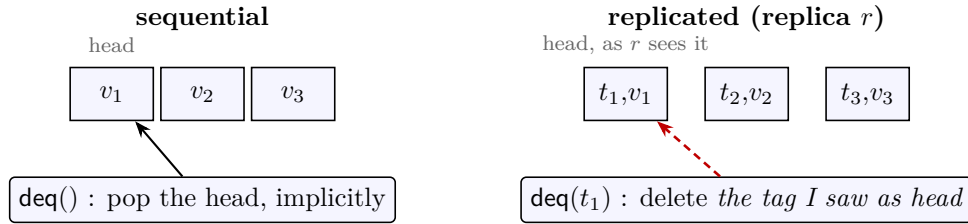


Figure 9: The observation gap, on the queue. The sequential `dequeue` is nullary; the replicated one names the tag its issuer read as the head of its own replica (dashed: the witness). Honesty is the faithfulness of the witness: `qOK` states that the fold at the issue point is literally $(t_1, v_1) :: \text{rest}$.

The sequential-honesty side conditions of §14 are then exactly the statement *the witness is what the sequential operation would have seen*, and each campaign theorem says: under that discipline, the witnessed-identity program implements the positional program. Where no witness is needed the alphabets coincide and the spec is a view homomorphism. The whole catalogue of §16 sorts by this principle, and the summary is one sentence: *every honesty condition in the campaign is the faithfulness of a witness that replaced a positional argument*.

Definition 15.1 (sequential honesty is witness faithfulness). *Every campaign honesty predicate has one shape: for every decomposition $\rho = \sigma \dashv\vdash o :: \tau$,*

(freshness) o ’s stamp is new at `seqFold D  $\sigma$` , and

(witness faithfulness) o ’s carried witness is what `seqFold D  $\sigma$` shows.

The six mechanized instances:

<i>predicate</i>	<i>freshness</i>	<i>witness faithfulness</i>
<i>stampsMono</i>	<i>stamps strictly increase:</i> $\rho.\text{Pairwise}(o.1 < o'.1)$	<i>none: the stamp itself is the arbitra-</i> <i>tion payload</i>
<i>mvrOK</i>	<i>o's stamp staked under no</i> <i>value</i>	<i>the snapshot O lists exactly the visible</i> <i>stamps</i>
<i>qOK</i>	<i>the enq tag is unstaked</i>	<i>deq t: the fold is literally $(t, v) :: \text{rest}$</i>
<i>eSeqOK</i>	<i>the stamp exceeds every</i> <i>prior insert stamp</i>	<i>eApplicable: the anchor is live, car-</i> <i>rying exactly π (Definition 16.8)</i>
<i>sSeqOK</i>	<i>same</i>	<i>sApplicable, two-sided (Theo-</i> <i>rem 17.5)</i>
<i>StepsHonest</i>	<i>fresh_ts at the state</i>	<i>accurate: the claimed path is the</i> <i>true ancestor chain (Definition 18.1)</i>

stampsMono constrains stamps pairwise rather than by split (equivalent to freshness at every split), and *StepsHonest* recurses along the history rather than quantifying over splits: the same content in structural clothing. The remaining four are literally the displayed $\forall$ -split shape. *stampsMono*, *mvrOK*, *qOK*, *eSeqOK*, *sSeqOK*, *StepsHonest*

16 The specifications, datatype by datatype

16.1 No gap: the alphabets already coincide

For ten of the thirteen flat datatypes the RDT operation is literally the sequential operation; the state carries invisible bookkeeping (tags, per-replica slots) and the spec is the evident program on the user-visible view. Table 1 is the tier's display, one row per datatype, every theorem cell a resolving Lean identifier (`SeqSpec_Flat.lean`, thirteen datatypes, fourteen theorems: the AWPQ's spec is two-part). No honesty hypothesis appears in the first ten rows: those theorems quantify over *all* histories, each view a fold homomorphism, no invariant needed. The three rows below the rule are the gap cases of the next two subsections; their honesty column names the one extra hypothesis each theorem carries.

16.2 Gap: arbitration by stamp instead of program order

LWW and FWW registers. The sequential register is *write* v with *program order* deciding who wins; the spec programs are “the last write's value” (`lwwSpecFold`) and “the first write's value” (`fwwSpecFold`). The RDT operation carries its Lamport stamp *as the arbitration key in the payload* (a max- resp. min-semilattice on stamped writes), because concurrent replicas have no shared program order to appeal to. The gap is a transfer of ordering authority from control flow to data, and the honesty condition is precisely its repair: **stampsMono** (stamps strictly increase along the history, the sequential Lamport condition), under which the arbitration key agrees with program order and `lww_seq_sound` / `fww_seq_sound` recover last- and first-write semantics. Without **stampsMono** the statements are false, and should be: a replica that back-dates a write is asking LWW to disagree with program order.

16.3 Gap: overwrite by snapshot instead of “everything”

Multi-valued register. The sequential *write* v implicitly overwrites everything before it. The RDT operation is *write* (v, O) where O is a **prepare-time snapshot**: the write-stamps the issuer

datatype	view	spec program	honesty	theorem
Counter	the state ($\mathbb{Z}$)	the history length	—	<code>counter_seq_sound</code>
Incr.-only counter	the state	the history length	—	<code>ioc_seq_sound</code>
PN-counter	the state	<code>pnSpec: #inc - #dec</code>	—	<code>pn_seq_sound</code>
Grow-only set	membership	some op added x	—	<code>goset_seq_sound</code>
Grow-only map	membership at (k, v)	some op added (k, v)	—	<code>gomap_seq_sound</code>
G-set (conditioned)	<code>gsetView</code>	some op added x	—	<code>gsetcond_seq_sound</code>
OR-set	<code>orView</code>	<code>orSpecFold: a plain</code> add/remove set	—	<code>orset_seq_sound</code>
OR-set (efficient)	<code>orEView</code>	<code>orSpecFold, the same</code> program	—	<code>orsete_seq_sound</code>
Enable-wins flag	<code>ewView</code>	<code>ewSpecFold: one</code> boolean	—	<code>ewflag_seq_sound</code>
Add-wins PQ	<code>awpqMemView</code> ; the inc component	<code>awpqSpecFold (inc</code> <code>inert); the issued-</code> increment log	—	<code>awpq_mem_seq_sound</code> , <code>awpq_inc_log_sound</code>
Multi-valued reg.	<code>mvrView</code>	<code>mvrSpecFold: the last</code> write	<code>mvrOK</code>	<code>mvr_seq_sound</code>
LWW register	<code>lwwView</code> $\circ$ <code>lwwSt</code>	<code>lwwSpecFold: the last</code> write	<code>stampsMono</code>	<code>lww_seq_sound</code>
FWW register	<code>fwwView</code> $\circ$ <code>fwwSt</code>	<code>fwwSpecFold: the first</code> write	<code>stampsMono</code>	<code>fww_seq_sound</code>

Table 1: Tier 1, the flat datatypes (`SeqSpec.Flat.lean`). Each theorem states: the view of `seqFold` equals the spec program’s result, under the honesty column’s hypothesis (none above the rule). Views are the file’s named projections; the two counters read the state itself, and `counter_seq_sound` / `ioc_seq_sound` conclude `seqFold D ρ = |ρ|` directly.

currently sees, named explicitly so that a *concurrent* write (absent from O) survives, which is the point of an MVR. The sequential spec is still the plain last-write register (`mvrSpecFold`: the final write’s value, `none` on the empty history).

Definition 16.1 (MVR sequential honesty). `mvrOK ρ`: for every decomposition $\rho = \sigma \uparrow\uparrow o :: \tau$,

$$\begin{aligned} &\forall v'. (\text{seqFold MVR } \sigma).1 (o.1, v') = \text{false}, \text{ and} \\ &\forall w O. o \text{ carries } \text{write}(w, O) \rightarrow (\forall n. n \in O \iff \text{mvrVis}(\text{seqFold MVR } \sigma) n) : \end{aligned}$$

the stamp is fresh (staked under no value), and the overwrite snapshot O lists exactly the issuer’s visible stamps (`mvrVis`: staked, not overwritten). This is the formal form of “I overwrite precisely what I can see.” *mvrOK*

Theorem 16.2 (the discovered invariant: overwritten stamps are staked). For every ρ with `mvrOK ρ`:

$$\forall n. (\text{seqFold MVR } \rho).2 n = \text{true} \rightarrow \text{mvrTag}(\text{seqFold MVR } \rho) n,$$

every overwritten stamp is a staked tag (`mvrTag`), which is what makes a fresh stamp provably not-yet-overwritten. The campaign’s first genuinely discovered invariant. *mvr_over_tags*

Theorem 16.3 (MVR, sequentially a last-write register). *For every ρ with $\text{mvrOK } \rho$ and every value v :*

$$\text{mvrView}(\text{seqFold MVR } \rho) v \iff \text{mvrSpecFold } \rho = \text{some } v,$$

the visible-value view of the fold is the last write.

mvr_seq_sound

16.4 Gap: dequeue() becomes delete-what-I-saw

The mergeable queue, the cleanest specimen (Figure 9). The sequential queue is

$$\text{enq } v : S \mapsto S \uparrow [v] \quad \text{deq}() : S \mapsto \text{tail}(S).$$

The RDT operation is $\text{deq } t$: remove *by tag*, t being the tag of the element the issuing replica read as the head of its own queue; the implementation filters t out of the whole list, wherever it sits.

Definition 16.4 (queue sequential honesty, and the FIFO spec). $\text{qOK } \rho$: *for every decomposition $\rho = \sigma \uparrow o :: \tau$,*

$$\begin{aligned} &\forall v. o \text{ carries } \text{enq } v \rightarrow o.1 \notin \text{qTags}(\text{seqFold } Q \sigma), \text{ and} \\ &\forall t. o \text{ carries } \text{deq } t \rightarrow \exists v \text{ rest. } \text{seqFold } Q \sigma = (t, v) :: \text{rest} : \end{aligned}$$

enqueue stamps are fresh, and a dequeue's witnessed tag is literally the current head. The spec program keeps the sequential shape, blind to the tag: qSpecStep appends the value on $\text{enq } v$ and takes tail on $\text{deq } t$; qSpecFold folds it from $[]$.

qOK, qSpecStep, qSpecFold

Theorem 16.5 (the discovered invariant: tags stay duplicate-free). *For every ρ with $\text{qOK } \rho$: $(\text{qTags}(\text{seqFold } Q \rho)).\text{Nodup}$. Fresh stakes append disjointly, dequeue filters, and nodup survives sublists; this is exactly what makes filter-by-tag remove one element.*

q_tags_nodup

Theorem 16.6 (tier 2: the mergeable queue, sequentially a plain FIFO). *For every ρ with $\text{qOK } \rho$:*

$$(\text{seqFold } Q \rho).\text{map snd} = \text{qSpecFold } \rho,$$

delete-what-I-saw implements pop-the-head.

queue_seq_sound

The theorem is kernel-checked list induction end to end; the campaign's in-file `#print axioms` audit trail lives in the sided, Peritext, and refutation files (§21 collects the provenance). The same witness discipline is what the convergence side already consumed (`qHonest_of_applicable`); nothing was invented for the occasion.

16.5 Gap: insert-at-index becomes insert-after-who-I-saw

The embedded-chain RGA. The sequential text buffer is $\text{insert}(i, e)$ and $\text{delete}(i)$, index-addressed. The RDT operation is $\text{ins}(e, \pi, a)$: a is the *id* of the element the issuer saw at position $i - 1$ (its anchor; the sentinel 0 for the head), and π is the anchor's stored coordinate as the issuer read it (the proof-carrying prefix); $\text{del}(x)$ names the id seen at position i .

Definition 16.7 (the buffer in id-clothing). *The spec program over (id, element) pairs:*

$$\begin{aligned} \text{eSpecStep } S \ (t, r, \text{ins}(e, \pi, a)) &= \begin{cases} (t, e) :: S & a = 0 \\ \text{eInsAfter } a \ (t, e) \ S & \text{otherwise,} \end{cases} \\ \text{eSpecStep } S \ (t, r, \text{del}(x)) &= S.\text{filter } (p.1 \neq x), \end{aligned}$$

with $\text{eInsAfter } a \ p$ splicing p immediately after the entry carrying id a ; eSpecFold folds it from $[]$. On a sequential history the anchor id determines the index uniquely, so this is the positional program with positions named by their occupants. *eInsAfter, eSpecStep, eSpecFold*

Definition 16.8 (embed sequential honesty). $\text{eSeqOK } \Gamma \ \rho$: for every decomposition $\rho = \sigma ++ o :: \tau$,

$$(\forall x \in \text{eInsIds } \sigma. x < o.1) \wedge \text{eApplicable } o \ (\text{eFold}_\Gamma \ \sigma),$$

the stamp exceeds every prior insert stamp (the Lamport condition), and the op is applicable at the current fold. **eApplicable** is Part II’s issuer-side guard: an insert’s anchor precedes its stamp ($a < t$) and either is the sentinel with the empty prefix ($a = 0, \pi = []$) or is live carrying exactly π as its stored coordinate; a delete’s target is live. *eSeqOK*

The theorem (Theorem 17.2) is the strongest form in the campaign: the canonical sorted state equals the spec buffer *record for record*. The same spec, taken at the rehoming op alphabet (ids only, values dropped), is what the rehoming RGA is refuted against (§18); its gap is not the witness (the rehoming op carries one too, plus a path) but the implementation moving *other* elements’ referents on delete.

16.6 Gap: one mark becomes a boundary pair

Fused Peritext. The sequential marked-text editor has an *atomic mark*(i, j, m). The fused RDT has no such operation: a mark is **two** ordinary inserts of boundary elements $\langle m \rangle$ and $\langle /m \rangle$ sharing a fresh mark id, anchored at the range ends; mark removal is two deletes. An atomic positional operation is traded for a *pair* of witnessed inserts, which is the atomicity horn of the mark-positioning trilemma, accepted knowingly. The spec therefore lives at the token level.

Definition 16.9 (the naive marked-text editor). *The editor’s document is a plain buffer of (id, element) entries over $\text{char} \oplus \text{boundary}$: $\text{editorFold } \rho = \text{eSpecFold } \rho$ at the rich payload, Definition 16.7 verbatim. Its screen is one left-to-right walk maintaining the open-mark set: a start boundary opens its mark, the matching close removes it, characters display with what is open, boundaries are invisible (editorRender); the id-tagged variant editorRenderIds additionally reports each character’s id. *EditorBuf, editorFold, editorRender, editorRenderIds**

Theorem 17.6 says the datatype’s render is that editor’s screen, id for id in the `_ids` variant; the range-level reading is recovered by the positional intent theorems (**render_id_active_iff_between**: a character carries mark m iff it sits, in reading order, strictly between m ’s boundaries; an element-list-level theorem of the shared read layer, **Peritext_Rehoming/Peritext_Read.lean**, applying to this instance’s `docElts` verbatim). The pair encoding is also why the character/boundary distinction is load-bearing in the delete theorem: deleting a *character* re-formats nothing (**renderIds_del**), while deleting a *boundary* is mark removal and must re-format its span (machine-checked necessary:

`boundary_delete_breaks_filter_eq`). The boundary-pair encoding is one of the repository's two mark models; the other, the document-order model of separate mark records that the runtime ships, with its own read rules, no-leak theorem, and garbage collector, is §9.

17 The proofs: four tiers, three kinds of invariant

The campaign ran easiest to hardest, and the difficulty gradient turned out to be a gradient of invariant *kinds*.

Tier 1 (the thirteen flat datatypes, fourteen theorems), `SeqSpec_Flat.lean`. Table 1 is the display. Mostly view homomorphisms, no invariant at all. The one exception needs *identity hygiene*: a fact about tag bookkeeping that holds on every reachable single-replica state and is exactly what the induction consumes: the MVR's `mvr_over_tags` (Theorem 16.2, overwritten stamps are staked). The AWPQ, despite its two-part spec, needed none: both of its theorems are invariant-free fold inductions.

Tier 2 (the mergeable queue), `MergeableQueue_SeqSpec.lean`. One identity-hygiene invariant, `q_tags_nodup` (Theorem 16.5), feeding the capstone Theorem 16.6 of §16.4.

Tier 3 (the embedded-chain RGA), `EmbedRGA_SeqSpec.lean`. The invariant becomes *geometric*. The spec splices an insert immediately after its anchor; the datatype sorts records by birth-chain coordinates; the bridge is the adjacency lemma (Figure 10), whose proof costs only sum-telescoping: a chain's deltas sum to its id, and every existing id is below the fresh stamp.

Theorem 17.1 (the adjacency lemma). *For chains $c_r \neq c_a$ and a delta δ exceeding every delta c_r hangs directly off c_a (whenever $c_r = c_a ++ d :: \text{rest}$, $d < \delta$):*

$$\text{chainBefore } c_r (c_a ++ [\delta]) \iff \text{chainBefore } c_r c_a,$$

appending the fresh delta changes no display verdict against the anchor: the newcomer lands immediately after it. *`chainBefore_snoc_iff`*

Theorem 17.2 (tier 3: the embed RGA, sequentially the naive buffer). *For every ordered prefix code Γ and every ρ with `eSeqOK  $\Gamma$   $\rho$` (Definition 16.8):*

$$(\text{eFold}_\Gamma \rho). \text{map } \text{eProj} = \text{eSpecFold } \rho,$$

the canonical sorted state is the spec buffer, record for record.

`embed_seq_sound`

Two transports come for free. The published tombstoned RGA is read-equivalent to the embed on honest event sets (the compaction theorem, §12), so it inherits the buffer spec:

Theorem 17.3 (the tombstoned-RGA transport). *For every Γ , every ρ with `eSeqOK  $\Gamma$   $\rho$` , and every id list L that reads the published RGA's fold, i.e. L pairwise-sorted by `visible_lt (rgaFold  $\rho$ )` and $\forall t. t \in L \iff \text{visible}(\text{rgaFold } \rho) t$:*

$$L = (\text{eSpecFold } \rho). \text{map } \text{fst},$$

and the conclusion never mentions the code. *Element agreement* is the companion `rga_seq_read_element`: every buffer entry (t, e) is read visible by the published RGA at id t with element e . (`EmbedRGA_SeqSpec_RGA.lean` is a standalone build target: the two RGA models share top-level names.) `rga_seq_read_eq_buffer`

And the sided generalization simulates the one-sided instance on all-R histories unconditionally (Theorem 5.6, `sFold_liftOp`), so the buffer spec transports to the sided datatype’s all-R fragment at zero proof cost. The full two-sided theorem needs the two sided adjacency lemmas at the kernel order:

Theorem 17.4 (the sided adjacency pair). *For sided chains $c_r \neq c_a$ and a fresh delta δ :*

(R) *if $d < \delta$ whenever $c_r = c_a \mathbin{++} (R, d) :: \text{rest}$, then*

$$\text{schainBefore } c_r (c_a \mathbin{++} [(R, \delta)]) \iff \text{schainBefore } c_r c_a;$$

(L) *if $d < \delta$ whenever $c_r = c_a \mathbin{++} (L, d) :: \text{rest}$, then*

$$\text{schainBefore } (c_a \mathbin{++} [(L, \delta)]) c_r \iff \text{schainBefore } c_a c_r;$$

a fresh R-extension displays immediately after its anchor, a fresh L-extension immediately before it (the L band is mirrored: the newest L-child hugs the node from above). `schainBefore_snocR_iff`, `schainBefore_snocL_iff`

Theorem 17.5 (the sided capstone). *The two-sided spec is sentinel-rooted: `sSpecFold` folds `sSpecStep` from the seed $[(0, 0)]$, where an R-insert splices immediately after its anchor’s entry (`sInsAfter`), an L-insert immediately before it (`sInsBefore`), and delete filters; `sRFold $\Gamma$` is the canonical fold seeded with the sentinel record $(0, 0, [])$ (`rootRec`), so both insert directions have a splice point and the front insert is no special case. For every Γ and every ρ with `sSeqOK  $\Gamma$   $\rho$` (monotone insert stamps and `sApplicable` at each unrooted fold, the prefix-split shape of Definition 15.1):*

$$(\text{sRFold}_\Gamma \rho).map \text{sProj} = \text{sSpecFold } \rho.$$

The read corollary `sided_seq_read` drops the sentinel on both sides. Both carry in-file audits: axioms $\{\text{propext}, \text{Quot.sound}\}$. `sided_seq_sound`, `sSeqOK`

Tier 4 (fused Peritext), `PeritextEmbed_SeqSpec.lean`. The invariant kind is *composition*, and that is the finding.

Theorem 17.6 (tier 4: the render is the editor’s screen). *For every Γ and every rich-payload history ρ with `eSeqOK  $\Gamma$   $\rho$` (Definition 16.8 at $\alpha := \text{PeritextElt}$):*

$$\text{renderRichText } (\text{eFold}_\Gamma \rho) = \text{editorRender } (\text{editorFold } \rho),$$

$$\text{renderIds } (\text{eFold}_\Gamma \rho) = \text{editorRenderIds } (\text{editorFold } \rho),$$

screen for screen and, in the second form, id for id. In-file audits printed for both. `peritextEmbed_seq_sound`, `peritextEmbed_seq_sound_ids`

The proof is Theorem 17.2 made payload-generic (the fold’s $(\text{id}, \text{element})$ sequence is the naive buffer) composed with one shared render fold (`editorRenderIdsAux_map_eProj`: the datatype’s render and the editor’s screen are literally the same pure fold over that sequence). No new invariant was needed. This is also why the tier had to wait for the re-based Peritext: against the rehoming kernel the same statement is false (§18).

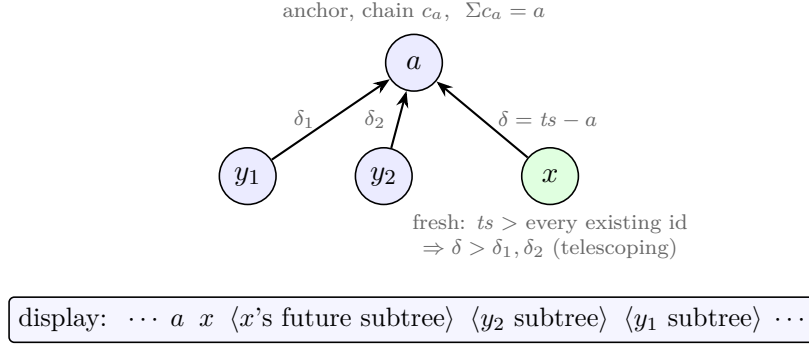


Figure 10: The adjacency lemma, the geometric heart of tier 3. Siblings display newest-first; a fresh insert’s delta beats every existing child’s because deltas telescope to ids and every existing id is below the fresh stamp. Hence the newcomer lands *immediately* after its anchor: exactly what the sequential buffer’s splice does.

18 The refutations

The statement shape is falsifiable per datatype, and three refutations follow, all machine-checked, each with its honesty guards discharged on the witness so the failure is never an artifact of a malformed trace.

The rehoming RGA fails at do. Figure 11. The refuted statement is displayed in full, then negated; its honesty guard is the datatype’s *own* conditioning, so the failure is no artifact of a malformed trace.

Definition 18.1 (the refuted statement). RehomingSeqSound: *for all op lists ops and id lists ids,*

$$\begin{aligned} &\text{StepsHonest ops} \rightarrow \text{ids pairwise descending} \rightarrow \\ &(\forall i. \text{contains}(\text{replay ops}) i \rightarrow i \in \text{ids}) \rightarrow \\ &\text{document}(\text{replay ops}) \text{ ids} = \text{bufFold ops}, \end{aligned}$$

the exact statement shape tiers 1–3 prove: on every honest single-replica history, read with a complete descending candidate list, the document is the naive buffer. Here replay is the datatype’s own do_fold, bufFold the §16.5 buffer program at this op alphabet (bufIns after the named anchor, head for the sentinel, filter delete), and StepsHonest requires every op accurate (the claimed path is the true ancestor chain) and fresh_ts at the state it executes against: the same predicates the datatype’s commutation layer conditions on. RehomingSeqSound, StepsHonest, bufFold

Proposition 18.2 (refutation 1: the rehoming RGA is not the naive buffer). $\neg \text{RehomingSeqSound}$. *The witness opsW is four ops on one replica, stamps 1,2,3,9: insert a then b at the root (siblings re-sort newest-first, reading $\langle b, a \rangle$), insert c under a, delete a. Honesty is discharged on the witness (stepsHonest_opsW); the naive buffer keeps survivor order, bufFold opsW = $\langle b, c \rangle$ (bufFold_opsW), while the rehoming do re-homes c to the root, where the newest-first tiebreak leapfrogs it over b: the read is $\langle c, b \rangle$ (replay_opsW_read). The*

should-pass half (seq_agrees_before_delete) shows both sides agree on the delete-free prefix, isolating the delete as the exact point of departure. In-file audits printed for both halves.
rehoming_seq_refuted

The design’s convergence capstone is untouched: it converges, to its own reordering fold. This refutation is why the design does not hold the repository’s canonical seat.

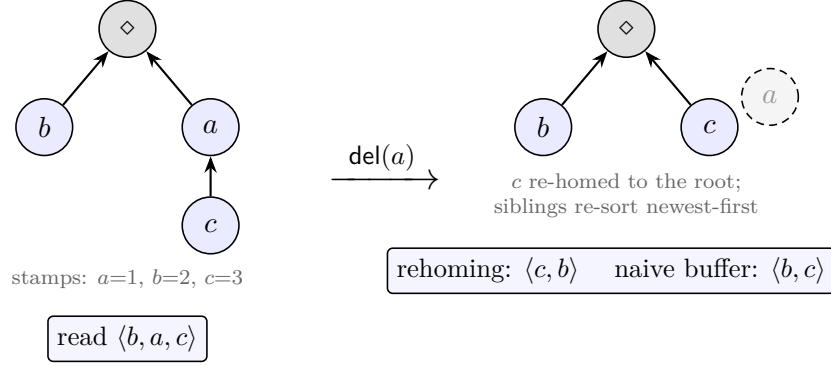


Figure 11: The four-op refutation. One replica, no merge. Deleting a swaps the surviving b and c in the rehoming RGA’s read; the naive buffer keeps their order. The embedded-chain RGA reads $\langle b, c \rangle$ on the same trace: c ’s immutable coordinate still carries a ’s chain as pure geometry, so nothing moves.

Fused Peritext on the rehoming kernel fails at the render. The same reorder lifted to $\text{char} \oplus$ boundary.

Proposition 18.3 (refutation 2: a fused delete re-formats a survivor). *On the rich-text witness docResidual (five **do_** steps; bold span exactly $\{X\}$, characters P and C plain), deleting the plain character P (id 3) and reading both sides at their complete descending candidate lists, projected at the bold mark:*

$$\begin{aligned} & \text{renderRichText}(\text{do_docResidual}(9, 0, \text{del}([2, 1], 3))) \\ & \neq (\text{renderRichText docResidual}).\text{filter}(\text{codepoint} \neq P) : \end{aligned}$$

*the post-delete render is not the pre-delete render minus the deleted character. The untouched C re-homes past the close boundary into the bold span (**fused_delete_moves_char_into_span**), on one replica, no boundary touched. This upgraded the re-base of Peritext onto the embed kernel from housekeeping to a correctness fix.*
fused_delete_reformats_survivor

On the embed kernel the corresponding *general* theorem holds, with the survivor claim quantified over every state:

Theorem 18.4 (character deletes never re-format, on the embed kernel). *For every Γ , every state s , all ts, r, x , if x is a character in s (every record of s with id x carries some $\text{char } c$):*

$$\text{renderIds}(\text{eUpdate}_{\Gamma} s (ts, r, \text{del}(x))) = (\text{renderIds } s).\text{filter}(c.1 \neq x),$$

the post-delete render is the pre-delete render with exactly x 's entries removed: every surviving character keeps codepoint and formatting predicate bitwise, because character entries are open-set-neutral and delete is a filter on an order-canonical state. In-file audit: {propext, Quot.sound}, no choice. The character hypothesis is machine-checked necessary: a boundary delete refutes the filter equation (`boundary_delete_breaks_filter_eq`), as it must, since a boundary delete is mark removal and re-formats its span. The user-facing corollary at the plain read is `renderRichText_del`. `renderIds_del`

Shesha passes sequentially and fails at the merge. The mirror image.

Theorem 18.5 (Shesha's positive half: sequentially the naive linked list). *Unconditionally, for every op list ops:*

$$\text{read}(\text{fold ops}) = \text{seqFold ops},$$

the datatype's read is the naive-list fold: insert after the named anchor, filter delete. (This file's `seqFold` is its own naive-list program, a name coincidence with Definition 14.1 that plays no role here.) No honesty hypothesis: the totality choices of both sides agree on every input. Corollary `delete_preserves_survivor_order`, on every state, not only reachable ones: $\text{read}(\text{delete } s \ d) = (\text{read } s).\text{filter}(\cdot \neq d)$; Shesha uniquely preserves survivor order under delete among the tombstone-free designs of its generation. `sequential_soundness`

Proposition 18.6 (refutation 3: Shesha's merge admits no pre-splice row store). *The now-retired row-store residue formerly named `shesha_rows_residue` asserted: for every honest Shesha configuration with transitive irreflexive visibility, downward-closed branch event sets, canonical witnesses, and SCoh-aligned replays, some pre-splice row store (per-anchor rows, duplicate-free, id-bounded, reversing both branches' same-anchor insert orders) expands to every merged row. The declaration and its downstream positive capstone have been removed because that statement is machine-checked **false**: its full quantifier prefix is negated wholesale by `shesha_rows_residue_refuted` (`Shesha/Shesha_Rows_Refuted.lean`, in-file audit printed), on the concrete core*

$$\text{row}(\text{mergeL } st_0 \ st_1 \ st_2) \ 0 = [3, 4, 2]$$

(`cz_merge_read`): the three-way merge splits the deleted marker 1's children 3,2 around the concurrent sibling 4, an order no row store expands to, producing a read that is the fold of no delivery-respecting linearization. `shesha_rows_residue_refuted`

19 The two-axis separation

Finding (do-faithfulness and merge-faithfulness are independent axes). *Together the refutations witness the independence (Figure 12, every placement a named kernel theorem): the rehomeing family occupies merge-yes/do-no (`rga_ra_linearizable3_eq` with Proposition 18.2), Shesha do-yes/merge-no (Theorem 18.5 with Proposition 18.6), and the embedded-chain family shows the top corner is achievable, by one design idea in both sequence datatypes: immutable birth coordinates, so that deletion moves nothing and the merge decides nothing.*

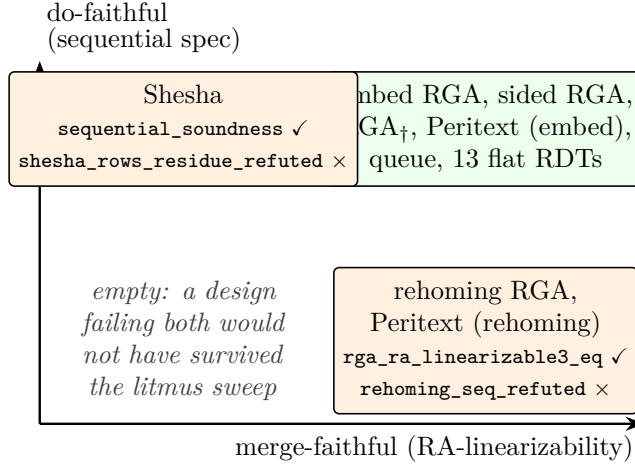


Figure 12: Every placement is a named kernel theorem. The two failure modes are orthogonal: converging to a reordering fold (bottom right) and folding correctly into a non-linearizable merge (top left).

20 What composition does not give

It is tempting to compose the guarantees: RA-linearizability says every version is $\text{fold}(\rho)$ for some delivery-respecting ρ over the event union; the campaign says folds of sequentially honest histories equal the spec program. May one conclude that every merged document is the naive program’s output on *some* ordering of everyone’s operations? **No**, and the failure is precise.

The campaign’s theorems quantify over *sequentially honest* histories; an RA-linearizability witness is delivery-respecting but need not be sequentially honest on either count. Stamp monotonicity fails harmlessly: when the state is a function of the event set (fold-canonicity, Theorem 12.3, `e_fold_canon`, displayed in Part II and not re-displayed here), any enumeration can be re-sorted. Applicability is the real corner.

Example 20.1 (the dead-anchor corner). *Figure 13. In the union of two branches, an insert’s anchor may be deleted by a concurrent operation carrying a smaller Lamport stamp: LCA ins1, branch A ins2 after 1, branch B del1. The merged fold keeps 2 (OR-set survival on immutable coordinates). The branch-order replay [ins1, ins2, del1] reproduces it, but in the stamp-sorted replay [ins1, del1, ins2] the delete precedes the insert, the anchor is dead at the insert’s prefix, and the naive buffer no-ops the splice: $\text{buffer } \langle \rangle \neq \langle 2 \rangle$, while the datatype (whose coordinates travel in the op) places the element correctly. Some delivery-respecting interleavings of the union match the spec program and some do not; deliberately, no Lean identifier is attached, because the openness is the point (Open Question 25).*

Whether a matching order always exists is genuinely open; it is recorded as Open Question 25 in the consolidated list.

Independently of the question, the composition that *is* sound and kernel-checked today: every version is the datatype’s fold of a delivery-respecting enumeration (RA-linearizability); the fold is canonical per event set (Theorem 12.3); each replica’s own contribution obeyed the sequential spec while it was being made (this campaign); and merge-level intent is delivered by separate, named theorems where it holds (delete-order preservation as sublist stability, subtree convexity

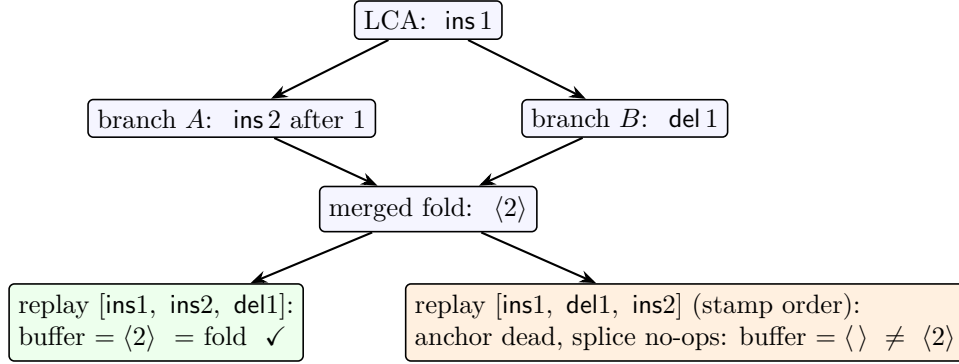


Figure 13: The dead-anchor corner. The merged fold keeps 2 (OR-set survival on immutable coordinates); whether a naive-buffer replay of the union agrees depends on which delivery-respecting order is chosen, and the stamp-sorted one can be wrong. This is the same corner the sequence-CRDT trilemma names for insert-after-already-deleted-anchor.

as non-interleaving, read-equivalence to the published RGA as the compaction theorem). The campaign does not manufacture a merge-level user story; it pins the single-replica story so that merge-level claims have something sound to stand on.

21 The verified matrix

The anomaly-comparison matrix ([whiteboard/anomaly-matrix/](#)) ran six implementations and four production libraries through 25,917 merges per row, every cell backed by executed code. Its in-repo rows are also kernel-checked; the table below is the matrix’s load-bearing core with each cell a Lean identifier. External libraries (Yjs, Automerge, Loro, list-positions) remain empirical by nature; their rows live in the matrix report unchanged.

design	sequential spec (do)	RA-lin (merge)
embedded-chain RGA	<code>embed_seq_sound</code>	<code>embed_ra_linearizable3</code>
sided RGA (two-sided)	<code>sided_seq_sound</code>	<code>sided_embed_ra_linearizable3</code>
published RGA _†	<code>rga_seq_read_eq_buffer</code>	<code>rgaWithTombstones_ra_linearizable3_eq</code>
Peritext (embed)	<code>peritextEmbed_seq_sound</code>	<code>peritextEmbed_ra_linearizable3</code>
mergeable queue	<code>queue_seq_sound</code>	<code>queue_ra_linearizable3</code>
thirteen flat RDTs	the fourteen theorems of Table 1	<code>*_ra_linearizable3_eq</code> (eleven; see note)
rehoming RGA	<code>refuted: rehomining_seq_refuted</code>	<code>rga_ra_linearizable3_eq</code>
Peritext (rehoming)	<code>refuted</code> (render): <code>fused_delete_reformats_survivor</code>	<code>peritext_ra_linearizable_up_to_eq</code>
Shesha	<code>sequential_soundness</code>	<code>refuted: shesha_rows_residue_refuted</code>

Table 2: The verified matrix: the load-bearing core of the anomaly comparison, every cell a resolving Lean identifier. Positive rows above the rule, the three refutations below it (§18).

Every positive theorem above is kernel-clean (axioms $\subseteq \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$, convention fold (iii) of §1); where an in-file audit is printed and recorded, its result can be stronger: `renderIds_del` audits at $\{\text{propext}, \text{Quot.sound}\}$, no choice (re-run and observed); the sided capstones audit at the full kernel-clean triple, choice included. Refutations by concrete witness follow the SPOT convention (`native_decide`, adding the compiler axioms); the refuted *statements* are quantified properties whose honesty guards are discharged

on the witness. On the flat row: the sequential names are the lowercase per-datatype theorems of Table 1; the RA-lin capstones are capitalized (`ORSet_ra_linearizable3_eq`, ..., `LWW_ra_linearizable3_eq`) and match the `*_ra_linearizable3_eq` pattern for eleven of the thirteen; the Counter and the conditioned G-set conclude through the conditioned-discharge route (`counter_ra_linearizable3_cd`, `gset_ra_linearizable3_cd`).

Part III

The verified peer-to-peer runtime

The preceding parts are a metatheory and its instances: theorems about what a merge computes and what a compaction preserves. This part is a systems pillar built on them. The same design runs as a general peer-to-peer MRDT runtime, datatype-parametric over the framework signature, of which collaborative sequence editing is one instance; the runtime’s garbage collectors are the executable form of Part I’s GC-safety and stability theorems, its save format is the run table of Part II, and a benchmark measures it against production CRDT libraries on the standard editing-trace corpus. One boundary is drawn throughout: the Lean development is the verified artifact, and the JavaScript runtime is an *unverified transliteration* of the verified kernel that mirrors its semantics and whose GC callbacks correspond, named theorem by named theorem, to the mechanized results. The runtime does not re-prove convergence; it exercises the semantics the theorems certify and asserts the observable consequences (equal reads, reads unchanged by GC, git round-trip). Every performance number is a run of checked-in scripts on one machine, labeled measured or measured-in-model.

22 The runtime: one content-addressed replica

Datatype-parametric, proven over three datatypes. The core object is `DistributedReplica` (`runtime/src/replica.js`): a single content-addressed commit store carrying three capabilities at once — delta gossip over the wire, the certified stability GC, and the keep-set commit GC — every one of them datatype-agnostic (`init/apply/merge3/read`) except state compaction, which a datatype opts into by additionally providing `compact/remapState`. The runtime is therefore not a text editor with a CRDT inside; sequence editing is one instance. The test suite runs convergence, the content-address round-trip and tamper gate, and the commit GC over the embedded-chain RGA and an OR-set, and the certified state GC over the RGA (refuse-then-fire) with the OR-set correctly declining it (an OR-set has no coordinate re-coding to do). The pieces that Part I proved generic run generically; the third datatype, rich text, follows the certified-GC paragraph below.

Content addressing: one hash for wire and disk. Separate peers assign different local ids to the same commit, so every commit is named by the SHA-256 of its canonical content, a Merkle DAG folding in parent ids (`runtime/src/hash.js`): an authored commit hashes `{parents, replica, seq, payload}`, a merge commit hashes its *sorted* parent ids (so `merge(a, b)` and `merge(b, a)` are literally the same commit and never diverge into a spurious criss-cross), the root hashes `{root}`. Ingest is guarded: a peer recomputes each incoming commit’s state (`apply` for authored, `merge3` for merges) and the recomputed content id must match the wire id, so a transmitted state is never trusted. The pure-JS SHA is checked bit-for-bit against the platform crypto library. The *same* id names a commit on the wire and on disk, which is what makes git persistence content-addressed with no second hash.

Delta sync, head-sync preserved. Two peers exchange their DAG frontiers (head content-ids), each computes the ancestor-set difference the other lacks, and ships it as a *delta*: per commit

only the op payload, parent refs, and author, never a whole state. A whole-state snapshot (the run-table serializer below) is used only for a genuine bulk catch-up, chosen when the delta would be larger (a brand-new or very-far-behind peer). Crucially a peer only ever merges its current head with a head another peer just advertised, never a stale interior commit: this is exactly the head-sync hypothesis the GC-safety theorem (Theorem , §) consumes, made structural rather than advisory. Merges route through the same criss-cross gate as the in-process runtime.

The document is a git repository. Persistence writes one JSON file per commit named by its SHA, a heads file, and a materialized `doc.txt`, then commits them *in a git repository*; loading replays the commits into a fresh replica whose reads equal the original and which re-enters the live wire with identical SHAs. `git clone` of the repository therefore yields a self-contained, loadable document (clone is a join; a fresh replica id is a fork), and because `doc.txt` tracks the content, plain `git log -p` reads as sensible edit history. The content-address SHA is the git-object discipline reused: the same id on disk as on the wire.

Certified GC in the running system. The stability callback is the executable `settledAt_of_allHeard` (Theorem , §). `compactStable` replaces asserted settledness with a *checked* certificate built from the frontier: if any registered replica has not been heard from since the cut, compaction is refused — a no-op reporting the missing replica — and fires only once that replica is heard from. Absence of evidence is refusal, never assumption; this is the runtime witness of the theorem’s `createReplica` breaker. The directed test drives the discriminating-remove countermodel of §: a lagging replica makes `compactStable` refuse, then fire when heard from, with reads identical to a never-compacted control throughout, and a FAIL companion in which the *old* asserted compaction at the same point diverges (a frozen-delta order flip), so the certificate’s refusal is load-bearing, not pessimism. Under a certified cut the in-flight crutch vanishes: every op concurrent with the cut is already delivered (`SettledAt`), so passing an empty in-flight set is *proved* sufficient rather than asserted (a composite of Theorem 8.7 and Proposition 8.9; deliberately no single lemma). The keep-set commit GC runs off the same frontier read from above (§), and the run-table serializer (§8.5) supplies the save format.

The third datatype: rich text, with a firing marks GC. Peritext ships as a ported datatype (`runtime/src/datatypes/peritext.js`): the document-order mark model of §9 layered on the embed shadow, plugging into `DistributedReplica` over the same `init/apply/merge3/read` contract with no Peritext-specific runtime code, so rich text inherits delta gossip, content addressing, and commit GC by parametricity. Its state GC no longer refuses: `compactStable` fires under the same certificate through the marks keep-set (`compact-peritext.js`), the executable face of Theorems 9.9 and 9.11, with the delete-must-settle refinement, the conservative deviations, and the measured suite and cost numbers of §9; the flip that justified the old refusal stays executable as a negative control.

The production candidate replaces repeated absolute coordinates with immutable shared path nodes (`sharedEmbedRGA.js`). Its direct compactor consumes settled evidence, nonempty in-flight paths, and frozen mark anchors without materializing every absolute path. It skips guarded sibling renumbering and blocks fusion at guarded nodes. `compactibleSharedPeritext` uses the same kernel beneath Peritext’s retention roots and guarded mark-pair deletion. The 151-test runtime suite includes a never-compacted twin, returning pre-compaction peers, cross-epoch

randomized histories, retained dead mark boundaries, certified empty-document collection, and snapshot recovery. These are executable validation results, not Lean extraction or a proof of the JavaScript implementation.

Implementation boundary. The demo (`p2p-demo/`) runs several replicas over a WebSocket relay, persists and clones the document through git, and performs certified GC during a live session. The relay is a transport topology, not shared semantic state: each `DistributedReplica` holds its own commit DAG, fetches an ancestry delta, and records the peer’s advertised head as acknowledgement evidence. Cross-epoch merges translate returning states through the certified epoch maps; incomparable compacted cuts use the epoch-DAG transport rather than throwing. These paths have executable twin and randomized tests, but the JavaScript remains an unverified transliteration of the Lean model.

The remaining limits are concrete. The WebSocket deployment is a star relay, not a network-layer mesh. The SHA gate authenticates commit content, not author identity. Membership is cooperative and closed for each collection certificate. The Lean distributed refinement models fetch, head-sync, local GC, and acknowledgements, but it does not verify the JavaScript transport, serialization code, or cryptographic implementation.

23 Performance

Where does the shipped artifact sit against production CRDT libraries? The benchmark (`benchmarks/`) runs the runtime against Yjs 13.6, Automerge 3.3 (wasm), Loro 1.13 (wasm), and list-positions 2.0 on three workloads, and every cell is a run of the checked-in scripts on one machine (node v26.3.1, Apple M4 Pro, 24 GB); nothing is quoted from literature, and each system’s convergence and final text are gated before any number is reported. The corpus is the standard collaborative-editing replay set (the josephg traces: `automerge-paper` is Kleppmann’s LaTeX-editing trace, `seph-blog1` is Gentle’s blog draft, and `friendsforever/clownschool` are concurrent sessions, here also replayed single-author as the `_flat` twins). One honesty note up front: we benchmark *on* this corpus but not *against* Eg-walker’s own implementation (§10), which is an event- graph replay engine rather than a state library; that comparison is a genuine gap.

Per-metric methodology, stated natively. *Apply* is timed with a high-resolution clock around each single-character op, reported as median and p95 after a 1000-op warmup (JIT and wasm lazy init); timer overhead of 30 to 60 ns per op is *not* subtracted and biases the sub-microsecond medians upward. For ours each op includes the adapter’s position-to-id bookkeeping (an id-array splice) on top of the datatype `apply`. *Save size* is the bytes of each library’s native serialization, and what a save *contains* differs, so every row is labeled: Automerge’s `save` is the full change history (compressed; no state-only mode exists), Loro’s `snapshot` carries state plus history while `shallow-snapshot` drops history at the frontiers, Yjs’s update drops deleted content but not tombstone id structure (v2 run-length codes the same content), list-positions saves live characters plus position metadata as JSON, and ours saves live state only (delete is pure removal). Ours is reported in four labeled representations, discussed below. *Load* is from the primary save into a fresh document, median of five, including materializing the text once; for ours it includes re-deriving the display order (the coordinate sort). *Merge/sync* times the

bidirectional exchange per round in the concurrent workload. *Heap* numbers are flagged not comparable across the wasm boundary (Automerge and Loro hold state in wasm linear memory, invisible to the JS heap counter). Labels throughout distinguish *measured* from *measured-in-model*.

Apply latency: the interface-inherited loss. With a copied-Map interface, per-character apply lost to production by three to four orders of magnitude. The loss is *interface-inherited*: the persistent `apply` returned a fresh `Map`, an $O(\text{live set})$ copy per keystroke, linear in the document (order machinery contributes nothing measurable: minting a coordinate is $O(1)$ and inserts never compare), and a transient or batched apply path reaches the microsecond class without touching the verified semantics. The state container is therefore a persistent hash-array-mapped trie (`pmap.js`, dependency-free; `set` returns a new root sharing all but an $O(\log n)$ path, with transients for batched application and deterministic sorted iteration), the three-way merge is a delta merge from the LCA, and nothing else moves. The swap is licensed by `tableOf_congr` (the state is a function of the live-chain set, §8.3); the datatype interface, commit granularity, and wire format are unchanged, and every save byte, fingerprint, and content id is bit-identical (the full suites, 103 runtime plus 10 p2p tests including the marks-GC tests of §9.6, agree; the serializer gates and the Python projection cross-check, bit-for-bit). The measurement gives the following median apply times by trace and final size:

apply, median (final chars)	ff_flat (21k)	cs_flat (21k)	seph (57k)	am-paper (105k)
ours (absolute coordinates)	0.54 μs	0.38 μs	4.21 μs	4.92 μs
ours (shared paths)	0.54 μs	0.38 μs	4.08 μs	4.75 μs
ours, copied-Map interface	364 μs	347 μs	1.66 ms	3.16 ms
Yjs	1.5 μs	1.7 μs	1.4 μs	1.4 μs
Automerge	24 μs	26 μs	27 μs	27 μs
Loro	0.6 μs	0.8 μs	0.6 μs	0.5 μs
list-positions	0.6 μs	0.7 μs	1.0 μs	0.7 μs

That is 418 to 913 $\times$ on the medians (whole-trace replay on **automerge-paper**: 713s down to 1.94s), landing in the same decade as Yjs and Loro, with Automerge now the slowest applier in the table. The residual size trend in our row (0.4 to 4 μs) is not the datatype: instrumented separately, the datatype `apply` is a flat 0.49 μs mean on **automerge-paper**, and the growth is the benchmark adapter’s own $O(n)$ position-to-id bookkeeping (an id-array splice per op), outside the datatype. Allocation behavior improved with the copies gone: peak heap on **automerge-paper** fell from 433 to 132 MB, with retained heap at 4 to 22 MB (the trie retains structurally what the copies used to churn; absolute coordinates still share prefixes as cons strings, so the bit cost materializes only at serialization). The representation change primarily affects recovery and collection, not apply. Absolute-coordinate recovery takes 152 to 1158 ms across the four traces; shared-path recovery takes 16 to 111 ms. On **automerge-paper**, direct shared collection takes 170 ms, compared with 2612 ms for the absolute-coordinate compactor.

Merge and sync. The unique-LCA three-way merge under the head-sync runtime is Yjs-level and 17 to 59 $\times$ faster than the wasm libraries in these sessions:

system	sync median		wire payload/sync	
	freq	bulk	freq	bulk
ours (embed RGA, shipped)	107 μ s	754 μ s	in-process [†]	
ours (shared paths)	258 μ s	1.35 ms	5.9 KB	118.7 KB
Yjs	90 μ s	1.13 ms	1.6 KB	15.1 KB
Automerge	1.82 ms	18.3 ms	in-process	
Loro	6.29 ms	17.1 ms	0.4 KB	4.6 KB
list-positions	34 μ s	626 μ s	6.4 KB	123.6 KB

(**freq** = 60 rounds of 25 ops/replica, final 1804 chars; **bulk** = 6 rounds of 500 ops/replica, final 3604 chars; commit GC runs outside the timed window at millisecond totals.) Two qualifications apply. First, the HAMT swap shows up here as a mild regression at this scale: the copied-Map **freq** sync median was 73 μ s, and per-id lookups are now $O(\log n)$ trie walks rather than $O(1)$ Map hits (the same swap cut the local op in this workload from 26 to 2.4 μ s, so the trade buys three orders on typing for a fraction of one on small-doc sync). Second, these are small documents, and while the delta merge no longer *rebuilds* the live set in memory, it still scans $O(\text{live set})$ in time, so the numbers do not extrapolate to 100k-character documents. [†]The shared-store benchmark merges in-process, so the payload column is not applicable; the separate-store wire protocol (§22) ships a delta that is a function of that round’s ops, constant across rounds while a whole-state resync grows with the document, and the runtime’s sync tests measure it at 0.28 to 0.60 $\times$ the whole-state baseline in steady state.

Save size, two artifact classes. Our original runtime stores absolute chain coordinates as bit-strings, so the naive save is far above production; the run table (§8.3) is the designed live-read-only successor. The shared-path runtime instead saves a continuation-capable graph with birth provenance. These artifacts retain different information and must not be ranked as interchangeable state saves. The measured sizes, in bytes per live character, are:

representation (ours)	ff	cs	seph	am-p
as-shipped JSON	1637	2503	2991	2320
+ settled-cut compaction	871	1486	933	1295
run-table serialized, SHIPPED	1.5	1.6	1.6	1.2
+ compaction, SHIPPED	1.1	1.1	1.3	1.1
shared-path continuation snapshot	2.8	3.0	2.8	2.4
+ direct guarded GC	2.8	2.8	3.2	2.7
run-table projection (model)	3.8	3.8	3.9	4.0

In absolute terms the shipped, compacted run-table save is **24 / 22 / 73 / 120 KB** for the four traces, down from the as-shipped JSON’s **35 / 53 / 170 / 243 MB**: four orders of magnitude, lossless (`decode(encode(s))` reads identically, gated on every trace and by a 120-trial merge/delete property test). Its save-time cost is the honest column: the run-table encode takes 0.2 to 1.7s across the traces where production saves are milliseconds, pre-existing and unchanged by the apply fast path. The serializer is anchored to the shipped state, not a different datatype: its `accountingBits` routine reproduces the projection’s bit totals bit-for-bit, and the shipped bytes sit *below* the projection because the model deliberately charges the recoverable positional fields (per-record run id and offset, and the tail-attachment parent offset) that a real encoder drops. Against production, the shipped run-table save reaches rough parity with the smallest state-only saves:

system, variant	contains	ff	cs	seph	am-p
ours, run-table + compaction (SHIPPED)	state	1.1	1.1	1.3	1.1
ours, run-table projection	state (model)	3.8	3.8	3.9	4.0
Yjs update-v1	state + tombstone ids	3.8	4.4	6.0	3.0
Yjs update-v2	run-length coded	1.9	2.0	2.4	1.5
Automerger save	full history	1.3	1.2	3.6	1.2
Loro snapshot	state + history	2.9	3.1	5.9	2.4
Loro shallow-snapshot	state only	1.0	1.1	0.9	0.6
list-positions	state (JSON)	4.7	2.9	10.1	4.7

The shipped save is below Yjs update-v2 and Automerger’s compressed history and on par with Loro’s shallow-snapshot (0.6 to 1.1 B/char, still the smallest); history-carrying rows (Automerger always, Loro snapshot) are never compared against state-only rows without the label saying so.

Delete-heavy storage. The live-only Sal artifacts, Yjs updates, Loro’s shallow snapshot, and list-positions all shrink across the measured delete phases; full-history Automerger and Loro snapshots grow. The churn workload runs five cycles of insert 2000 then delete 1800 characters, then a final 200-character insert, and asks per row whether the save *grows across a delete phase* (bytes, ours in the packed binary estimate):

system, variant	c1-ins	c1-del	c5-ins	c5-del	final	grows?
ours (binary est.)	22278	2191	72322	25891	31596	never
ours, compacted (est.)	11576	1090	18884	6524	7882	never
Yjs update-v1	32931	31671	163331	161995	165395	never
Yjs update-v2	9539	8812	45776	44841	45856	never
Automerger save	8004	11387	56294	59724	60742	always
Loro snapshot	13349	15337	76144	78290	79679	always
Loro shallow-snapshot	6170	878	8677	3296	3882	never
list-positions	121933	94172	454518	428465	436314	never

The final document is 1200 live characters in every row. Automerger’s save grows by roughly 3.4 KB across every delete phase (history-carrying, monotone by design) and Loro’s full snapshot likewise; Yjs never grows but cannot shed tombstone structure (165 KB v1 for those 1200 characters against our compacted 7.9 KB and Loro’s shallow 3.9 KB). This is the storage-on-delete axis of the retention taxonomy (§11) measured directly. The result distinguishes history-carrying artifacts from reclaiming state artifacts; it does not support an “only system that shrinks” claim.

Measurement limitations. Timer overhead is not subtracted and biases the sub-microsecond medians; Automerger is driven one change per character (the automerger-perf convention; batching would lower its per-op cost and history size); list-positions is a positions library, not a full CRDT, and its sync row is its documented op-log integration with an unoptimized JSON payload; heap numbers are not comparable across the wasm boundary; and cross-system merged *order* may legitimately differ on concurrent insertions, so only intra-system convergence is gated.

24 Push-button discharge: an SMT translation-validation layer

This is a tool, not a theorem, and the section labels it as such. The eight verification conditions the flat capstone consumes (§, `flat_ra_linearizable3_eq`) are, for a flat instance, algebraic enough to hand to an SMT solver, and the `smt/` layer does exactly that: per instance it discharges the eight by `z3`, via unconditional laws that each *imply* the corresponding conditioned VC (the `DeltaVCs3` on-ramp (`feasibleDeltaVCs3_of_delta`, Definition) for the feasible laws, `cdVC3_of_all_comm` for the causal-delta bound: the `smt` bundle’s `all_comm` law is that lemma’s hypothesis). This is *translation validation*: the Lean metatheory stays the source of truth, and the trust gap is the fidelity of the hand-mirrored `z3` term to the Lean signature, closed by calibration rather than by a verified translator.

Calibration, exact against Lean. Four shapes span the space: Counter/PN (numeric group), GSet (set lattice), LWW (timestamp lex), and ORSet (instance-set with kill sets, non-commuting). The first four return UNSAT (VC valid) on all eight, matching their Lean discharges exactly. The OR-set is the informative one: six of eight are push-button, and the two that come back `sat` are precisely the two that are *configuration-conditioned in Lean* (its `feasible_local_redistribute`, Flat VC , which needs canonical-state tag-freshness, and `CDVC3`, Flat VC , which needs the maximal-remove trichotomy). Their unconditional overapproximation is genuinely false, so the solver’s `sat` is not a mismatch with Lean but a precise identification of the two VCs that required hand proof. Five deliberately broken mutants (drop the $\neg l$ in a counter merge, `max`-merge a PN, project a GSet merge, arbitrate LWW on the timestamp alone, drop the $\neg l$ guard in the OR-set) are each caught with a readable countermodel.

The remove-wins loop, and the normal form it forces. The success metric was a remove-wins set whose remove-records are garbage-collected against the LCA. Its literal specification comes back `sat` on both feasible-redistribute laws, and the loop’s finding is why: the LCA-conditioned supersession drop is *order-sensitive* (whether a remove is collected depends on adds introduced by the very delta being redistributed), so the merge is not a convergent semilattice and the delta laws fail unconditionally, with a countermodel exhibited. The repair the loop forces is a *normal form*: normalize each element’s remove-records to the per-element maximum timestamp (a remove superseded by a higher one is intrinsically absent), and likewise its adds; the merge becomes a product of two `max`-semilattices, LCA-blind, and all eight VCs then discharge push-button, zero hand proof, with the semantics preserved exactly (an element is present iff its maximum add strictly exceeds its maximum remove). Three further kill-tests (eager GC without the LCA guard; an add-wins tie-break; `min`-ing the remove timestamp) are each refuted by at least one genuine `sat` countermodel; a few individual kill queries hit the solver timeout instead of returning, but every killed variant carries `sat` verdicts independent of those (`smt/results/results.json`). The SMT loop therefore did not merely certify the remove-wins spec; it refuted the literal one and pushed the design to the max-timestamp normal form under which the whole bundle is push-button.

Recommendation. The right integration is certificate replay: use the SMT layer as a pre-flight oracle that tells a new flat instance’s author instantly which VCs hold and which need conditioning (the OR-set-style boundary) and produces the countermodel when a design is

broken, with a thin Lean tactic re-proving the UNSAT cells natively (most already close by `omega` or `cases <;> rfl`). The residual hand proof then shrinks to exactly the configuration-conditioned cells the triage flags — for the whole commuting class, and for the repaired remove-wins design, empty.

Part IV

Audit and mechanization scope

25 Mechanization scope

The model kernel lives under `Sal/MRDTs/RGA_Embed/`; conditioned instances, sequential specifications, recoding, compaction, and mark proofs live under `Sal/ConditionedMRDTs/MRDT_Instances/`. The executable system and reproducible measurements are in `runtime/`, `p2p-demo/`, and `benchmarks/`. The cited Lean chains use no `sorryAx`; negative claims are concrete checked witnesses. Open work is tracked in the repository roadmap rather than mixed into the paper’s claims.